An AI-Assisted Concept-to-Schematic Studio for Building-as-a-Product Design

Multi-agent reasoning, retrieval over building codes, physics-based site layout, deterministic hotel schematic generation and layout optimization

Naimul Haque
Semester project with RNGD

October 2026

Abstract. Early building design is slow because a customer's program, the site, the zoning and building codes and a manufacturer's product catalogue must be reconciled by hand before any schematic exists. This paper presents the RNGD AI Studio, a working prototype that automates that reconciliation and produces schematic designs. It has five parts:

1. A *multi-agent pipeline.* Language-model agents read an unstructured request and retrieved code text, while deterministic Python computes every quantity. A physics-style solver places the building and parking, computer vision re-measures the drawing, a critic decides whether to iterate, and one arbiter call makes the final decision.
2. A *parametric IFC4 generator*, scored against sample zoning and RNGD's real 2021 IBC rule set.
3. A *viewer for real IFC models.*
4. A *hotel schematic generator* for RNGD's first use case. It turns a detailed program (by default the Project Renegade Living + Lodging space allocation, 28 rows, 117 keys) into a podium-and-tower layout. It screens the layout against the IBC rules and the 2010 ADA guest-room tables, and draws an 18×12 in drawing set in RNGD's sheet format.
5. A *layout optimizer.* It replaces the genetic algorithm (GA) of RNGD's optimization prototype with an architectural decoder and a deterministic search, with no language model in the fitness. In the hotel mode it also generates the building shape, core position and floor count, and ranks the layouts (Pareto, TOPSIS).

We describe the methods with their mathematics and the most important code, show the inputs and outputs of every stage, and evaluate two questions: does the system respond to its inputs, and does it hallucinate?

- *Hotel generator.* Four different programs gave four different, correct buildings, and each rebuild was bit-for-bit identical.
- *LLM agents.* The intake agent extracted the request's values exactly (3/3 runs). The zoning agent recovered all 9 limits in the knowledge base (2/2 runs). Every number in the arbiter's narrative traces back to computed data.
- *Pipeline.* Across three full runs the decision was stable (REVISE), while the wording and severity of conflicts varied.
- *Layout optimizer.* On three sites with the same program and scoring, the reference GA placed 171 of 216 rooms, 24 guest rooms without a window. The updated GA placed 197 and the systematic search 192, both with 0 windowless, repeatably.

All 51 automated tests pass.

Prototype for a semester proof of concept. Pipeline, IFC-studio zoning and BaaP catalogue data are synthetic; the IBC rule set, ADA standards, real IFC file and hotel program are supplied by RNGD. Not for construction.

Contents

1 Introduction **3**

2 Inputs and data sources **3**

3 System architecture **4**

4 The multi-agent pipeline **4**
4.1 Blackboard and trace . . . 4
4.2 Run order . . . 5
4.3 Agents: inputs, outputs and the division of labour . . . 5
4.4 Mathematics computed by the pipeline . . . 5
4.5 Decision rules . . . 6
4.6 A real run, input to output . . . 6

5 Knowledge base and retrieval **8**

6 Physics engine **8**

7 Computer-vision QA and the critic **9**

8 IFC Design Studio **10**

9 Viewing a real IFC model **11**

10 Hotel schematic generation **12**
10.1 Program . . . 12
10.2 Guest floors . . . 13
10.3 Code and ADA screen . . . 14
10.4 From numbers to drawings . . . 14
10.5 The app: inputs, outputs and interaction . . . 15
10.6 The drawing set . . . 17

11 Layout optimization: from the reference GA to the updated optimizer **19**
11.1 Summary . . . 19
11.2 Architecture . . . 19
11.3 What was wrong, and how it was solved . . . 20
11.4 The mathematics . . . 21
11.5 Results on the three QA sites . . . 23
11.6 Results in Hotel Schematic (generative shape, core and floors) . . . 27
11.7 App screenshots . . . 30
11.8 What is still open . . . 33
11.9 Relation to known methods . . . 33

12 Implementation: the most important code **34**
12.1 Model access with a provider guard and JSON repair . . . 34
12.2 An agent prompt and its schema . . . 34
12.3 Arithmetic stays in Python . . . 35
12.4 Retrieval . . . 36

12.5 Physics engine . . . 36
12.6 Vision calibration and the critic loop . . . 37
12.7 Real IFC recentring . . . 38
12.8 Hotel generator . . . 38
12.9 Layout optimizer . . . 40

13 Evaluation: dynamic or hallucinating? 40
13.1 The hotel generator responds to inputs and repeats exactly . . . 40
13.2 The layout optimizer improves on the reference and repeats exactly . . . 42
13.3 The LLM agents extract what the documents say . . . 42
13.4 Run-to-run variability . . . 43
13.5 Tests and timings . . . 43

14 Discussion 43
14.1 Guardrails against hallucination . . . 43
14.2 Engineering lessons (bugs found and fixed) . . . 44
14.3 Positions on RNGD's kickoff questions . . . 44
14.4 Limitations . . . 44
14.5 Open questions for RNGD . . . 44

15 Conclusion and future work 45

A Glossary 46
A.1 Building, codes and real estate . . . 46
A.2 AI and software . . . 48

B Screenshot gallery 49

C Reproducibility 65

1 Introduction

RNGD designs buildings as products (*Building-as-a-Product*, BaaP): repeatable, prefabricated modules rather than one-off designs. The first step of every project, turning a customer's program into a schematic that fits the site and the codes, is still largely manual. RNGD's product requirements name hotels as the first use case. They ask for a schematic proposal with massing, site land use, room placement, circulation, prefabrication opportunities, assumptions and unmet requirements. The kickoff meeting added practical rules: circulation of about 33%, wall burden of 6–8%, area rounded up to the nearest 100 SF, 30–34 ft structural bays, and a preference for 2D output first.

This semester project built a working studio for that workflow. Its contributions are:

1. **A grounded multi-agent pipeline** (Section 4). Large language models (LLMs) read and judge, while all arithmetic is done in Python. The agents share a typed blackboard [17] and a full trace.
2. **A retrieval layer** (Section 5) over synthetic sample documents and RNGD's real documents: the 2021 IBC rule database [10, 22] and the 2010 ADA Standards [24].
3. **A physics-based site layout** with independent computer-vision verification and a critic loop (Sections 6 and 7).
4. **Parametric IFC4 generation and real-IFC viewing** (Sections 8 and 9) [9, 11].
5. **A deterministic hotel schematic generator** (Section 10). It takes a detailed program and produces a layout, an IBC and ADA screen, interactive 3D, and a drawing set modelled on RNGD's Living + Lodging set [21].
6. **A layout optimizer** (Section 11). It starts from the genetic algorithm of RNGD's design-optimization prototype, identifies why its plans leave rooms without windows and floor area unused, and replaces the representation and the search. It is compared with the reference on the same sites, and it also runs inside the hotel mode with a generated building shape, core and floor count.
7. **An evaluation** of determinism, sensitivity and hallucination (Section 13), and an explanation of the code (Section 12).

Construction and AI terms are defined in the glossary (Appendix A).

2 Inputs and data sources

Table 1: Inputs to the system. "Real" means supplied by RNGD or an official standard; "synthetic" means sample data written for the prototype.

Input	Kind	Used by
Customer request (Elm Street Multifamily, free text)	synthetic	intake agent
Site conditions JSON + site sketch PNG	synthetic	zoning agent, vision input
Sample zoning code, BaaP catalogue, stacking rules, design guidelines	synthetic	retrieval, BaaP matching, IFC studio
RNGD IBC 2021 R1 rule database (42 rules, CSV)	**real**	retrieval; IFC and hotel code screens
Companion workbook (config, occupant loads, hotel space mapping, tests)	**real**	retrieval, report appendices
2010 ADA Standards (PDF, 279 pages)	**real**	retrieval; ADA Tables 224.2/224.4 in the hotel screen
Living + Lodging space allocation (28 rows, 117 keys)	**real**	hotel default program
Real IFC model (CDI Solutions, Metairie Bank, IFC2X3, 89 MB)	**real**	IFC viewer
Kickoff meeting transcript	**real**	sizing rules, scope
Optimization prototype (`QA/`): 3 site grids, 72-room program, reference GA results	**real**	layout optimizer and its comparison
Review meeting transcript (10/08/2026)	**real**	layout rules for the optimizer

3 System architecture

Figure 1 shows the studio. One Streamlit [23] app hosts five modes that share a knowledge base and a chat assistant. The assistant answers using the live state of whichever mode is open.

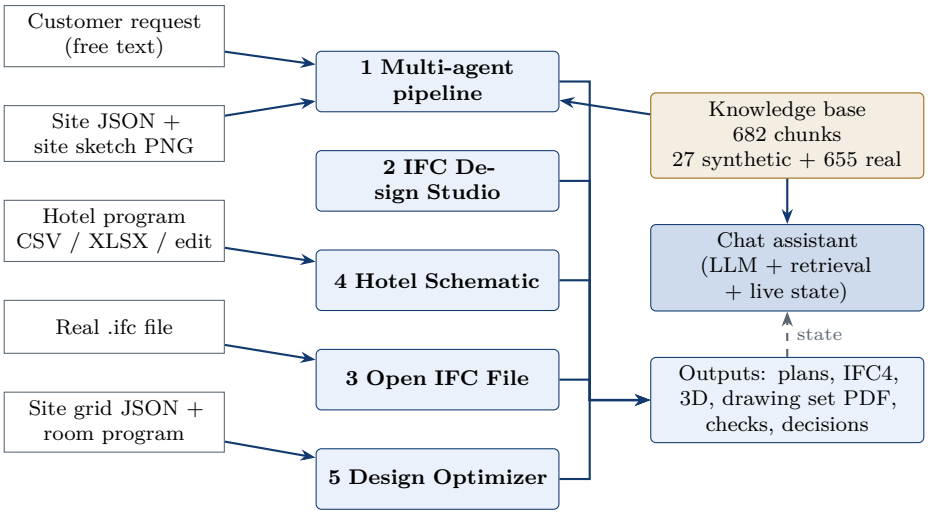


Figure 1: The five modes, the shared knowledge base and the assistant. The layout optimizer (Section 11) runs as its own mode on the prototype's site grids and as the "Optimize layouts" tab of Hotel Schematic.

Table 2: Technology stack.

Layer	Technology	Used for
Language, UI	Python 3.11, Streamlit 1.52	engines, agents, five-mode app, program editor
LLMs	OpenRouter [18]	light model `openai/gpt-oss-20b` (parsing), reasoning model `openai/gpt-oss-120b`, arbiter `anthropic/claude-sonnet-5`; Chinese-developed models blocked in code; 90 s timeout
Retrieval	Sentence-BERT [20], MiniLM [27]	384-d embeddings, cosine search (TF-IDF fallback)
State	pydantic	typed blackboard `ProjectState` with a full trace
Simulation, vision	NumPy [4], SciPy [26], scikit-image [25]	rigid-body relaxation; colour masks, connected components
BIM	IfcOpenShell [9]	write IFC4, read and triangulate IFC2X3
Optimization	NumPy, SciPy, `concurrent.futures`	decoder, ten-objective fitness, systematic search and memetic GA, parallel plate screening, Pareto/TOPSIS ranking; optimizer report prose by `anthropic/claude-sonnet-5.5` (outside the search)
Graphics	Plotly [19], Matplotlib [7]	interactive 3D and plans; vector drawing sheets
Documents	LaTeX (Tectonic), PyMuPDF	report PDFs and merging
Verification	pytest, Streamlit AppTest, Playwright	51 tests; browser run and screenshots

4 The multi-agent pipeline

4.1 Blackboard and trace

All agents read and write one typed object, `ProjectState`: the raw inputs, then `program`, `site`, `baap`, `conflicts`, the simulation, vision and critic histories, and finally `final_decision`. Every step runs inside `traced_call`, which records agent, action, model, status, latency and a one-line summary, so every run can be audited (Fig. 4b).

4.2 Run order

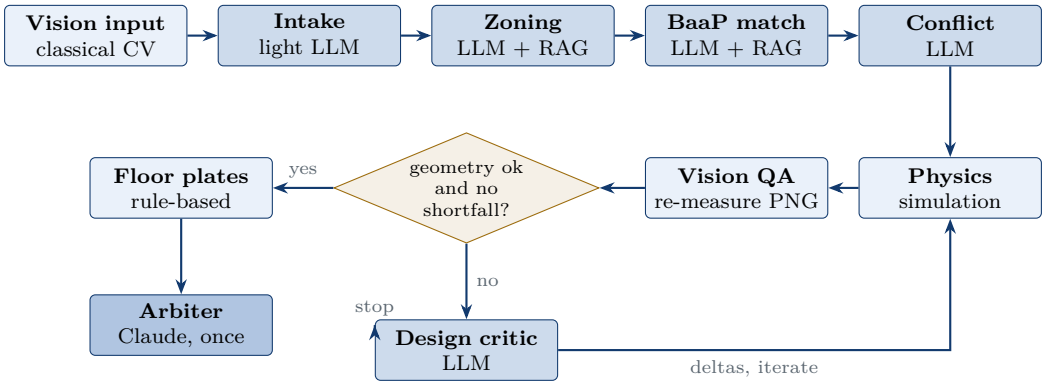


Figure 2: Pipeline run order. Shaded boxes call an LLM; the others are deterministic. The loop runs at most three times.

4.3 Agents: inputs, outputs and the division of labour

The central design rule is that *the model never does arithmetic*: it reads and judges, while Python computes.

Table 3: What each stage receives, what the LLM does, what Python does, and what is written to the blackboard.

Stage (model)	Input	LLM does	Python does	Output
Vision input (none)	sketch PNG + georeference	–	colour masks, components, pixel→ft, compare with stated lot	`cv_cross_check`
Intake (20b)	request text	extract units, mix, floors, amenities, parking, budget, hard/soft requirements	validate the JSON schema	`program`
Zoning (20b + RAG)	district, 6 retrieval queries	read code excerpts; return limits or `null`, with citations	**buildable envelope**	`site`
BaaP (120b + RAG)	program, stacking rules	adjacency plan, utilisation notes	**module counts, GSF, footprint, cores, accessible units**	`baap`
Conflict (120b + RAG)	program, site, BaaP, numbers	name conflicts, rate severity, recommend	**FAR, coverage, stalls, parking area, land left**	`conflicts`
Physics (none)	envelope, footprint, stalls	–	rigid-body relaxation	positions, penalty
Vision QA (none)	rendered PNG	–	re-measure footprint, coverage, violations, stall count	`cv_metrics`
Critic (120b)	CV metrics, convergence	adjust geometry or stop	clamp to 3 parameters; stop at max iterations	`critic`
Floor plates (none)	module counts	–	bay packing, double-loaded corridor	ground + typical floor
Arbiter (Claude)	whole blackboard	decision, confidence, narrative, issues, next steps	schema validation	`final_decision`

4.4 Mathematics computed by the pipeline

Buildable envelope. With lot frontage W_{lot} and depth D_{lot}, setbacks s_w, s_e, s_f, s_r and a non-buildable easement of width e on the east side:

$$x_{min} = s_w, \qquad x_{max} = \min(W_{lot} - s_e,\ W_{lot} - e), \qquad y_{min} = s_f, \qquad y_{max} = D_{lot} - s_r, \tag{1}$$

$$W_{env} = x_{max} - x_{min}, \quad D_{env} = y_{max} - y_{min}, \quad A_{env} = W_{env}\, D_{env}. \tag{2}$$

For Elm Street (200×300 ft lot, setbacks 10/10/15/20 ft and a 20 ft east easement), $W_{env} = 180 - 10 = 170$ ft, $D_{env} = 280 - 15 = 265$ ft and $A_{env} = 45{,}050$ SF.

Module counts and areas. For mix fractions p_t over T unit types and N units, the last type absorbs rounding:

$$n_t = \mathrm{round}(N p_t) \ \ (t < T), \qquad n_T = N - \sum_{t<T} n_t, \tag{3}$$

$$GSF_u = \sum_t n_t a_t, \qquad GSF_b = \frac{GSF_u}{\phi}, \qquad A_{fp} = \frac{GSF_b}{F}, \qquad c = \left\lceil \frac{N/F}{45} \right\rceil, \tag{4}$$

with module areas a_t from the catalogue, grossing factor $\phi = 0.85$, F floors and c cores. Elm Street gives modules 24/54/42, $GSF_b = 111{,}988\,\text{SF}$ and $A_{fp} = 27{,}997\,\text{SF}$.

Feasibility. These numbers are handed to the conflict agent, which is told not to recompute them:

$$GSF_{max} = A_{lot} \cdot FAR, \qquad A_{fp,max} = A_{lot}\,\frac{cov}{100}, \qquad S_{req} = \text{round}(r_p N), \qquad A_{park} = 330\,S_{req}, \tag{5}$$

$$A_{left} = A_{env} - A_{fp}, \qquad \text{surface parking fits} \iff A_{park} \le A_{left}. \tag{6}$$

Elm Street: $S_{req} = 1.25 \times 120 = 150$, so $A_{park} = 49{,}500\,\text{SF}$, but $A_{left} = 17{,}053\,\text{SF}$. Surface parking cannot fit; this is a computed fact that the LLM only names and rates.

4.5 Decision rules

The loop exit is code: stop when geometry_passed $\wedge$ shortfall $= 0$. Otherwise the critic proposes deltas to three parameters only, each clamped to $\ge 20\,\text{ft}$:

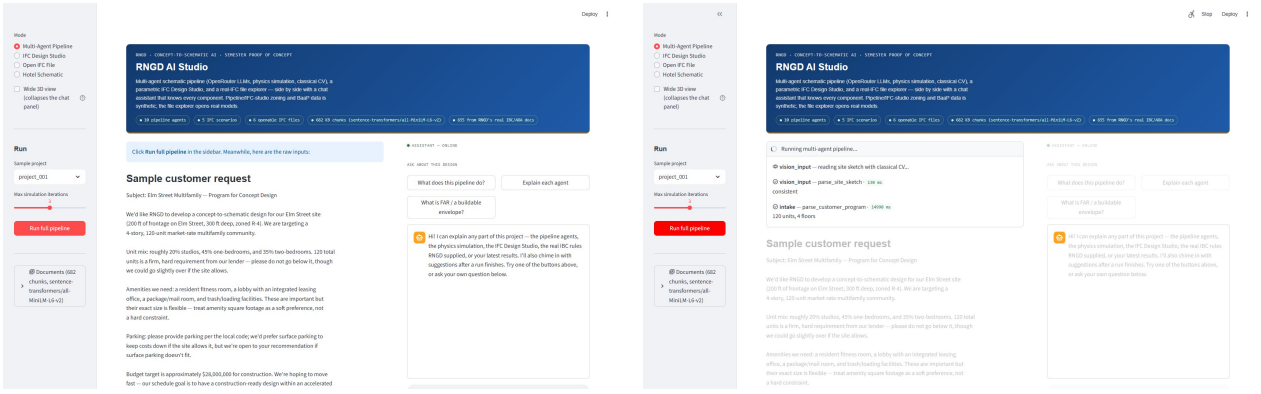
$$w \leftarrow \max(20,\, w + \Delta w), \qquad h \leftarrow \max(20,\, h + \Delta h), \qquad d \leftarrow \max(20,\, d + \Delta d), \tag{7}$$

or says to stop. Stopping is forced at the last iteration. The arbiter is instructed not to approve while a critical or high conflict is unresolved, and is told that `geometry_passed` says nothing about parking.

4.6 A real run, input to output

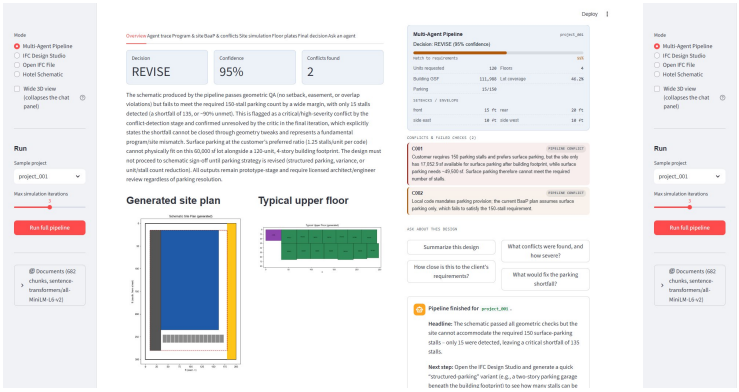
Table 4: Trace of a full run on 2026-10-05 (17:29).

#	Agent	Model	s	Result
1	vision_input	classical CV	0.4	sketch consistent with site conditions
2	intake	gpt-oss-20b	14.4	120 units, 4 floors
3	zoning	gpt-oss-20b	9.5	envelope 170 × 265 ft (45,050 SF)
4	baap_matching	gpt-oss-120b	20.8	120 modules, 111,988 GSF
5	conflict	gpt-oss-120b	4.6	1 conflict (parking, high); not feasible as requested
6	simulation	physics	0.0	6 steps, penalty 0.438, converged
7	vision_qa	classical CV	0.7	geometry OK; **stalls 15/150** (shortfall 135); coverage 46.3%
8	design_critic	gpt-oss-120b	4.7	stop: “cannot be resolved by geometry tweaks”
9–10	floor_plate	rule-based	1.7	typical 14 units placed (19 deferred); ground 11 (18 deferred)
11	arbiter	claude-sonnet-5	10.4	**REVISE** (confidence 0.93)

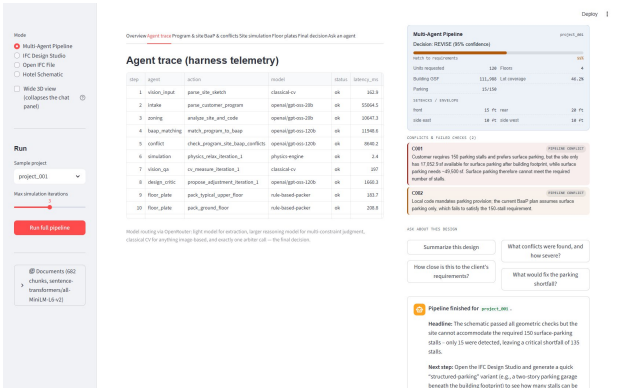


(a) Input: request and site sketch. **(b)** Running: live agent steps.

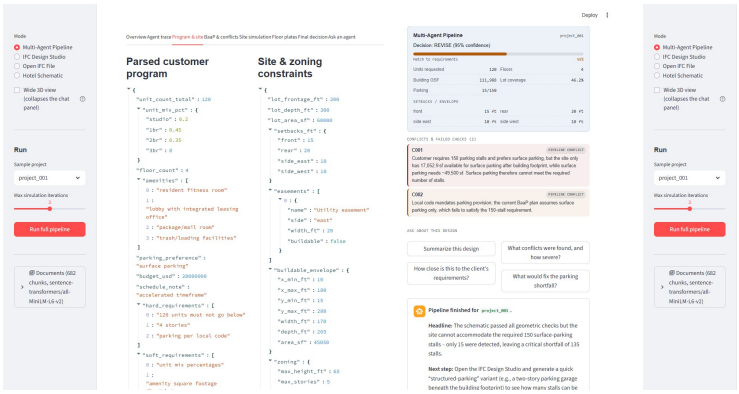
Figure 3: Pipeline input and live progress in the app.



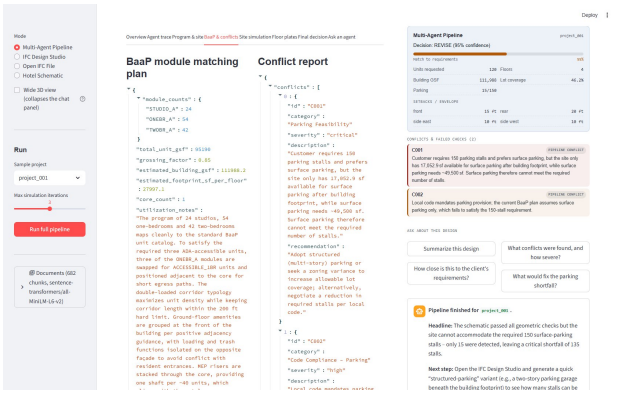
(a) Output overview: decision, confidence, conflicts.



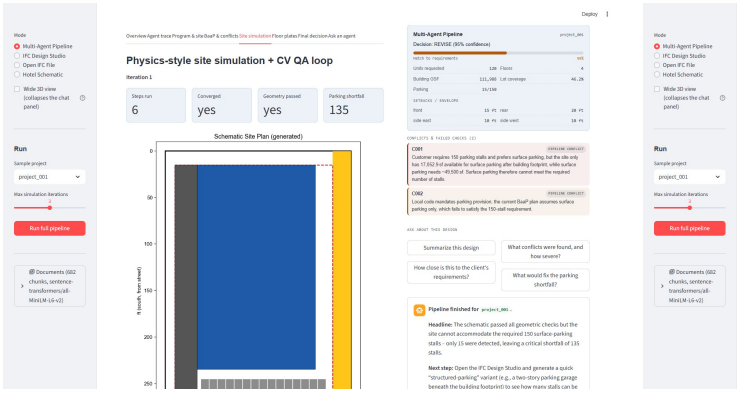
(b) Agent trace.



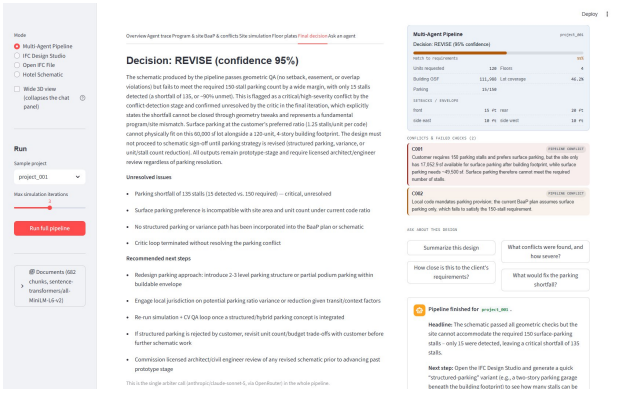
(c) Intake and zoning outputs.



(d) BaaP modules and conflicts.

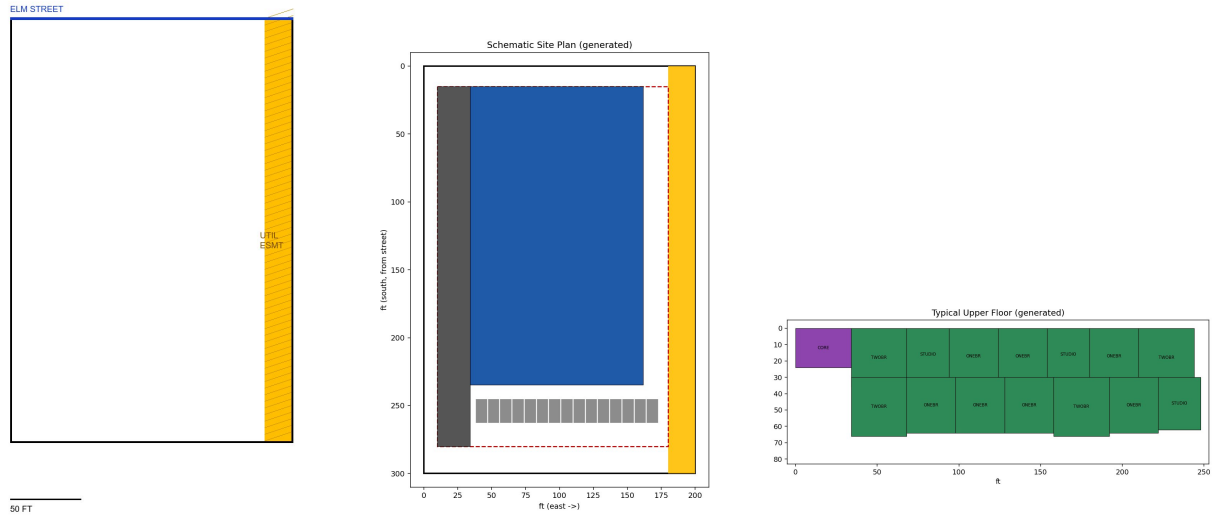


(e) Simulation and vision QA.



(f) Arbiter's final decision.

Figure 4: Pipeline outputs in the app (browser run, 2026-10-05 15:55). The screenshots and Table 4 come from different runs with the same inputs; Section 13.4 compares them.



(a) Input site sketch. **(b)** Simulated site plan. **(c)** Typical floor plate.

Figure 5: Pipeline geometry for the run in Table 4: from the sketch to the relaxed site plan (building blue, driveway grey, one row of 15 stalls, east easement yellow) and the packed floor plate.

5 Knowledge base and retrieval

Table 5: Knowledge-base contents. Every chunk is tagged `synthetic` or `real`.

Source	Kind	Chunks	Chunking
zoning_code_sample.md, baap_catalog.md, baap_stacking_rules.md, design_guidelines.md	synthetic	27	heading, the
RNGD IBC rule CSV	real	42	one per rule
RNGD IBC workbook	real	5	one per shee
2010 ADA Standards PDF	real	608	1,200-char w
Total		**682**	

Retrieval-augmented generation [12] grounds the agents in documents. Chunks c_i and query q are embedded with MiniLM [27] through Sentence-BERT [20] and normalised, so the score is the cosine similarity

$$s_i = \mathbf{e}_i^\top \mathbf{e}_q = \cos\theta_i, \qquad \|\mathbf{e}\|_2 = 1, \qquad \mathcal{R}_k(q) = \arg\operatorname*{top\text{-}k}_i \; s_i. \tag{8}$$

If the embedding model is unavailable, a TF-IDF backend with $s_i = \mathbf{t}_i^\top \mathbf{t}_q / (\|\mathbf{t}_i\|\|\mathbf{t}_q\|)$ is used instead.

Multi-query retrieval. One broad query on a multi-topic corpus is dominated by the best-matching section. The zoning agent therefore issues six narrow queries $q_1, \ldots, q_6$ and merges their results without duplicates:

$$\mathcal{C} = \bigcup_{j=1}^{6} \mathcal{R}_2(q_j) \quad \text{(deduplicated by source and heading)}. \tag{9}$$

This fixed a real failure where setbacks were not retrieved. As an example, the query "dead-end corridor maximum length sprinklered" returns IBC-R1-027 from RNGD's CSV first (score 0.689), the sample code's fire-access section second (0.540) and IBC-R1-026 third (0.493).

6 Physics engine

The site is a 2D rigid-body system. The building ($w \times h$, with $w = \min(0.75\, W_{env}, 150)$ and $h = A_{fp}/w$) is movable and starts at the front. The driveway ($24\,\text{ft} \times D_{env}$) and the $9 \times 18\,\text{ft}$ stalls are fixed.

Parking capacity is geometry. With 24 ft aisles (row pitch 42 ft):

$$k_{row} = \left\lfloor \frac{W_{env} - 24 - 4}{9} \right\rfloor, \qquad y_0 = y_{min} + h + 10, \qquad n_{rows} = \left|\{j \in \mathbb{N} : y_0 + 42j + 18 \le y_{max}\}\right|, \qquad S = \min(S_{req},\, k_{row}\, n_{row} \tag{10}$$

For Elm Street, $k_{row} = \lfloor 142/9 \rfloor = 15$ and $h = 27{,}997/127.5 = 219.6$ ft, so $y_0 = 244.6$ and only $j = 0$ fits ($262.6 \le 280$). That gives one row and $S = 15$, exactly the "15 of 150 stalls" in Table 4.

Forces. Containment pushes a body back inside the envelope:

$$F_x^c = \max(0, x_{min} - x_0) - \max(0, x_1 - x_{max}), \qquad F_y^c = \max(0, y_{min} - y_0) - \max(0, y_1 - y_{max}). \tag{11}$$

For overlapping bodies a, b with overlaps $o_x = \max\big(0, \min(a_{x1}, b_{x1}) - \max(a_{x0}, b_{x0})\big)$ (and similarly o_y), separation acts along the axis of least overlap, split half to each body:

$$\mathbf{F}_a^s = -\mathbf{F}_b^s = \tfrac{1}{2} \begin{cases} (\sigma_x o_x,\, 0) & o_x < o_y \\ (0,\, \sigma_y o_y) & \text{otherwise} \end{cases}, \qquad \sigma = \begin{cases} -1 & c_a < c_b \\ +1 & \text{otherwise.} \end{cases} \tag{12}$$

Integration and convergence. Semi-implicit Euler with damping $d = 0.55$ and gain $g = 0.5$:

$$\mathbf{v}_{t+1} = d\,(\mathbf{v}_t + g\,\mathbf{F}_t), \qquad \mathbf{x}_{t+1} = \mathbf{x}_t + \mathbf{v}_{t+1}, \qquad \Pi_t = \sum |F^c| + \sum |F^s|. \tag{13}$$

The run converges when $\Pi_t < 0.5$ ft and stops at 250 steps; positions are then snapped to a 1 ft grid.

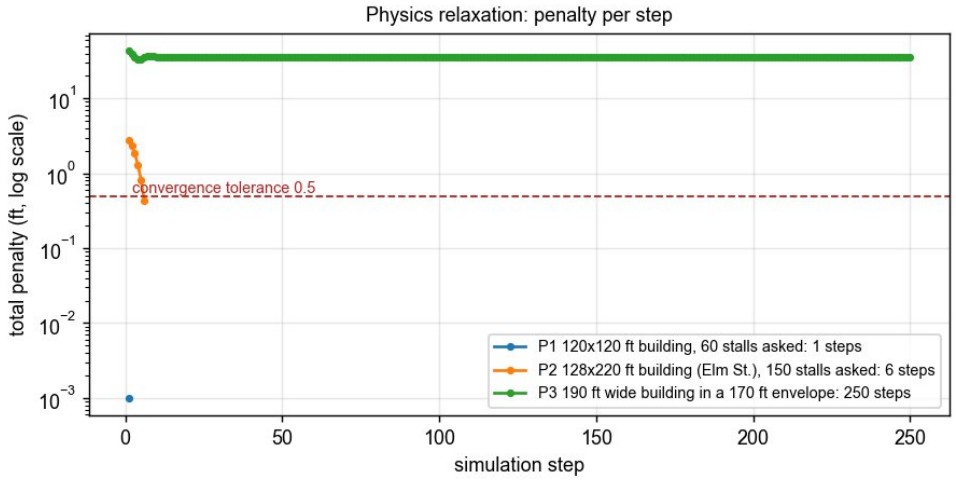


Figure 6: Penalty per step in three experiments. P1: a building that already fits converges in one step. P2: the Elm Street building converges in six steps. P3: a 190 ft-wide building in a 170 ft envelope can never satisfy containment, so the penalty plateaus at 36 ft. P3 is the case the critic must report as a program/site conflict.

The engine does not change the number of stalls, because the rows are fixed at set-up. The preference for the building at the front is set by the initial placement; no attraction force is applied.

7 Computer-vision QA and the critic

The QA agent re-measures the *rendered* PNG rather than trusting the simulation. The design decision (kept from the project's start) is classical CV, not a vision LLM: measuring a setback or counting stalls is a precise geometric question.

1. **Calibration.** From two reference points written by the renderer, $x_{ft} = x_{ft}^0 + (x_{px} - x_{px}^0)\, s_x$ with $s_x = \dfrac{x_{ft}^1 - x_{ft}^0}{x_{px}^1 - x_{px}^0}$ (same for y).
2. **Colour masks.** A pixel belongs to class k if $\sum_{c \in \{R,G,B\}} |I_c - \mathrm{ref}_c^k| \le 30$.

3. **Building.** The mask's bounding box gives A_{fp}; coverage is $100A_{fp}/A_{lot}$; a setback violation is recorded when the box is outside the envelope by more than 1 ft.
4. **Stalls.** Connected components of the parking mask, keeping components larger than 15% of the largest: $S_{det} = |\{k : |C_k| > 0.15 \max_j |C_j|\}|$, and shortfall $= \max(0, S_{req} - S_{det})$.

`geometry_passed` and `parking_shortfall` are separate fields, so a design can be geometrically clean and still fail on parking.

8 IFC Design Studio

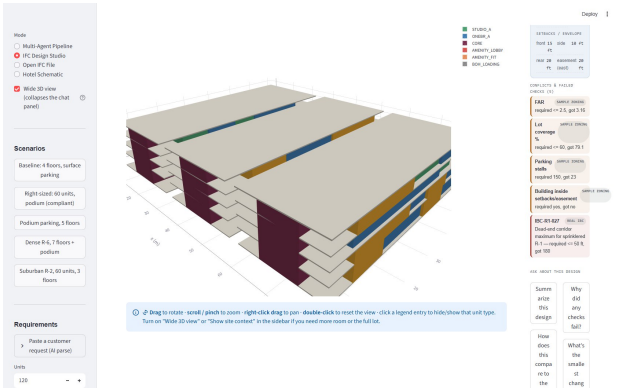
A deterministic generator produces a building from a parameter vector **p** (lot, easement, zoning preset, units and mix, floors, wings, parking type, corridor width, floor heights):

1. **Modules.** The mix is turned into module counts from the shared catalogue, including accessible units, core, lobby, fitness and BOH.
2. **Layout and selection.** Units are placed on double-loaded wings inside the setbacks and easement. The generator tries 1–3 wings and keeps the layout that scores best on: all units placed, inside the envelope, fire lane present, fewest wings.
3. **Parking.** Surface stalls come from a scan of free land; the podium option adds stalls under the building.
4. **Checks.** Ten sample-zoning checks, plus the measurable subset of RNGD's IBC rules (height, stories, exit travel, dead-end corridor, exits per story).
5. **IFC4 output.** IfcOpenShell writes project, site, building and one storey per level, with the volumes and slabs on each storey; the 3D view re-reads the file.

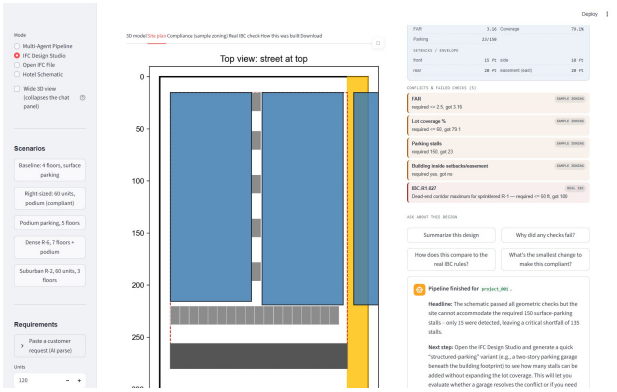
Table 6: The five scenarios (computed).

Scenario	Units	Stories	FAR	Cover.	Parking	Sample zoning	Real IBC fails
Baseline, 4 floors, surface	120/120	4	3.16	79.1%	23/150	**FAIL** (4)	R1-027
Right-sized, 60 units, podium	60/60	5	2.31	46.3%	75/75	**PASS**	R1-027
Podium, 5 floors	120/120	6	3.74	62.4%	120/150	**FAIL** (7)	R1-007, 008, 027
Dense R-6, 7 floors	120/120	8	4.01	50.2%	90/90	**FAIL** (3)	R1-005–008, 027
Suburban R-2, 3 floors	60/60	3	1.62	54.1%	10/120	**FAIL** (3)	R1-027

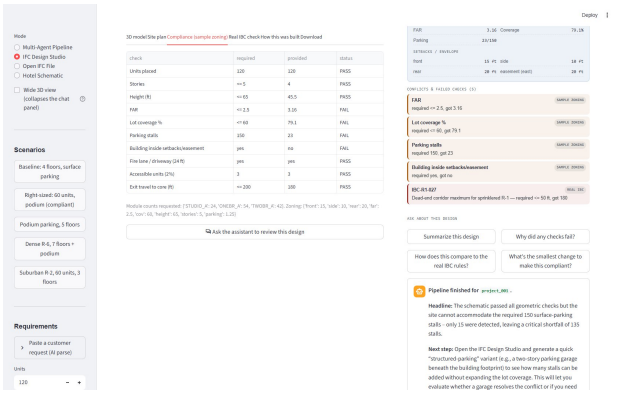
A real finding follows from the cross-check: even the scenario that passes all sample-zoning checks fails RNGD's 50 ft dead-end corridor rule (IBC-R1-027), because a single core leaves a long dead end.



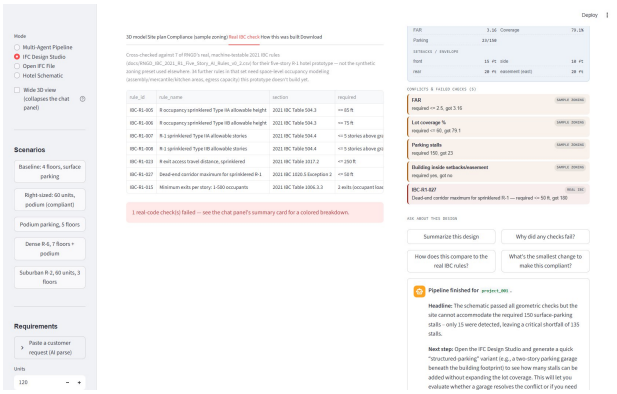
(a) Baseline IFC4 model in 3D.



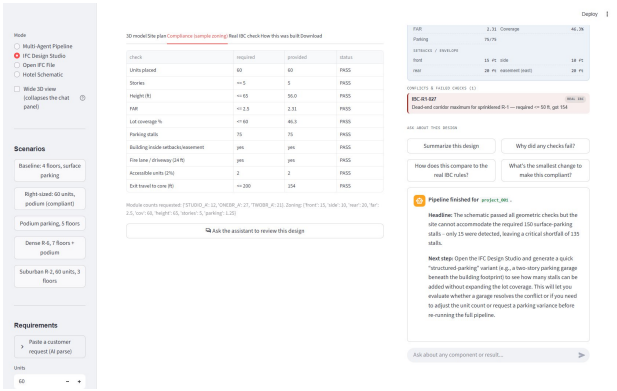
(b) Baseline site plan.



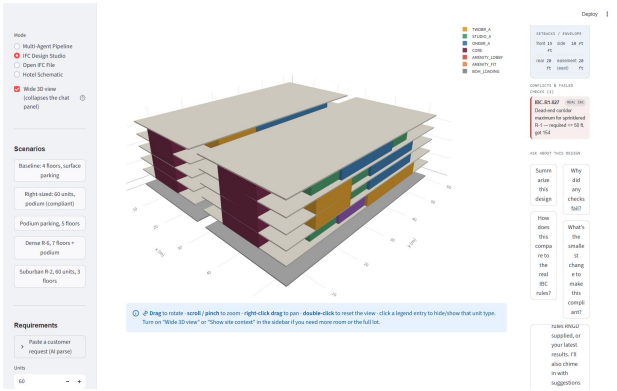
(c) Baseline: sample-zoning checks.



(d) Baseline: RNGD's real IBC rules.



(e) Right-sized: all sample checks pass.



(f) Right-sized model in 3D.

Figure 7: IFC Design Studio: input scenario to output model and checks.

9 Viewing a real IFC model

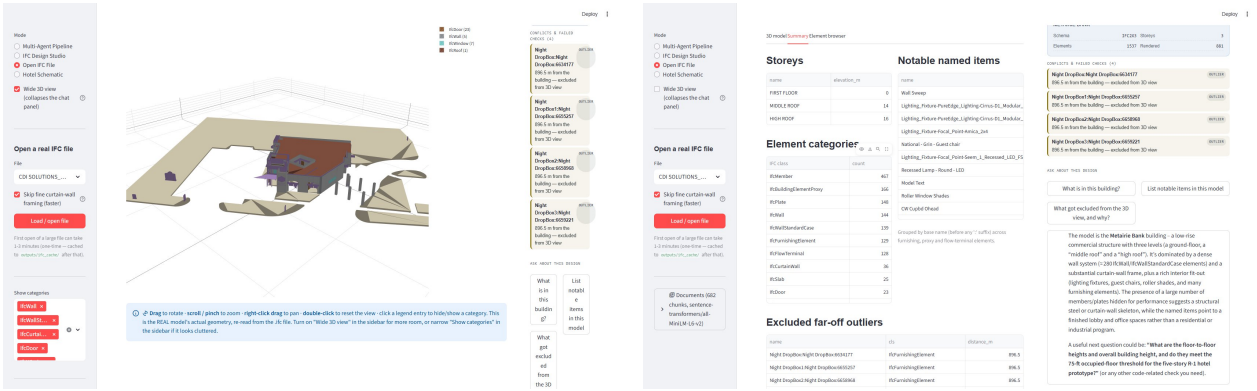
IfcOpenShell reads the file's header, spatial structure and element counts, and its geometry iterator triangulates every element. Meshes are merged per IFC class with NumPy and cached. Real files often contain stray elements far from the building, which shrink the view to a dot. The model is therefore recentred on the median of its structural elements, and outliers are excluded from the view (never from the file):

$$\mathbf{o} = \text{median}\{\mathbf{c}_e : e \in \text{structural}\}, \qquad \text{exclude } e \iff \|\mathbf{c}_e - \mathbf{o}\| > \max(4\,\delta,\, 50\,\text{m}), \tag{14}$$

with δ the building diagonal.

Table 7: The real model (CDI Solutions, Metairie Bank).

Schema / project	IFC2X3 / METAIRIE BANK
Storeys	FIRST FLOOR (0.0 m), MIDDLE ROOF (14.0 m), HIGH ROOF (16.0 m)
Elements / meshes	1,537 IfcElements / 881 triangulated meshes
Largest classes	IfcMember 467, IfcBuildingElementProxy 166, IfcPlate 148, IfcWall 144, IfcWallStandardCase 139
Building diagonal	134.8 m
Excluded outliers	4 “Night DropBox” items at 896.5 m



(a) Recentred 3D view. **(b)** Model summary.

Figure 8: Opening a real IFC file.

10 Hotel schematic generation

The hotel mode uses *no language model* to create the design. It is a deterministic function

$$\mathcal{D}: \text{(program, envelope, } F,\ h_1,\ h_t) \longmapsto \text{(levels, rooms, checks, drawings)}, \tag{15}$$

where F is the number of guest floors and h_1, h_t are the floor-to-floor heights.

10.1 Program

The default program is RNGD’s Living + Lodging space allocation [21], row for row: 117 keys in nine types and 59,350 SF net. Rows are *(name, count, total area, category)*. Programs can also be uploaded as CSV/XLSX or edited in the app (Fig. 11). Rows are routed as follows:

- guest rows become keys, each with area (row total)/(count);
- small repeated rooms (count ≥ 2, under 150 SF each: Ice, IT, Jan, Supplies) are placed once on every guest floor;
- small single BOH rooms go to the Level 1 core zone;
- tenant space becomes the retail block;
- stairs and corridors are generated, not packed;
- every other row goes to the Level 1 bands.

10.2 Guest floors

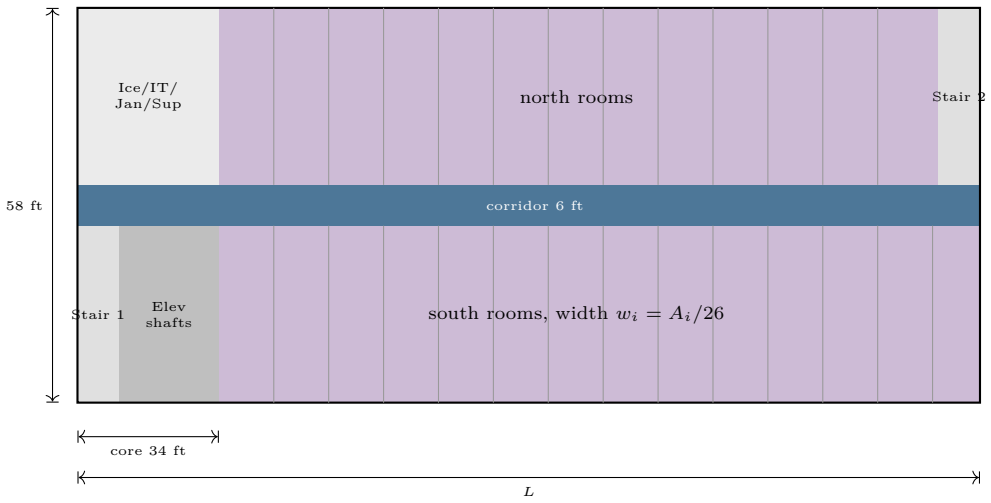


Figure 9: A guest floor: double-loaded corridor, west core (Stair 1, two elevators, shafts), per-floor support rooms, and Stair 2 at the east end, so the corridor has an exit at each end.

Room size. With room depth $d = 26\,\text{ft}$, $w_i = A_i/26$; a King is $343/26 = 13.2\,\text{ft}$ wide.

Keys to floors and sides. Keys are sorted by area (largest first) and assigned round-robin, so suites spread over the floors. On each floor, rooms are balanced greedily between the south and north sides; the north side starts 10 ft behind because of Stair 2.

Algorithm 1 Guest-floor layout

Require: keys $k_0, \ldots, k_{N-1}$ sorted by area (descending), guest floors F
for $k = 0$ **to** $N - 1$ **do**
  floor$(k) \leftarrow k \bmod F$
end for
for each floor f **do**
  $R_S \leftarrow 0,\ R_N \leftarrow 10$ ▷ Stair 2 occupies the north end
  for each key i on floor f **do**
    $s^* \leftarrow \arg\min_{s \in \{S,N\}} R_s$; place i at $x = 34 + R_{s^*}$ on side s^*; $R_{s^*} \leftarrow R_{s^*} + A_i/26$
  end for
end for
$L \leftarrow 34 + \max_f \max(R_S, R_N)$ ▷ all guest floors share the longest wing

Massing.

$$G_{level} = 58\,L, \qquad H = h_1 + F\,h_t, \qquad h_1 = 16\ \text{ft},\ h_t = 10.75\ \text{ft (from the L+L elevations)}. \tag{16}$$

Default program: $L = 217\,\text{ft}$ and $H = 16 + 5(10.75) = 69.75\,\text{ft}$.

Level 1. The core is stacked under the tower. The lobby goes first on the street band; the remaining rooms are sorted by size and go to the shorter band. The retail block is full depth at the east end and wraps Stair 2, with length $\ell = (A_{ten} + A_{stair})/58$. Default: bands end at 168.7 ft and retail adds 65.4 ft, so Level 1 is 234 ft long, inside the 265 ft envelope.

10.3 Code and ADA screen

Table 8: Hotel code screen: formulas and default results. Rules come from RNGD's CSV [22]; ADA tables from [24].

Rule	Test	Default
IBC-R1-007 stories	$F + 1 \le 5$ (Level 1 is a story above grade plane)	6: **FAIL**
IBC-R1-005 height	$H \le 85\,\text{ft}$	69.75: **PASS**
IBC-R1-010 area per story	$\max G_{level} \le 72{,}000\,\text{SF}$	13,575: **PASS**
IBC-R1-012 high-rise screen	$h_1 + (F-1)h_t \le 75\,\text{ft}$	59.0: **PASS**
IBC-R1-015 exits	2 stairs and $OL_{level} \le 500$	316: **PASS**
IBC-R1-018 exit separation	$L - 10 \ge \frac{1}{3}\sqrt{L^2 + 58^2}$	$207 \ge 75$: **PASS**
IBC-R1-023 travel	$T = \frac{L-10}{2} + 26 \le 250\,\text{ft}$	129.5: **PASS**
IBC-R1-025 corridor	$72 \ge 44\,\text{in}$	**PASS**
IBC-R1-027 dead end	$\max(5,\ L - (L-5)) = 5 \le 50\,\text{ft}$	**PASS**
IBC-R1-028 door	$34 \ge 32\,\text{in}$ clear	**PASS**
ADA 224.2	Table 224.2: 101–150 rooms need $5 + 2$ (roll-in) $= 7$	7: **PASS**
ADA 224.4	Table 224.4: 101–150 rooms need 12 communication rooms	not designated: **REVIEW**
ADA 224.5	accessible rooms dispersed over room types	3 types: **PASS**

Occupant load uses the CSV's IBC Table 1004.5 factors f_r (residential 200, business 150, storage 300, mercantile 60, exercise 50, kitchen 200, lobby 15 net):

$$OL_{level} = \sum_{r \in \text{level}} \left\lceil \frac{A_r}{f_r} \right\rceil . \tag{17}$$

Gross area is reported three ways. The source sheets give 74,195 SF and the drawn total is 76,508 SF. The meeting rule gives

$$G_{rule} = 100 \left\lceil \frac{N_{net}(1+c)(1+w_b)}{100} \right\rceil = 100 \left\lceil \frac{59{,}350 \times 1.33 \times 1.08}{100} \right\rceil = 85{,}300 \text{ SF}. \tag{18}$$

10.4 From numbers to drawings

Each room type is defined once in local coordinates (u, v): u along the corridor, v from the corridor ($v = 0$) to the window ($v = 26$). The definition includes walls with openings, door swings, bathroom fixtures, beds by type, closet, suite partition, spa tub and ADA turning circles (Fig. 10a). A south-side room at (x_r, y_r) of width w is placed by

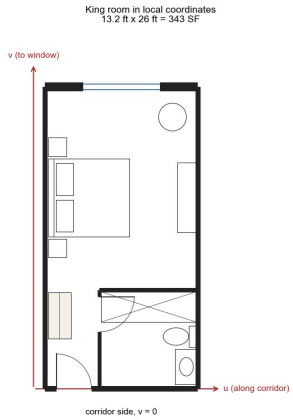
$$(x, y) = \big(x_r + (w - u \text{ if mirrored else } u),\ \ y_r + 26 - v\big), \tag{19}$$

with alternate rooms mirrored so bathrooms pair up (north side: $y = y_r + v$).

For the 3D views, the same primitives are extruded to boxes and drawn by an orthographic projector. With camera azimuth θ and elevation φ:

$$\mathbf{c} = (\cos\varphi\cos\theta,\ \cos\varphi\sin\theta,\ \sin\varphi), \quad \mathbf{r} = (-\sin\theta, \cos\theta, 0), \quad \mathbf{u} = \mathbf{c} \times \mathbf{r}, \quad (X, Y) = (\mathbf{p}\cdot\mathbf{r},\ \mathbf{p}\cdot\mathbf{u}). \tag{20}$$

A face with normal $\mathbf{n}$ is drawn only if $\mathbf{n} \cdot \mathbf{c} > 0$. Faces are drawn far to near by depth $\mathbf{p} \cdot \mathbf{c}$, the painter's algorithm [16]. Walls on the camera side are drawn last so they correctly hide the interior.



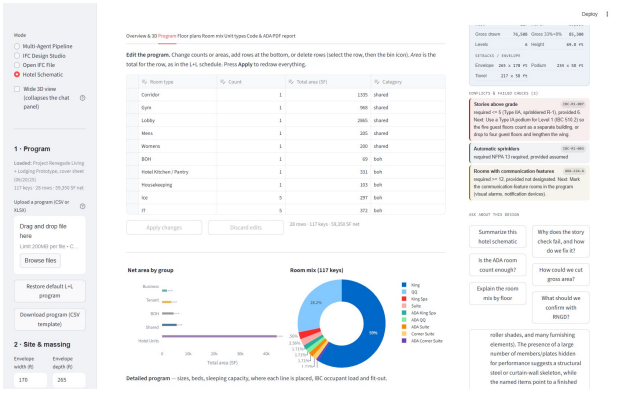
(a) King in (u, v).



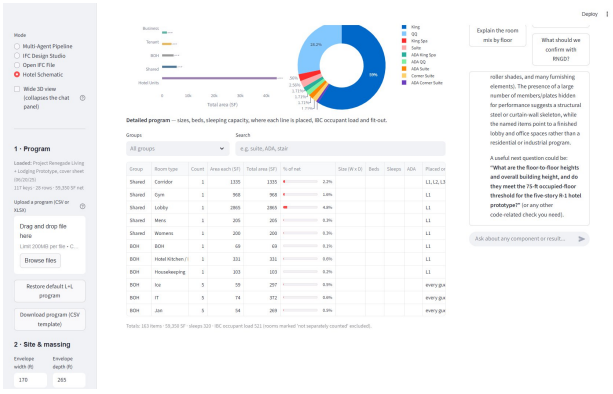
(b) Sheet A501: unit 3D views.

Figure 10: One local definition per room type drives every plan and every 3D view.

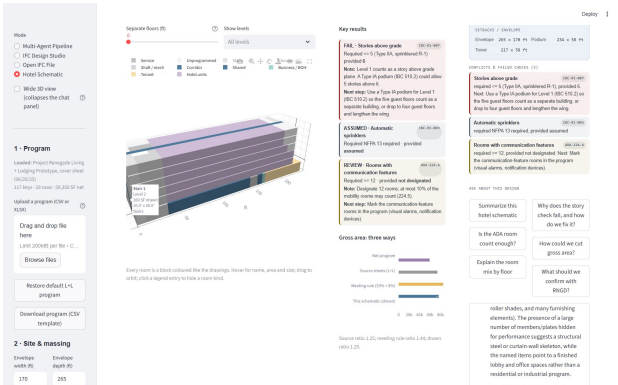
10.5 The app: inputs, outputs and interaction



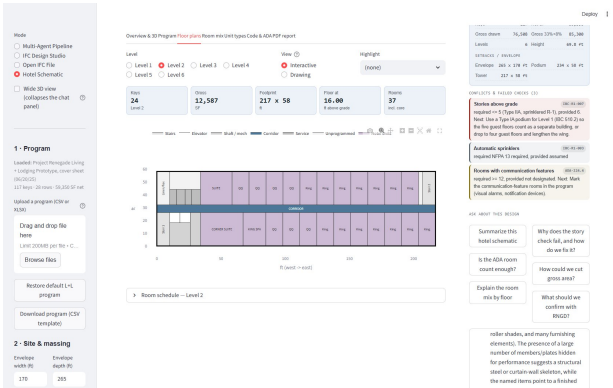
(a) Input: editable program.



(b) Detailed program.



(c) Output: KPIs, room-by-room 3D, fixes.

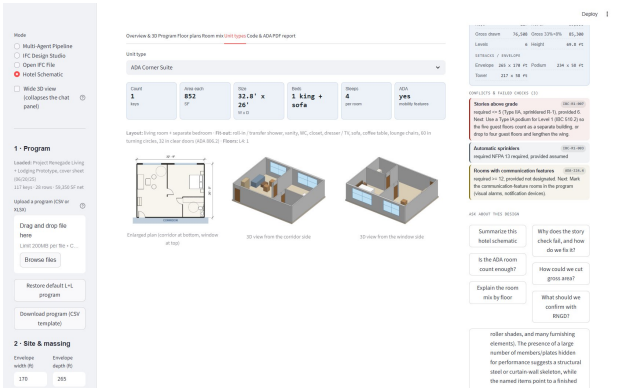


(d) Interactive Level 2 plan.

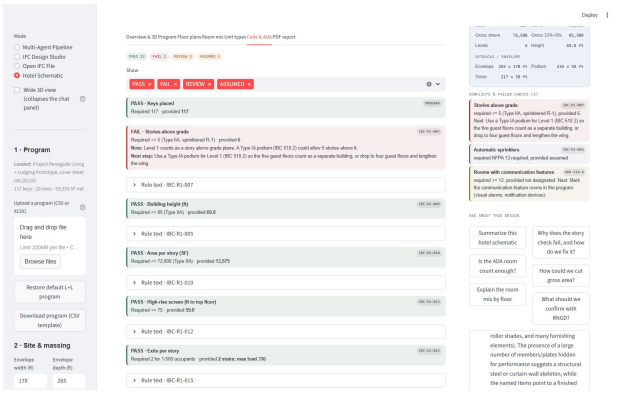
Figure 11: Hotel Schematic mode (1): program in, design out.



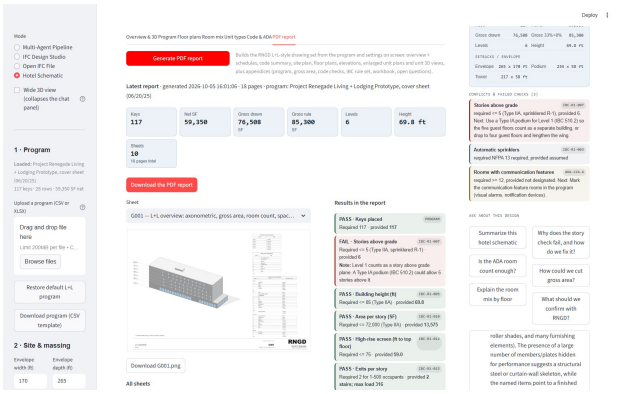
(a) Room mix by floor.



(b) Unit type: plan and 3D.



(c) Code and ADA screen.



(d) Generated report and its results.

Figure 12: Hotel Schematic mode (2): mix, units, code screen and report.

10.6 The drawing set

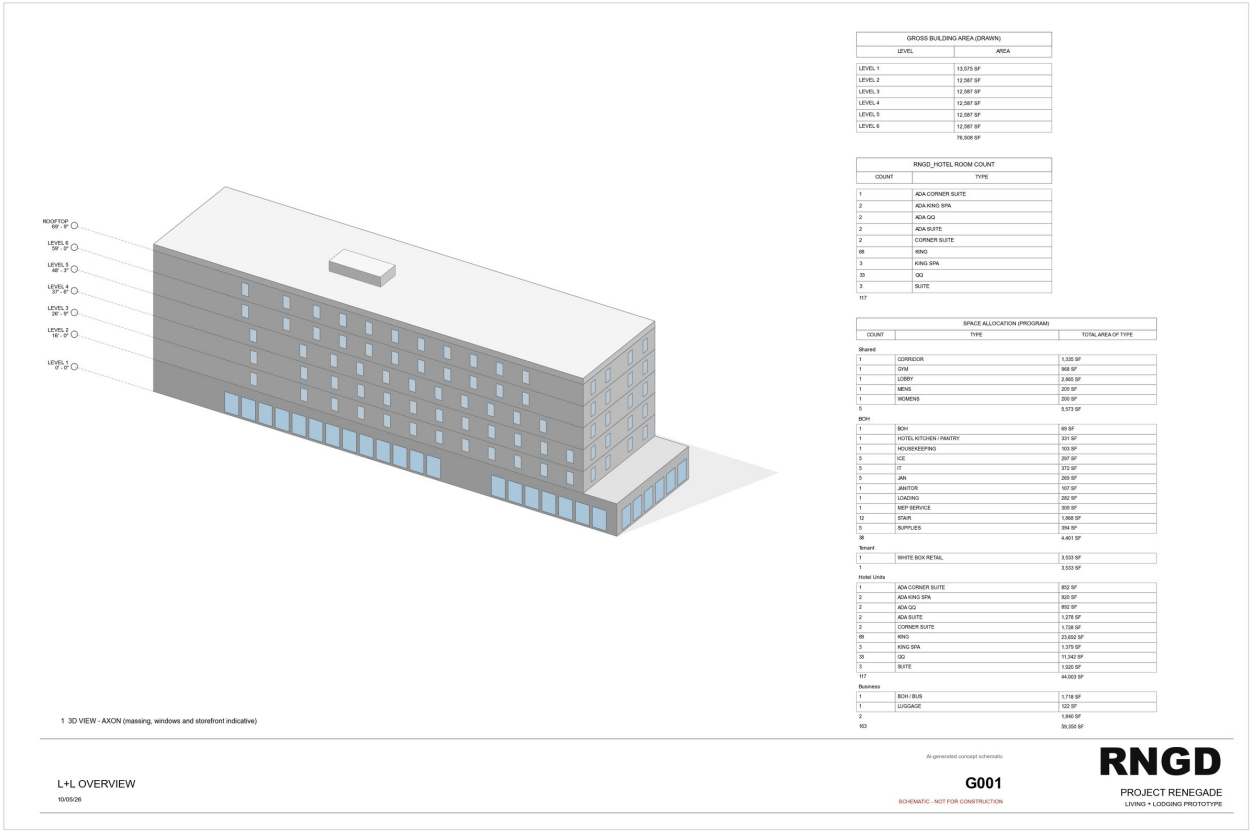


Figure 13: Sheet G001 in RNGD's L+L format: axonometric with level markers and the gross area, room count and space allocation schedules.

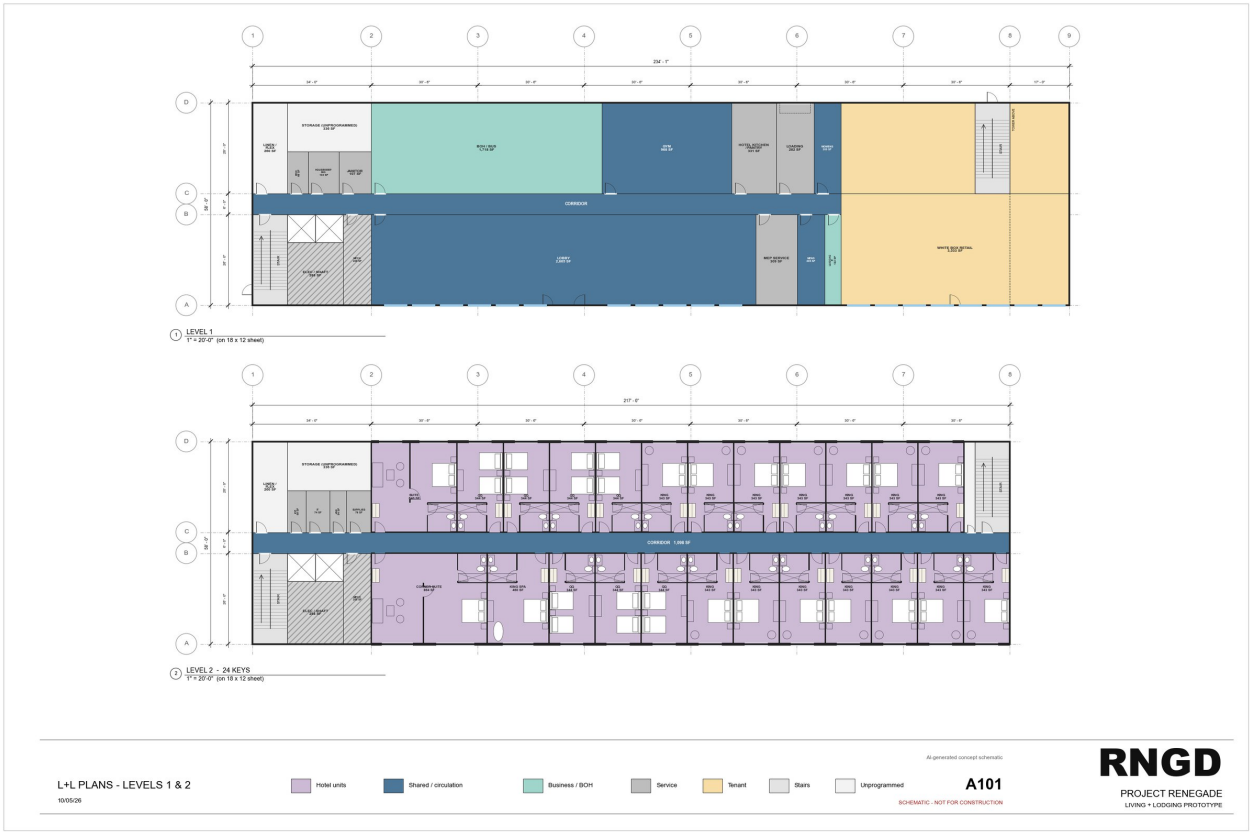
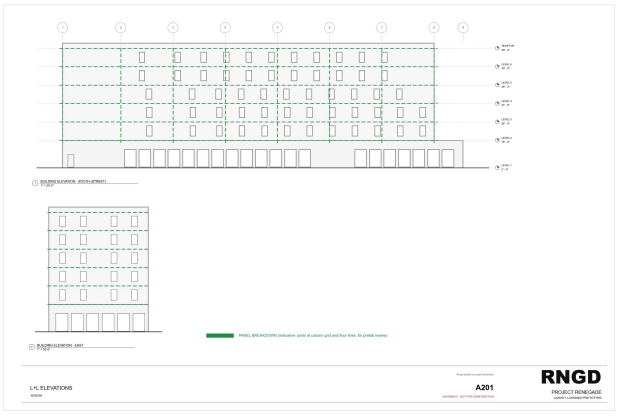


Figure 14: Sheet A101: Level 1 and Level 2 plans with column grid (bays of at most 32 ft), dimension strings, door swings and fixtures.



(a) A201 elevations with panel breakdown.



(b) A401 enlarged unit plans.

Figure 15: Elevations and enlarged plans.

11 Layout optimization: from the reference GA to the updated optimizer

11.1 Summary

RNGD's design-optimization prototype (Büşra's genetic algorithm, files in `QA/`) takes a multi-level 1 ft site grid and a room program and evolves a corridor and a room placement for 500 generations. This project rebuilt that optimizer in four steps and measured every step on the same three sites with the same program and the same evaluator (Table 9):

- **A, the reference GA** as delivered: 171 rooms placed over the three sites, 24 of the guest rooms without a window.
- **C, the updated GA**: a memetic GA on a new architectural decoder, no LLM in the fitness: 197 rooms, 0 without a window, mean fitness 0.928 against 0.863.
- **D, systematic search** (the default now): the same decoder, driven by enumeration and pattern search instead of crossover: 192 rooms, 0 without a window, mean fitness 0.922, in 59 s for all three sites and with the same answer on every run.

Per site (A → D): fp1: 61 → 72 rooms, 7 → 0 windowless, fitness 0.880 → 0.950; fp2: 53 → 60 rooms, 8 → 0 windowless, fitness 0.860 → 0.904; fp3: 57 → 60 rooms, 9 → 0 windowless, fitness 0.851 → 0.911. Variant B, the first attempt, kept the reference idea (a GA that moves free corridor segments and packs rooms) and placed 174 rooms in total with 14 windowless guest rooms in 336 s: better scoring alone did not fix the plans, which is why the representation was changed.

The same engine now also runs inside **Hotel Schematic**, where the shape of the building, the position of the core and stairs and the number of floors are generated instead of given (Section 11.6).

11.2 Architecture

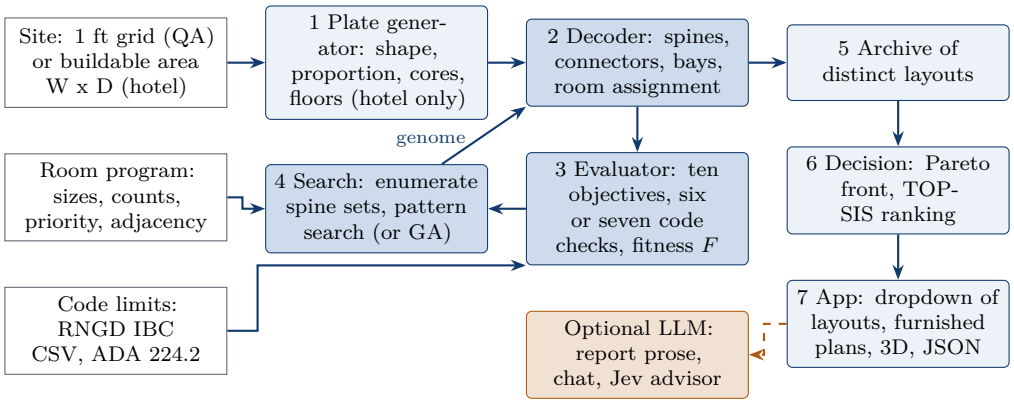


Figure 16: Architecture of the layout optimizer. The darker boxes form the optimization loop and are deterministic. The language model (orange) sits outside the loop: it only writes report prose, answers chat questions, or, if a TypeSafe key is configured, advises on the final shortlist.

#	Component	File	What it does
1	Plate generator	`optimize/typology.py`	Turns the hotel program into rooms with BaaP sizes; generates Bar, L, U, T, H and Courtyard plates inside the buildable area, with the elevator core and stairs placed by scheme (ends, centre, knuckle); one site grid per plate.
2	Decoder	`optimize/planner.py`	Genome → corridor spines → orthogonal connectors → modular bays → rooms assigned to bays. Every output is a valid plan.
3	Evaluator	`optimize/metrics.py`	Ten objectives in $[0, 1]$, the code checks (exits, exit separation, travel, corridor width, dead end, ADA; stories for the hotel) and the fitness F. Limits come from RNGD's IBC CSV.

#	Component	File	What it does
4	Search	`optimize/search.py`, `explore.py`	Systematic enumeration of spine sets plus pattern search (default) or the memetic GA; for the hotel, all plates are screened in parallel processes and the best of each shape refined.
5	Archive	`optimize/explore.py`	Keeps every distinct layout with its objectives and criteria.
6	Decision	`optimize/decide.py`	Pareto front and TOPSIS ranking over twelve criteria; optional Jev advisor (TypeSafe) on the shortlist only.
7	Views	`optimize/plan_view.py`, `hotel_opt_ui.py`, `optimizer_ui.py`	Dropdown of the top or all layouts, furnished interactive plans, 3D massing, comparison charts, JSON export.
–	Reference loader	`optimize/baseline.py`	Rebuilds the reference designs from `QA/floorplan*.json` so they are scored by the same evaluator (their shared-wall lengths reproduce the values stored in the files).
–	Report writer	`optimize/report.py`	One LLM call (`anthropic/claude-sonnet-5.5` through OpenRouter) writes the prose of the per-run PDF from the computed data and retrieved code passages; every number in its text is traced back to the data.

Only the last row uses a language model. Steps 1 to 6 are plain Python on NumPy, SciPy and scikit-image arrays, so the same inputs always give the same layouts and the same ranking.

11.3 What was wrong, and how it was solved

The problems below were seen in the reference results, in the first version of this app's optimizer (which followed the same idea and added an LLM reviewer to the fitness) and in the team review of 10/08/2026.

Observed problem	Cause	What was changed	Evidence in this run
Guest rooms with no window; corridors loaded on one side only (review in the 10/08 meeting).	The corridor genome places segments anywhere; rooms are packed wherever they touch a corridor, so the distance to the facade is left to chance.	Decoder: corridor spines are chosen from lines scored for double-loaded window rooms (score $s(k)$); guest rooms may only take bays with a window (hard rule).	Windowless guest rooms over the three sites: 24 (A) $\rightarrow$ 0 (D).
Floor area left unused; rooms rotated or isolated.	Rooms are placed one by one as free rectangles, so leftovers between rooms cannot be used.	Modular bays of one room frontage M along the whole corridor; rooms take whole bays, the last bay absorbs the remainder; unused bays are reported as flex space.	Unused share and corridor share per variant in Table 9.
Not clear whether the requested keys can fit at all.	No bound was computed; the GA simply reports what it placed.	Area and window upper bounds, shown next to every result.	Table 10.
Crossover produces plans that do not make sense; slow convergence.	Half of one plan spliced onto half of another: the value of a corridor depends on the other corridors.	Search over a small structured space: enumerate spine sets, then first-improvement pattern search. The GA is kept as an option and now runs on the same decoder.	At the same budget D is within 0.010 fitness of the GA (C) with no crossover and no population: 192 against 197 rooms. Most of the gain over A comes from the decoder, not from the search.
With crossing corridors, facade strips stayed empty (seen on the square site fp2 in the first version of the new decoder: 39 rooms).	Bays were built corridor by corridor, so the windowless interior side of the first corridor took the strip that the next corridor's window rooms needed.	Two passes over all corridors: window bays first, windowless bays in what is left.	fp2: 60 rooms (D) and 62 (C), all guest rooms with a window.
Corridor stubs beyond the last room; dead ends over the 50 ft limit.	Spines were cut to the bays they could serve, not to the rooms actually placed.	After the assignment each spine is cut back to the doors and junctions it serves; the dead end is measured to the nearest exit door or through-route.	Longest dead end, D: 46 ft; A: 41 ft (same measure).

Observed problem	Cause	What was changed	Evidence in this run
Runs take minutes; results change between runs.	500 generations in the reference; an LLM reviewer inside the fitness in the earlier version of this app (about 10 minutes, cloud calls).	No LLM in the fitness; deterministic seeds; cached static scores in the assignment; plates evaluated in parallel processes.	Variant D: 59 s for the three sites; identical result on a repeat run: yes.
Egress and accessibility not part of the score.	The reference score rewards placed rooms and contiguity.	Ten objectives including exit travel, dead ends, ADA Table 224.2 count and dispersion, core access; limits read from RNGD's IBC CSV.	All variants pass the checks on these sites; the checks are now reported for every layout.
The shape, the core and the floor count are inputs.	The grid fixes the L-shaped plate and both cores.	Hotel Schematic: plates (six shapes), core schemes (three) and the floor count are generated and searched; a dropdown lists the top or all layouts.	740 distinct layouts in 36 s (Section 11.6).

11.4 The mathematics

Problem. A site is a stack of L levels of 1 ft cells. On each level a set Ω of cells is *placeable*; cores (stairs, elevators) and structure are fixed in the QA sites and generated for the hotel. The program is a list of room instances $i = 1..N$, each with a type $t(i)$, nominal frontage f_i and depth d_i (area $\bar{a}_i = f_i d_i$), a priority p_i, a set of allowed levels and flags (needs a window, wants a corner, wants to be near a core). A layout chooses a corridor $C \subset \Omega$ (the same on every level, so floors stack) and an axis-aligned rectangle for some of the rooms, such that rectangles do not overlap each other or C, lie inside Ω, and each room has a door on C. The goal is to maximise the fitness F below.

Decoder: from a short genome to a full layout. The search never moves rooms directly. It works on a genome $g \in [0,1]^{44+N}$ (ten candidate spines with four genes each: two trims, a shift, an on/off gene; four connector genes; one order key per room) and the decoder builds the layout:

1. **Module.** The bay width is the weighted median guest-room frontage, $M = \text{median}_w\{f_t\}$ with weights equal to the room counts; D is the median guest-room depth and w the corridor width.
2. **Candidate spines.** Every row and column k that can hold a straight corridor band gets the score

$$s(k) = \sum_{x \in \text{band}(k)} \big[\, g_a(x) + g_b(x) + 1.5\, n_a(x) + 1.5\, n_b(x) + 0.5\, r_a(x) + 0.5\, r_b(x) \,\big],$$

   where, for the two sides a and b of the band at position x, $g = 1$ if a room at least $0.92\, d_{\min}$ deep fits, $n = 1$ if in addition the facade is about one room depth away (a window room), and $r = 1$ if it is within the deepest room's depth. So a line that is double-loaded with window rooms on both sides scores $2 \times (1 + 1.5 + 0.5) = 6$ per foot, a single-loaded line 3, and a line through deep space with no facade 2. The best non-overlapping lines (at least $d_{\min} + w$ apart) become the ten candidates.
3. **Connectors.** Active spines are trimmed to their last room and joined to each other and to every core with orthogonal L-shaped paths (never diagonal); pieces that stay unconnected are dropped.
4. **Bays.** Each side of each straight corridor run of length ℓ is cut into $n = \lfloor \ell / M \rfloor$ bays. The remainder $\rho = \ell - nM$ is absorbed by the last bay if $\rho < 0.4M$ and becomes a narrow bay otherwise. A bay has a window if its back wall is on the facade. Bays are built in two passes over all corridors, window bays first and windowless bays in what is left, so the interior side of one corridor never takes the facade strip that another corridor's window rooms need.
5. **Assignment.** Rooms are taken in the order of $0.06\, p_i + g_i$ (priority, then the order key) and each takes

$$k_i = \max\Big(\text{round}\, \frac{\bar{a}_i}{MD},\ \Big\lceil \frac{0.85 f_i}{M} \Big\rceil \Big)$$

   consecutive free bays on one allowed level, the run with the highest score

$$\sigma = W_{\text{ext}}\,[\text{window}] - W_{\text{nw}}\,[\text{no window}] + W_{\text{corner}}\,[\text{corner}] + W_{\text{core}}\Big(1 - \tfrac{\min(\delta,150)}{150}\Big) + W_{\text{adj}}\, A$$
$$+ W_{\text{stack}}\, \tfrac{\min(S,2)}{2} + W_{\text{grp}}\,[\text{same type next door}] - 1.2\max(0, o - 1.2),$$

   with δ the corridor distance from the door to the nearest core, A the adjacency credit, S the number of floors with the same room type in the same bay, and o the ratio of the bay area to the room's nominal area. Two hard rules apply: a guest room is only placed where every bay has a window, and only if the run gives at least 85% of its nominal frontage and depth.

After the assignment, each spine is cut back to what it serves (the doors of placed rooms and its junctions with other corridors and cores), which removes corridor stubs and shortens dead ends. The dead end E is the longest corridor distance from any corridor cell to the nearest exit door or through-route between two exits.

Every decoded layout is therefore valid by construction (straight corridors, rooms perpendicular to them, no overlaps, all rooms on the corridor), and the search only has to decide *which* corridors exist and *in what order* rooms claim bays.

Fitness. Ten objectives $f_k \in [0,1]$ (1 is best) are combined with weights w_k:

$$F = \frac{\sum_k w_k f_k}{\sum_k w_k} \;-\; \tfrac{1}{2}\max\big(0,\ \min(n_c, 2) - c\big),$$

where n_c is the number of cores and c the number the corridor reaches (an exit that is not connected costs half the scale). With ρ the corridor share of the plate, T the longest exit travel and E the longest dead end:

$$f_{\text{place}} = \frac{\sum_{i\ \text{placed}} p_i \min(a_i, \bar{a}_i)}{\sum_i p_i \bar{a}_i}, \qquad f_{\text{day}} = \frac{\#\{\text{rooms needing a window that have one}\}}{\#\{\text{rooms needing a window}\}},$$

$$f_{\text{eff}} = \text{clip}\Big(1 - \frac{\rho}{0.30}\Big), \qquad f_{\text{egress}} = 1 - \tfrac{1}{2}\min\Big(1, \frac{(T-250)^+}{250}\Big) - \tfrac{1}{2}\min\Big(1, \frac{(E-50)^+}{50}\Big),$$

$$f_{\text{ada}} = \min\Big(1, \frac{n_{\text{ADA}}}{n_{224.2}}\Big) \cdot \min\Big(1, \frac{\text{floors with ADA}}{\text{target}}\Big), \qquad f_{\text{core}} = \text{mean}_i\ \Big(1 - \frac{\min(\delta_i, 120)}{120}\Big),$$

$$f_{\text{adj}} = \frac{\sum_j \omega_j \max\big(0,\ 1 - (\Delta_j - m_j)^+/m_j\big)}{\sum_j \omega_j}, \qquad f_{\text{corner}} = \frac{\#\{\text{corner rooms on a corner}\}}{\#\{\text{corner rooms}\}},$$

$$f_{\text{bal}} = \text{clip}\Big(1 - \frac{\text{sd}(K_\ell)}{\text{mean}(K_\ell)}\Big), \qquad f_{\text{comp}} = \text{clip}\Big(\frac{2S_{\text{shared}}}{0.6\,P_{\text{rooms}}}\Big).$$

Here a_i is the drawn area (credit is capped at the nominal area, so an oversized room is not worth more), Δ_j the distance between the two rooms of adjacency j with allowed distance m_j, K_ℓ the keys on guest floor ℓ, S_{shared} the total length of walls shared by two rooms and P_{rooms} the total room perimeter. The limits 250 ft (exit travel) and 50 ft (dead end) are read from RNGD's IBC rule CSV. The weights are placement 0.3, daylight 0.14, efficiency 0.1, egress 0.12, ada 0.08, core access 0.07, adjacency 0.07, corner 0.03, balance 0.05, compactness 0.04. There is no language model in F: it is arithmetic on the geometry.

Upper bounds (can the keys fit at all?). Two bounds hold for any layout, whatever the algorithm. A corridor of width w serves at most two room depths, so at least $w/(2D+w)$ of the used floor is corridor:

$$K \le K_{\text{area}} = \Big\lfloor \frac{L\,|\Omega|\,\big(1 - \frac{w}{2D+w}\big)}{\bar{a}} \Big\rfloor.$$

If every guest room needs a window, the rooms of one floor share the facade length P_f:

$$K \le K_{\text{window}} = L\Big\lfloor \frac{P_f}{\bar{f}} \Big\rfloor.$$

For a generated hotel plate with corridor length R and cores taking a frontage c_f, the bound per floor is $\lfloor (2R - c_f)/M \rfloor$. The bounds are loose (they ignore corners and the public rooms), but a request above them is impossible, and a result close to them is close to the best any search can do.

Search. With m candidate spines the decoder is driven by a subset of them, so the first stage enumerates

$$1 + \sum_{k=1}^{4} \binom{\min(m,7)}{k} \le 1 + 7 + 21 + 35 + 35 = 99$$

spine sets (all spines, and every set of up to four of the seven best), with rooms in priority order. The second stage starts from the three best distinct layouts and repeats: generate the neighbours (switch one spine on or off: 10 moves; trim either end or shift an active spine: 4 moves each; new connector elbows; four room-order nudges), evaluate them in a seeded random order, and accept the first one with $F' > F + 10^{-6}$. When all three starts are local optima, the rest of the budget is spent on iterated local search: the best layout is "kicked" with two moves at once, improved again by the same pattern search, and kept only if it beats the best so

far. The total budget is PG evaluations (P = population, G = generations, kept as the budget controls), the same as the GA's. The genetic algorithm (variant C) evaluates $P(G+1)$ layouts plus local-search steps, here about 804. Crossover is not used in the systematic search: splicing the corridor genes of one plan with the room order of another seldom gives a better plan, because the value of a corridor depends on which other corridors exist.

Plate generation (hotel). For K keys on F guest floors the busiest level needs net area A; with slack $\varsigma = 1.04$, core area A_c and packing efficiency $\eta = 0.93$ the corridor length to provide is

$$R = \frac{\varsigma A + A_c}{2D\eta}.$$

Each shape with bounding box $a \times b$ and wing depth $W_d = 2D + w$ (58 ft) offers $R_{\mathrm{Bar}} = a$, $R_{\mathrm{L}} = R_{\mathrm{T}} = a + b - W_d$, $R_{\mathrm{U}} = a + 2b - 2W_d$, $R_{\mathrm{H}} = a + 2b - 2W_d$ and $R_{\mathrm{Courtyard}} = 2a + 2b - 4W_d$; the generator solves $R_{\mathrm{shape}}(a, \beta a) \geq R$ for the smallest a at aspect ratios $\beta \in \{0.6, 1\}$ and clips the result to the buildable area in both orientations. The elevator count is $e = \max(2, \lceil K/75 \rceil)$, and the two stairs of the "centre core" scheme are the pair of wing-end positions with the largest distance between them (exit separation, short dead ends).

Decision. Each layout j has a vector of twelve criteria $x_{j1..12}$ (keys, gross SF per key, facade per key, height, corridor share, dead end, code checks passed and others). Layout j is *dominated* if another layout is at least as good on every criterion and strictly better on one; the rest form the Pareto front. TOPSIS then ranks all layouts:

$$r_{jq} = \frac{x_{jq}}{\sqrt{\sum_j x_{jq}^2}}, \quad v_{jq} = \frac{\lambda_q}{\sum_q \lambda_q} r_{jq}, \quad d_j^{\pm} = \sqrt{\sum_q (v_{jq} - v_q^{\pm})^2}, \quad C_j = \frac{d_j^-}{d_j^+ + d_j^-} \in [0, 1],$$

where v_q^+ is the best and v_q^- the worst value of criterion q over all layouts and λ_q the weight from the chosen preset (Balanced, Most keys, Lowest cost, Guest experience, Low-rise) or the sliders. The layout with the largest C_j is ranked first.

11.5 Results on the three QA sites

All variants use `QA/rooms_hotel.json` (72 rooms on three levels) and are scored by the same evaluator. Variants B, C and D were run with population 24 and 30 generations as the budget, seed 7, no LLM. For A the search time is not stored in the files; its history holds 500 generations.

Table 9: Reference GA and the three later variants. "No window" counts guest rooms without a facade wall; "Unused" is the share of placeable area that is neither room nor corridor; "Checks" are the code checks passed.

Site	Variant	Rooms	No window	Fitness	Corridor	Unused	Dead end ft	Checks	Evaluated	Time
fp1	A Reference GA	61/72	7	0.880	0.151	0.16	33	6/6	500 gen.	n/a
	B GA, free-form	53/72	3	0.880	0.100	0.29	55	5/6	804	119 s
	C Updated GA	72/72	0	0.954	0.101	0.09	27	6/6	804	24 s
	D Systematic	72/72	0	0.950	0.101	0.09	25	6/6	695	20 s
fp2	A Reference GA	53/72	8	0.860	0.117	0.26	31	6/6	500 gen.	n/a
	B GA, free-form	60/72	8	0.893	0.111	0.19	36	6/6	804	102 s
	C Updated GA	62/72	0	0.908	0.146	0.06	43	6/6	804	22 s
	D Systematic	60/72	0	0.904	0.145	0.09	46	6/6	637	17 s
fp3	A Reference GA	57/72	9	0.851	0.157	0.18	41	6/6	500 gen.	n/a
	B GA, free-form	61/72	3	0.894	0.120	0.18	33	6/6	804	115 s
	C Updated GA	63/72	0	0.921	0.122	0.11	35	6/6	804	26 s
	D Systematic	60/72	0	0.911	0.119	0.09	29	6/6	683	21 s

Table 10: Upper bounds on the number of keys (Section 11.4) against what was asked and placed.

Site	Placeable SF / level	Program SF	Program / place-able	Facade ft / level	Mean key frontage ft	K_{area}	K_{window}	Keys asked	Keys A	Keys D
fp1	9,640	23,310	0.81	489	12.9	80	111	69	58	69
fp2	9,446	23,310	0.82	410	12.9	78	93	69	50	57
fp3	9,412	23,310	0.83	519	12.9	78	120	69	54	57

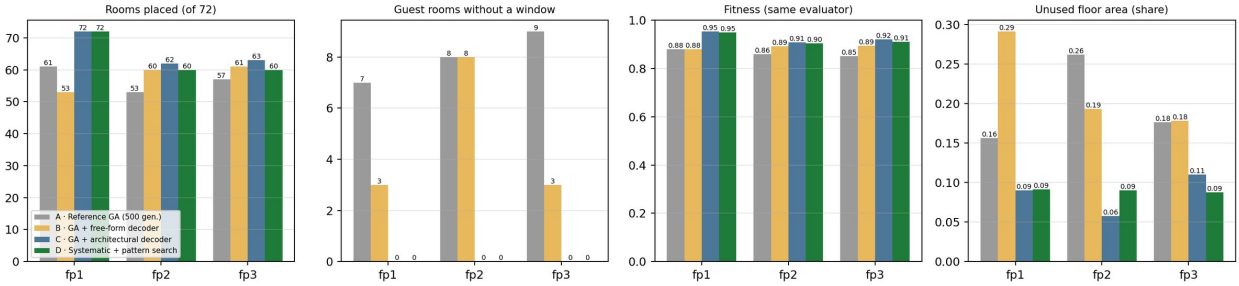


Figure 17: The four variants on the three sites (same program, same evaluator).

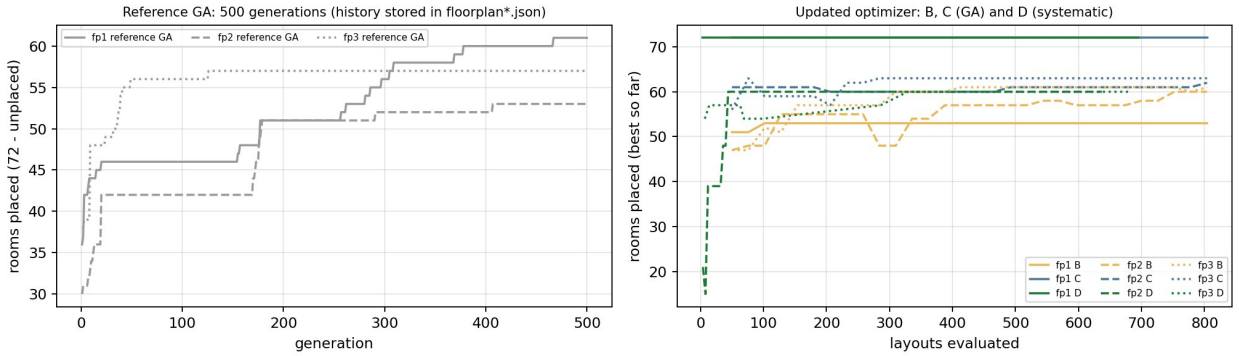


Figure 18: Left: rooms placed by the reference GA over its generations (from the history in the delivered files). Right: best rooms placed against layouts evaluated for the later variants.



Figure 19: Site fp1, Level 2, same program and same evaluator. Blue lines are window walls; a red dashed outline marks a guest room with no window; hatched bays are unused.



Figure 20: Site fp2, Level 2, same program and same evaluator. Blue lines are window walls; a red dashed outline marks a guest room with no window; hatched bays are unused.



Figure 21: Site fp3, Level 2, same program and same evaluator. Blue lines are window walls; a red dashed outline marks a guest room with no window; hatched bays are unused.

11.6 Results in Hotel Schematic (generative shape, core and floors)

Input: the Project Renegade Living + Lodging program (117 keys) and a buildable area of 170 x 265 ft. The search screened 158 plate and floor-count variants, evaluated 1,140 layouts and kept 740 distinct ones in 36 s on eight processes, with no LLM calls; a second run returned the same list: yes. 212 layouts are on the Pareto front. Pattern search improved the screened plates as follows: Courtyard 156x156 · center core (5 fl.) 0.870 → 0.870; L 202x121 · ends core (5 fl.) 0.863 → 0.881; Bar 265x58 · center core (5 fl.) 0.863 → 0.863; T 202x121 · center core (5 fl.) 0.856 → 0.865.

Table 11: Top ten layouts under the Balanced decision weights.

#	Layout	Guest floors	TOPSIS C_j	Pareto	Keys	Fitness	Corridor	SF / key	Height ft	Dead end ft	Code
1	Bar 265x58 · center core	4	0.917	yes	117	0.859	0.112	657	59	24	1.00
2	L 220x132 · knuckle core	4	0.906	yes	117	0.849	0.120	729	59	26	1.00
3	L 170x176 · knuckle core	4	0.905	yes	117	0.835	0.120	714	59	24	1.00
4	Bar 265x58 · ends core	4	0.904	yes	117	0.845	0.113	657	59	30	1.00
5	Bar 265x58 · ends core	4	0.903	yes	117	0.843	0.113	657	59	30	1.00
6	Bar 265x58 · ends core	4	0.903	yes	117	0.842	0.113	657	59	31	1.00
7	Courtyard 156x156 · ends core	4	0.902	yes	117	0.845	0.111	972	59	28	1.00
8	T 170x176 · knuckle core	4	0.899	no	114	0.840	0.122	733	59	24	1.00
9	T 170x176 · knuckle core	4	0.899	no	114	0.840	0.122	733	59	24	1.00
10	Bar 265x58 · center core	5	0.897	yes	117	0.863	0.112	788	70	24	0.86

Table 12: Best layout of each shape (Balanced weights).

Shape	Best layout of that shape	Guest floors	Overall rank	Keys	SF / key	Facade ft / key	Dead end ft	Code
Bar	Bar 265x58 · center core	4	1	117	657	27.6	24	1.00
L	L 220x132 · knuckle core	4	2	117	729	30.1	26	1.00
Courtyard	Courtyard 156x156 · ends core	4	7	117	972	33.5	28	1.00
T	T 170x176 · knuckle core	4	8	114	733	30.4	24	1.00
U	U 187x112 · center core	4	34	111	771	31.8	29	1.00

Shape	Best layout of that shape	Guest floors	Overall rank	Keys	SF / key	Facade ft / key	Dead end ft	Code
H	H 156x136 · knuckle core	4	206	106	854	34.9	59	0.86

Table 13: The first-ranked layout changes with the decision priority: the trade-off is the user's, the search only supplies the options.

Decision priority	Layout ranked first	Guest floors	Keys	SF / key	Height ft	Code
Balanced	Bar 265x58 · center core	4	117	657	59	1.00
Most keys	Courtyard 156x156 · center core	4	117	972	59	1.00
Lowest cost	Bar 265x58 · center core	4	117	657	59	1.00
Guest experience	Bar 265x58 · center core	4	117	657	59	1.00
Low-rise	Bar 265x58 · center core	4	117	657	59	1.00

First-ranked layout: Bar 265x58 · center core, 4 guest floors. Footprint 265 x 58 ft (15,370 SF), 2 elevators, bay module 13 ft. It places 117 of 117 keys and 142 of 150 program rooms; the frontage bound is 34 keys per typical floor. Corridor share 0.112, longest exit travel 64 ft, dead end 24 ft, fitness 0.859. Not placed: White Box Retail x1, IT x1, Jan x2, Gym x1, MEP Service x1, Loading x1, Luggage x1.

Rule	Check	Required	Provided	Status
IBC-R1-015	Exits per story (cores reached)	≥ 2	3	**PASS**
IBC-R1-018	Exit separation (core to core)	$\geq$ 1/3 diagonal = 90 ft	202 ft	**PASS**
IBC-R1-023	Exit access travel	$\leq$ 250 ft	63 ft	**PASS**
IBC-R1-025	Corridor width	$\geq$ 44 in	72 in	**PASS**
IBC-R1-027	Dead-end corridor	$\leq$ 50 ft	24 ft	**PASS**
ADA-224.2	ADA mobility rooms	$\geq$ 7 for 117 keys	7	**PASS**
IBC-R1-007	Stories above grade	$\leq$ 5 (Type IIA, sprinklered R-1)	5	**PASS**

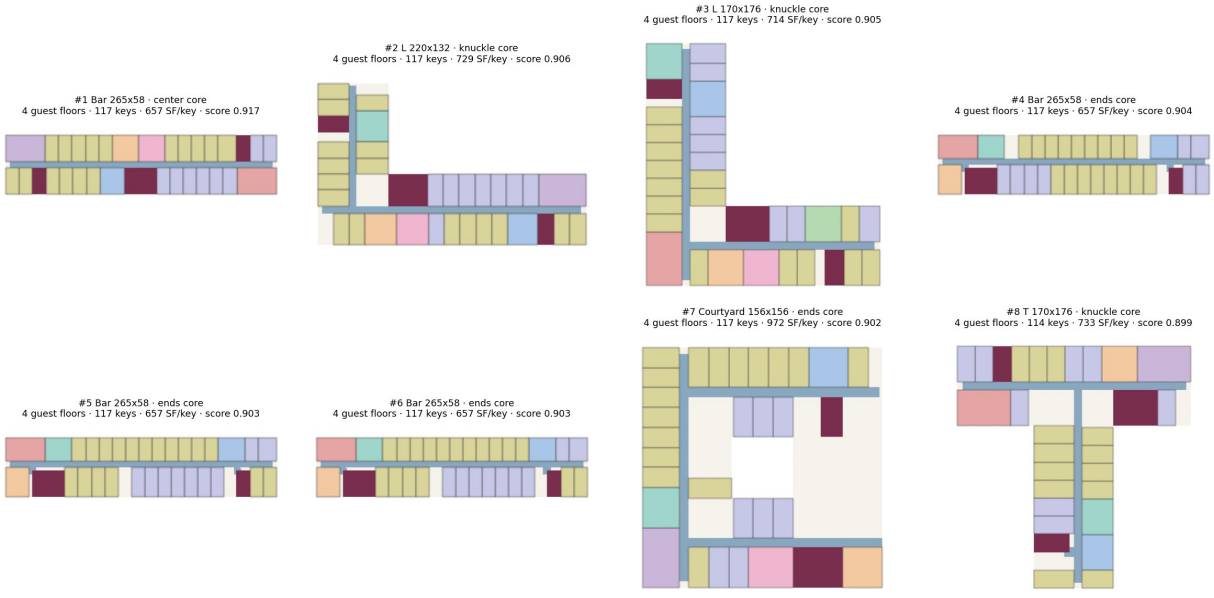


Figure 22: The eight first-ranked layouts (typical guest floor). Dark red: core and stairs; blue: corridor.

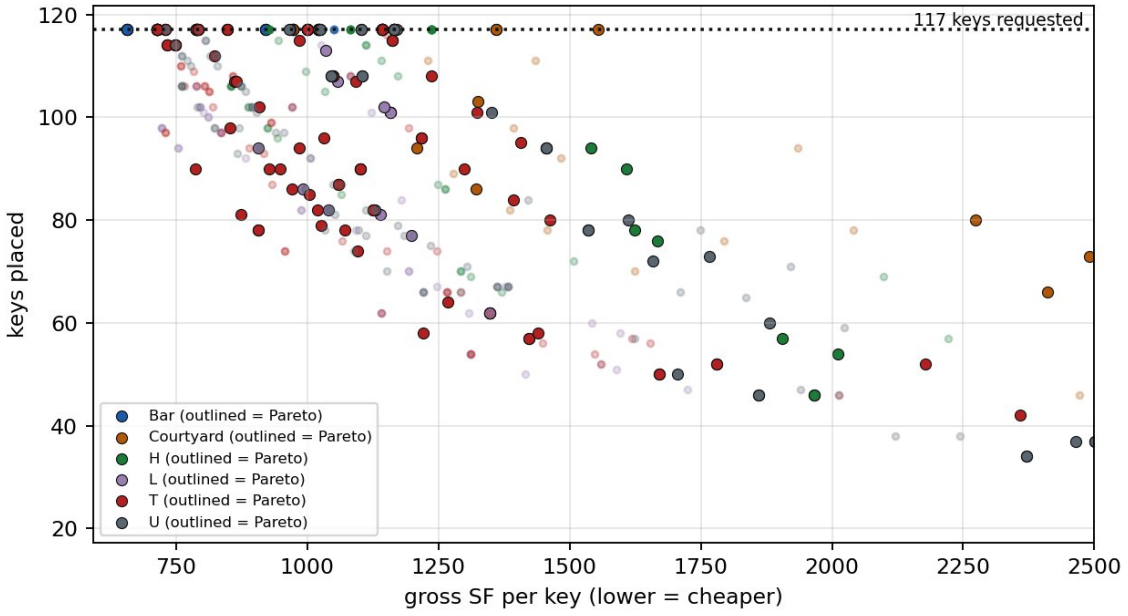


Figure 23: Every layout found: keys placed against gross area per key. Outlined points are on the Pareto front.



Figure 24: First-ranked layout, Level 2, as drawn in the app: furnished rooms, door swings, windows (blue), stairs and elevators.

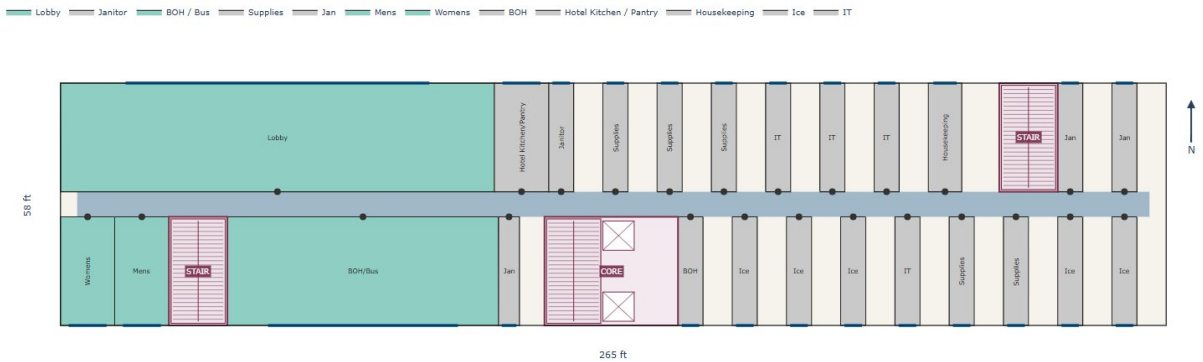


Figure 25: First-ranked layout, Level 1 (public and back-of-house rooms).

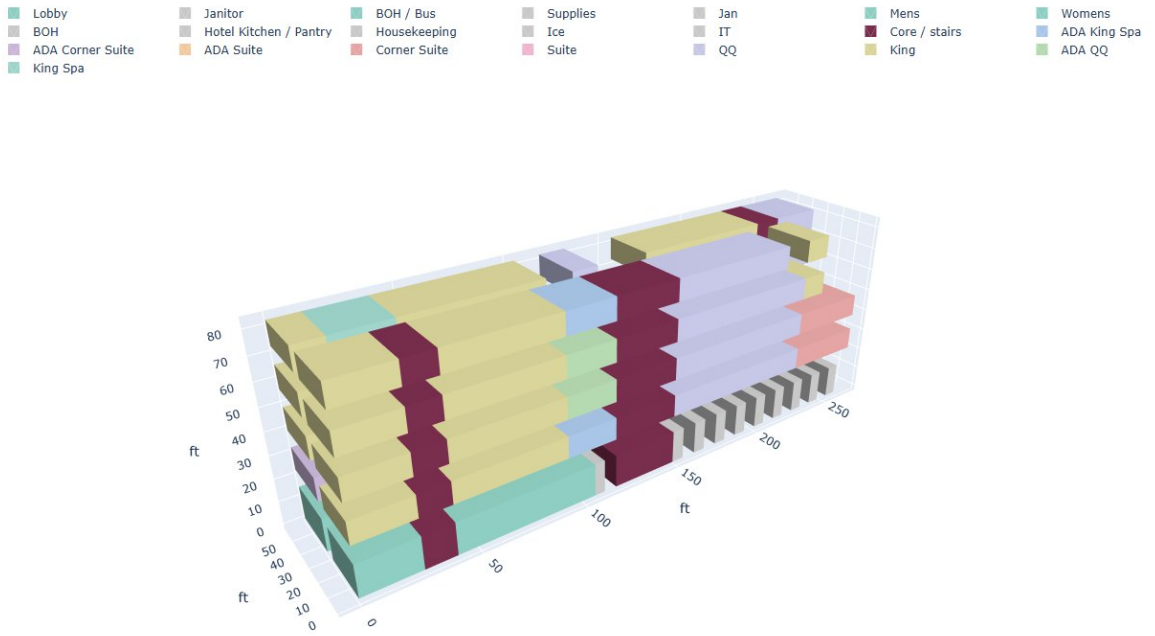


Figure 26: First-ranked layout, stacked massing with the floors pulled apart.

11.7 App screenshots

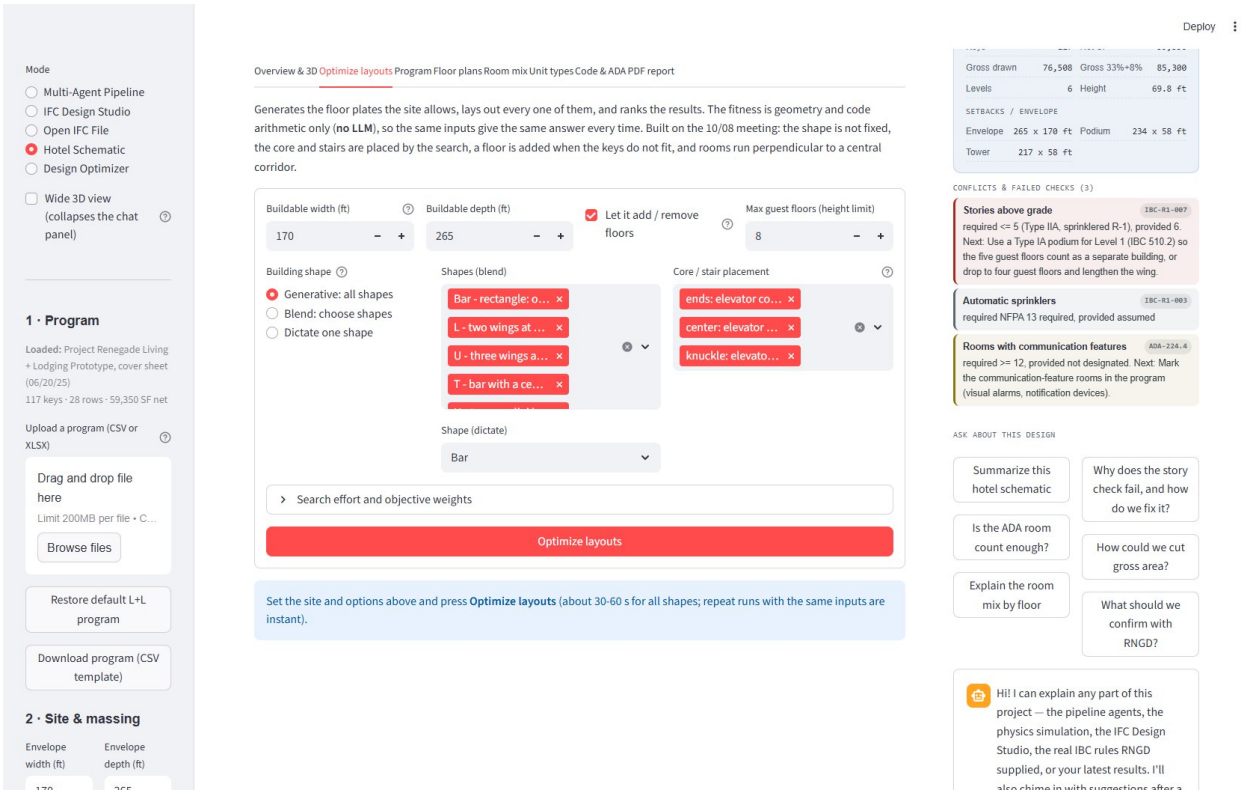


Figure 27: Optimize layouts: buildable area, shape mode (generative, blend or one shape), core placement and floors.

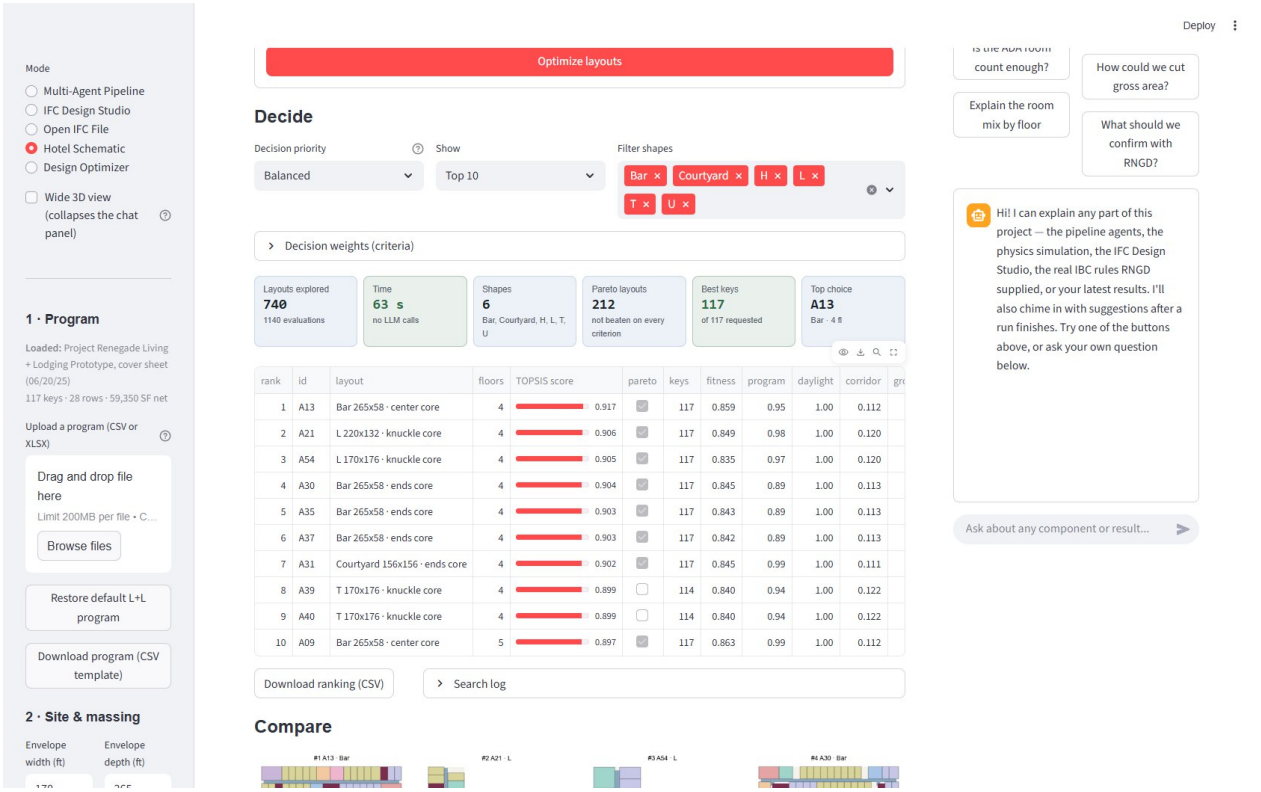


Figure 28: Decision step: every distinct layout ranked by TOPSIS under the chosen priority, with the Pareto flag; the Show dropdown switches between the top 5, 10 or 25, the Pareto front and all layouts.



Figure 29: Comparison: typical-floor thumbnails of the first-ranked layouts, keys against gross area per key for every layout, and the fitness gained by pattern search.

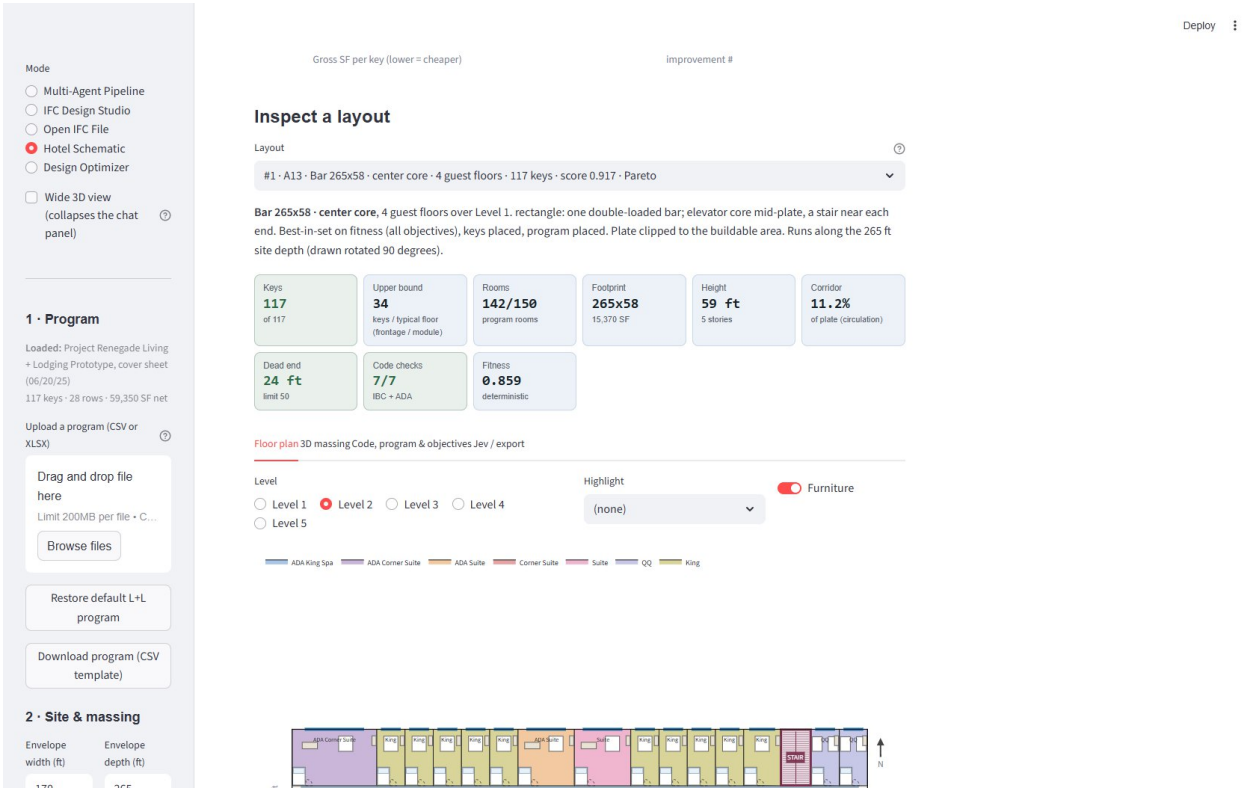


Figure 30: One layout inspected: keys, upper bound, footprint, corridor share, dead end, code checks and fitness.

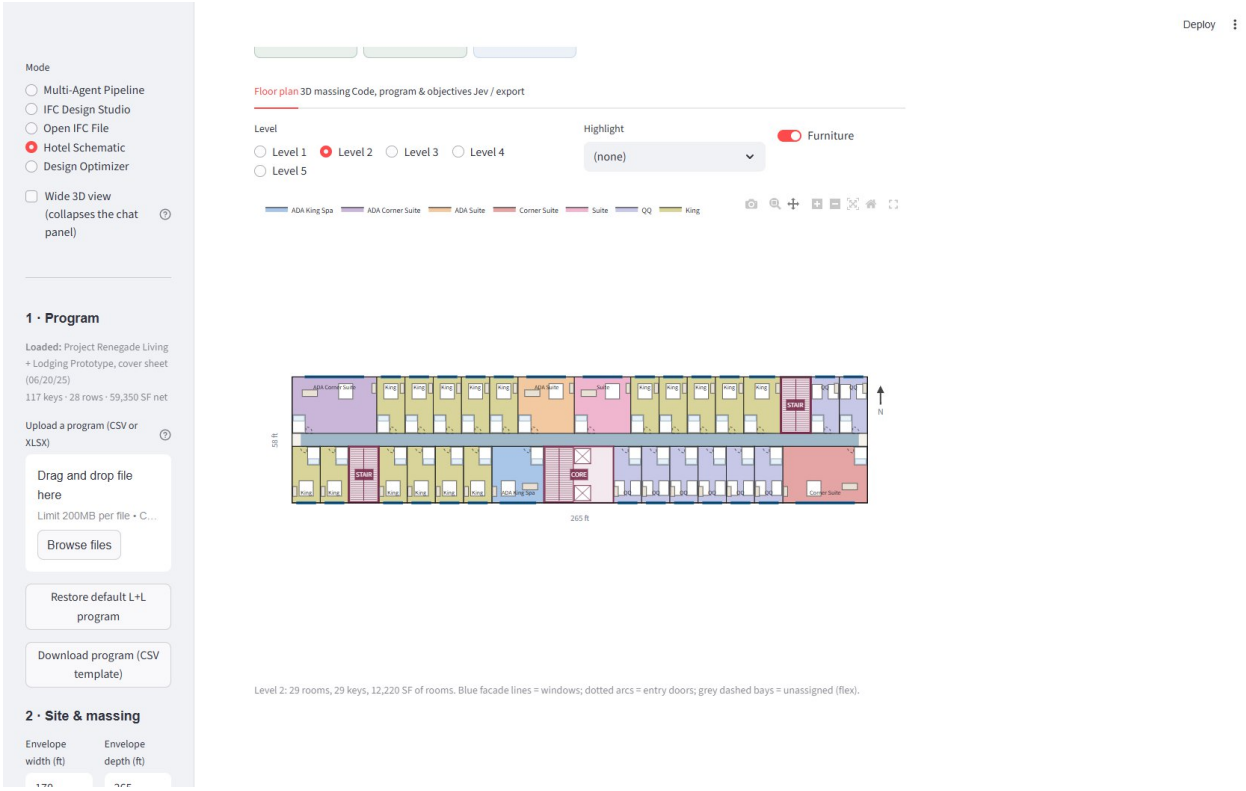


Figure 31: Interactive furnished plan of the selected layout (hover gives room data).



Figure 32: Stacked 3D massing of the selected layout, coloured by room type.

11.8 What is still open

- Level 1 uses the same plate as the guest floors (no wider podium), so large ground-floor rooms can be left out; they are listed for every layout.
- The decoder draws rectangles on a 1 ft grid: angled wings, curved facades and structural grids are not modelled.
- The upper bounds are loose; a tight bound would need an exact packing model.
- The centre-core scheme uses two stairs; on T, U and H plates the third wing can exceed the dead-end limit. The check flags it and the ranking moves such layouts down.
- Parking, cost and energy are not in the fitness. Facade length per key and gross area per key stand in for cost in the decision step.
- Jev (TypeSafe's hosted decision model) is wired only as an optional advisor; no key was configured for this run.

11.9 Relation to known methods

The reference prototype and variants B and C are genetic algorithms [3, 5]: a population of encoded plans is improved by selection, crossover and mutation. Variant C adds a local-search step to each generation, which makes it a memetic algorithm [14]. The room-order part of the genome is a random-key encoding [1]: one real number per room, sorted to give the order in which rooms claim bays, so any vector of numbers decodes to a valid order. The systematic search (variant D) combines exhaustive enumeration of a small structured space with a first-improvement pattern search in the spirit of Hooke and Jeeves [6], and then iterated local search [13]. The decision step keeps the non-dominated layouts [2] and ranks them with TOPSIS [8]. The main change from the reference is the encoding rather than the search method: the search no longer moves corridor segments and rooms directly, it chooses among corridors that already make architectural sense.

12 Implementation: the most important code

The listings below are extracted verbatim from the repository by `src/build_paper.py`; each starts with its source path.

12.1 Model access with a provider guard and JSON repair

Every LLM call goes through `call_json`. It refuses blocked model families, asks for JSON, and feeds a parsing error back to the model for a retry. This is the first guardrail against malformed or invented output.

```python
# src/rngd/llm.py
def assert_allowed(model: str) -> str:
    low = model.lower()
    for marker in BLOCKED_MARKERS:
        if marker in low:
            raise BlockedModelError(f"Model '{model}' is from a blocked provider ({marker}).")
    return model

def call_json(model: str, system: str, user: str, max_retries: int = 2,
              temperature: float = 0.2) -> tuple[dict[str, Any], float]:
    """JSON-mode call; feeds malformed-JSON errors back to the model and retries."""
    assert_allowed(model)
    client = _get_client()
    messages = [{"role": "system", "content": system}, {"role": "user", "content": user}]
    start = time.perf_counter()
    last_error = None
    for _ in range(max_retries + 1):
        resp = client.chat.completions.create(
            model=model, messages=messages, temperature=temperature,
            response_format={"type": "json_object"},
        )
        raw = resp.choices[0].message.content or ""
        try:
            parsed = json.loads(_strip_fences(raw))
            return parsed, round((time.perf_counter() - start) * 1000, 1)
        except json.JSONDecodeError as exc:
            last_error = exc
            messages.append({"role": "assistant", "content": raw})
            messages.append({"role": "user", "content": (
                f"That was not valid JSON ({exc}). Reply again with ONLY a single valid JSON object, "
                "no prose, no markdown fences.")})
    raise ValueError(f"JSON call failed after {max_retries + 1} attempts: {last_error}")
```

12.2 An agent prompt and its schema

The intake agent's instructions define the exact keys to return and tell the model to mark a requirement as hard only when the text says so; the output is validated by pydantic.

```python
# src/rngd/agents/intake_agent.py
SYSTEM = """You are RNGD's Program Intake Agent for a Building-as-a-Product (BaaP) \
concept-to-schematic design workflow. Extract a structured customer program from \
free-text customer requests.

Return ONLY a JSON object with exactly these keys:
- unit_count_total (integer or null)
- unit_mix_pct (object mapping "studio"|"1br"|"2br"|"3br" -> fraction 0-1, should sum to ~1.0)
- floor_count (integer or null)
- amenities (array of short strings)
- parking_preference (short string or null)
- budget_usd (number or null)
- schedule_note (short string or null)
- hard_requirements (array of short strings -- things the customer said must not change)
- soft_requirements (array of short strings -- things flagged as flexible/preference)
- assumptions (array of short strings -- anything you had to infer because it was not stated)

Be conservative: only mark something "hard" if the text says so explicitly (e.g. \
"firm", "must", "cannot go below", "hard requirement"). Everything else defaults to soft \
or goes in assumptions if you had to infer it."""

def run(rfp_text: str) -> tuple[CustomerProgram, float]:
    model = config.MODEL_ROUTE["intake"]
    parsed, latency_ms = llm.call_json(
        model=model,
        system=SYSTEM,
        user=f"Customer request:\n\n{rfp_text}",
    )
    return CustomerProgram.model_validate(parsed), latency_ms
```

12.3 Arithmetic stays in Python

The envelope, module counts and feasibility numbers (Section 4) are plain functions. Their results are *inputs* to the LLM prompts, never outputs of them.

```python
# src/rngd/agents/zoning_agent.py
def _compute_buildable_envelope(lot: dict, setbacks: dict, easements: list[Easement]) -> BuildableEnvelope:
    frontage, depth = lot["frontage_ft"], lot["depth_ft"]
    x_min = setbacks.get("side_west", 0.0)
    x_max = frontage - setbacks.get("side_east", 0.0)
    y_min = setbacks.get("front", 0.0)
    y_max = depth - setbacks.get("rear", 0.0)

    for e in easements:
        if not e.buildable:
            if e.side == "east":
                x_max = min(x_max, frontage - e.width_ft)
            elif e.side == "west":
                x_min = max(x_min, e.width_ft)
            elif e.side == "north":
                y_min = max(y_min, e.width_ft)
            elif e.side == "south":
                y_max = min(y_max, depth - e.width_ft)

    width = max(0.0, x_max - x_min)
    depth_env = max(0.0, y_max - y_min)
    return BuildableEnvelope(
        x_min_ft=x_min,
        x_max_ft=x_max,
        y_min_ft=y_min,
        y_max_ft=y_max,
        width_ft=width,
        depth_ft=depth_env,
        area_sf=round(width * depth_env, 1),
    )
```

```python
# src/rngd/agents/baap_matching_agent.py
def _module_counts(unit_count_total: int, unit_mix_pct: dict[str, float]) -> dict[str, int]:
    counts: dict[str, int] = {}
    assigned = 0
    items = list(unit_mix_pct.items())
    for i, (utype, pct) in enumerate(items):
        module_id = UNIT_TYPE_TO_MODULE.get(utype.lower(), "ONEBR_A")
        if i == len(items) - 1:
            n = unit_count_total - assigned  # last bucket absorbs rounding remainder
        else:
            n = round(unit_count_total * pct)
        assigned += n
        counts[module_id] = counts.get(module_id, 0) + max(0, n)
    return counts
```

```python
# src/rngd/agents/conflict_agent.py
def _compute_feasibility(program: CustomerProgram, site: SiteConstraints, baap: BaaPPlan) -> dict:
    lot_area = site.lot_area_sf
    zr = site.zoning
    max_gsf_by_far = lot_area * zr.max_far if zr.max_far else None
    max_footprint_by_coverage = lot_area * (zr.max_lot_coverage_pct / 100) if zr.max_lot_coverage_pct else None

    required_stalls = None
    if zr.parking_ratio_per_unit and program.unit_count_total:
        required_stalls = round(zr.parking_ratio_per_unit * program.unit_count_total)

    surface_parking_area_needed = required_stalls * SF_PER_STALL_WITH_AISLE_ALLOWANCE if required_stalls else None

    envelope_area = site.buildable_envelope.area_sf if site.buildable_envelope else None
    footprint = baap.estimated_footprint_sf_per_floor
    land_left_for_parking = (envelope_area - footprint) if envelope_area is not None else None

    return {
        "lot_area_sf": lot_area,
        "max_buildable_gsf_by_far": max_gsf_by_far,
        "requested_building_gsf": baap.estimated_building_gsf,
        "max_footprint_by_lot_coverage_sf": max_footprint_by_coverage,
        "estimated_footprint_per_floor_sf": footprint,
        "buildable_envelope_area_sf": envelope_area,
        "required_parking_stalls": required_stalls,
        "surface_parking_area_needed_sf": surface_parking_area_needed,
        "land_left_for_parking_after_footprint_sf": land_left_for_parking,
        "surface_parking_likely_fits": (
            (surface_parking_area_needed <= land_left_for_parking)
            if surface_parking_area_needed is not None and land_left_for_parking is not None
            else None
        ),
    }
```

12.4 Retrieval

```python
# src/rngd/vectorstore.py
def retrieve(self, query: str, k: int) -> list[dict]:
    q = self.model.encode([query], normalize_embeddings=True)[0]
    scores = self.embeddings @ q
    top = np.argsort(-scores)[:k]
    return [
        {
            "text": self.chunks[i].text,
            "source": self.chunks[i].source,
            "heading": self.chunks[i].heading,
            "source_kind": self.chunks[i].source_kind,
            "score": float(scores[i]),
        }
        for i in top
    ]

def retrieve_multi_as_context(self, queries: list[str], k_each: int = 2) -> str:
    """Issues several narrower queries and merges deduped hits. On a small,
    multi-topic corpus like this one, a single broad query tends to be
    dominated by whichever section best matches the query's dominant
    terms, crowding out other sections an agent also needs (e.g. a
    zoning query for "setbacks height FAR parking fire ADA" can rank
    Accessibility above Bulk Standards even though both are needed) --
    several targeted queries recall more of the relevant corpus than one
    broad query at the same total k.
    """
    seen: set[tuple[str, str]] = set()
    blocks = []
    for q in queries:
        for h in self.retrieve(q, k_each):
            key = (h["source"], h["heading"])
            if key in seen:
                continue
            seen.add(key)
            blocks.append(f"[source: {h['source']} | {h['heading']}]\n{h['text']}")
    return "\n\n".join(blocks)
```

12.5 Physics engine

The separation force acts along the axis of least overlap. The sign was a real bug: an early version pushed bodies together and diverged. `step` integrates with damping, and `run` stops on convergence.

```python
# src/rngd/simulation/physics_engine.py
def _separation_force(a: Body, b: Body) -> tuple[float, float]:
    ax0, ay0, ax1, ay1 = a.bounds()
    bx0, by0, bx1, by1 = b.bounds()
    ox = _overlap_1d(ax0, ax1, bx0, bx1)
    oy = _overlap_1d(ay0, ay1, by0, by1)
    if ox <= 0 or oy <= 0:
        return 0.0, 0.0
    acx, acy = (ax0 + ax1) / 2, (ay0 + ay1) / 2
    bcx, bcy = (bx0 + bx1) / 2, (by0 + by1) / 2
    # Push apart along the axis of least overlap (standard rectangle
    # separating-axis heuristic). `a` must move AWAY from `b`: if a's center
    # is left of b's, a moves further left (negative x), so the sign is the
    # opposite of the naive "a < b -> positive" comparison.
    if ox < oy:
        direction = -1.0 if acx < bcx else 1.0
        return direction * ox, 0.0
    else:
        direction = -1.0 if acy < bcy else 1.0
        return 0.0, direction * oy

def step(self, damping: float = 0.55, gain: float = 0.5) -> float:
    total_penalty = 0.0
    forces = {b.id: [0.0, 0.0] for b in self.bodies}

    for b in self.bodies:
        cfx, cfy = self._containment_force(b)
        forces[b.id][0] += cfx
        forces[b.id][1] += cfy
        total_penalty += abs(cfx) + abs(cfy)

    n = len(self.bodies)
    for i in range(n):
        for j in range(i + 1, n):
            a, b = self.bodies[i], self.bodies[j]
            sx, sy = _separation_force(a, b)
            if sx == 0 and sy == 0:
                continue
            total_penalty += abs(sx) + abs(sy)
            if a.movable:
```

```
                forces[a.id][0] += sx * 0.5
                forces[a.id][1] += sy * 0.5
            if b.movable:
                forces[b.id][0] -= sx * 0.5
                forces[b.id][1] -= sy * 0.5

    for b in self.bodies:
        if not b.movable:
            continue
        fx, fy = forces[b.id]
        b.vx = (b.vx + gain * fx) * damping
        b.vy = (b.vy + gain * fy) * damping
        b.x += b.vx
        b.y += b.vy

    self.history.append(total_penalty)
    return total_penalty

def run(self, max_steps: int = 250, tol: float = 0.5) -> dict:
    converged = False
    steps_run = 0
    for i in range(max_steps):
        penalty = self.step()
        steps_run = i + 1
        if penalty < tol:
            converged = True
            break
    return {
        "steps_run": steps_run,
        "max_overlap_penalty": round(self.history[-1] if self.history else 0.0, 3),
        "converged": converged,
        "history": self.history,
    }
```

12.6 Vision calibration and the critic loop

```
# src/rngd/vision/layout_qa.py
class PixelToFeet:
    def __init__(self, geom_sidecar: dict):
        (x0_ft, y0_ft), (x1_ft, y1_ft) = geom_sidecar["ref_ft"]
        (x0_px, y0_px), (x1_px, y1_px) = geom_sidecar["ref_px"]
        self.sx = (x1_ft - x0_ft) / (x1_px - x0_px)
        self.sy = (y1_ft - y0_ft) / (y1_px - y0_px)
        self.x0_ft, self.x0_px = x0_ft, x0_px
        self.y0_ft, self.y0_px = y0_ft, y0_px

    def to_ft(self, x_px: float, y_px: float) -> tuple[float, float]:
        x_ft = self.x0_ft + (x_px - self.x0_px) * self.sx
        y_ft = self.y0_ft + (y_px - self.y0_px) * self.sy
        return x_ft, y_ft
```

The loop exit is decided by code; the critic only proposes deltas, which are clamped.

```
# src/rngd/orchestrator.py
# A clean geometry pass with zero parking shortfall needs no critic
# opinion -- the design is complete. Everything else (a geometry
# violation, or any parking shortfall) goes to the critic, which
# decides whether another simulation pass can plausibly help or
# whether this is a capacity/program conflict no geometry tweak
# will fix (see design_critic_agent.py).
if cv.geometry_passed and cv.parking_shortfall == 0:
    break

with traced_call(state, "design_critic", f"propose_adjustment_iteration_{iteration}", config.MODEL_ROUTE["design_critic"])
     as t:
    critique, latency = design_critic_agent.run(cv, stats, iteration, max_iterations)
    state.critic_history.append(critique)
    t.summary = f"continue={critique.continue_iterating}: {critique.rationale[:120]}"
    t.details = {"latency_ms": latency, "adjustments": critique.adjustments}
_fire()

if not critique.continue_iterating:
    break
footprint_w = max(20.0, footprint_w + critique.param_deltas.get("footprint_width_delta_ft", 0.0))
footprint_h = max(20.0, footprint_h + critique.param_deltas.get("footprint_depth_delta_ft", 0.0))
driveway_w = max(20.0, driveway_w + critique.param_deltas.get("driveway_width_delta_ft", 0.0))
```

```
# src/rngd/agents/design_critic_agent.py
def run(cv: CVMetrics, sim_stats: SimulationStepStats, iteration: int, max_iterations: int) -> tuple[CriticSuggestion,
     float]:
    model = config.MODEL_ROUTE["design_critic"]
    parsed, latency_ms = llm.call_json(
        model=model,
        system=SYSTEM,
```

```
        user=(
            f"Iteration {iteration} of max {max_iterations}.\n"
            f"CV QA measurements:\n{json.dumps(cv.model_dump(), indent=2)}\n\n"
            f"Simulation convergence stats:\n{json.dumps(sim_stats.model_dump(), indent=2)}"
        ),
    )
    if iteration >= max_iterations:
        parsed["continue_iterating"] = False
    return CriticSuggestion.model_validate(parsed), latency_ms
```

12.7 Real IFC recentring

```
# src/rngd/ifc/explorer.py
def _recenter(meshes: list[dict], outlier_factor: float = 4.0, min_outlier_dist_m: float = 50.0) -> tuple[list[dict], dict]:
    """Translates every mesh so the building sits near the origin, and drops
    elements sitting implausibly far from the rest of the model (see
    STRUCTURAL_CLASSES above). This is a display-space transform applied only
    to the in-memory/cached mesh data -- the source .ifc file on disk is never
    modified. Returns (kept_meshes, report)."""
    structural = [m for m in meshes if m["cls"] in STRUCTURAL_CLASSES]
    reference = structural or meshes
    ref_verts = np.concatenate([m["verts"] for m in reference], axis=0)
    origin = np.median(ref_verts, axis=0)
    diag = float(np.linalg.norm(ref_verts.max(axis=0) - ref_verts.min(axis=0)))
    threshold = max(diag * outlier_factor, min_outlier_dist_m)

    kept, dropped = [], []
    for m in meshes:
        centroid = m["verts"].mean(axis=0)
        dist = float(np.linalg.norm(centroid - origin))
        if dist > threshold:
            dropped.append({"name": m["name"], "cls": m["cls"], "distance_m": round(dist, 1)})
            continue
        m = dict(m, verts=m["verts"] - origin)
        kept.append(m)

    report = {
        "origin_offset_m": [round(x, 2) for x in origin.tolist()],
        "building_diagonal_m": round(diag, 1),
        "excluded_outliers": dropped,
    }
    return kept, report
```

12.8 Hotel generator

Routing the program rows, balancing the wing (Algorithm 1), and the egress geometry used by the code screen:

```
# src/rngd/hotel/layout.py
def split_rows(program: Program, upper_levels: int):
    """Sort non-guest rows into: per-floor support (small rooms with count >= 2), core
    support on Level 1 (small BOH rooms), Level 1 band rooms, tenant blocks.
    Stairs and corridors are generated, not packed."""
    per_floor, core_l1, band, tenant = [], [], [], []
    for name, count, area, cat in program.rows:
        if cat == "guest" or count == 0:
            continue
        kind = _kind_for(name, cat)
        if kind in ("stair", "corridor"):
            continue
        each = area / count
        if count >= 2 and each < 150:
            # Small repeated rooms (Ice, IT, Jan, Supplies): one per guest floor, total area kept.
            per_floor.append((name, area / upper_levels, kind))
        elif cat == "boh" and area < 150:
            core_l1.append((name, area, kind))
        elif kind == "tenant":
            tenant.append((name, area, kind))
        else:
            band += [(name, each, kind)] * count
    return per_floor, core_l1, band, tenant
```

```
# src/rngd/hotel/layout.py
# First pass: guest wings, balanced so both sides end near the same x.
wings = []
for group in per:
    sides = {"S": [], "N": []}
    run = {"S": 0.0, "N": STAIR_W_FT}  # north side also has to fit Stair 2
    for r in group:
        s = "S" if run["S"] <= run["N"] else "N"
        sides[s].append(r)
```

```
        run[s] += r["area_sf"] / ROOM_DEPTH_FT
    wings.append((sides, run))
length = CORE_W_FT + max(max(run.values()) for _, run in wings)
```

```
# src/rngd/hotel/layout.py
# Egress geometry: stair doors open off the corridor at mid-stair-width.
s1_door = STAIR_W_FT / 2
s2_door = length - STAIR_W_FT / 2
dead_end = max(s1_door, length - s2_door)
travel = (s2_door - s1_door) / 2 + ROOM_DEPTH_FT  # corridor run to nearest stair + walk inside the room
```

The code screen evaluates RNGD's rules from the geometry, and the ADA table is encoded exactly:

```
# src/rngd/hotel/code_check.py
def evaluate(sch) -> list[dict]:
    from .layout import CORRIDOR_FT, DOOR_FT, diagonal

    p = sch.program
    upper = sch.levels[1:]
    max_ol = max(occupant_load(lv) for lv in sch.levels)
    sep = sch.stair_doors_x[1] - sch.stair_doors_x[0]
    env_w, env_d = sch.site["envelope_w"], sch.site["envelope_d"]
    L = max(lv.length_ft for lv in sch.levels)
    top_floor = upper[-1].elevation_ft if upper else 0
    checks = [
        {"rule_id": "-", "check": "Keys placed", "required": str(p.keys), "provided": str(p.keys - sch.unplaced_keys),
         "status": "PASS" if sch.unplaced_keys == 0 else "FAIL", "source": "Program", "note": ""},
        _c("IBC-R1-007", "Stories above grade", "<= 5 (Type IIA, sprinklered R-1)", str(sch.stories), sch.stories <= 5,
           "Level 1 counts as a story above grade plane. A Type IA podium (IBC 510.2) could allow 5 stories above it."),
        _c("IBC-R1-005", "Building height (ft)", "<= 85 (Type IIA)", f"{sch.height_ft:.1f}", sch.height_ft <= 85),
        _c("IBC-R1-010", "Area per story (SF)", "<= 72,000 (Type IIA)", f"{sch.footprint_sf:,.0f}", sch.footprint_sf <=
     72000),
        _c("IBC-R1-012", "High-rise screen (ft to top floor)", "<= 75", f"{top_floor:.1f}", top_floor <= 75),
        _c("IBC-R1-015", "Exits per story", "2 for 1-500 occupants", f"2 stairs; max load {max_ol}", max_ol <= 500),
        _c("IBC-R1-018", "Exit separation (ft)", f">= 1/3 diagonal = {diagonal(sch) / 3:.0f}", f"{sep:.0f}",
           sep >= diagonal(sch) / 3),
        _c("IBC-R1-023", "Exit access travel (ft)", "<= 250 (sprinklered)", f"{sch.max_travel_ft:.0f}",
           sch.max_travel_ft <= 250, "Half the stair-to-stair run plus the room depth."),
        _c("IBC-R1-025", "Corridor width (in)", ">= 44", f"{CORRIDOR_FT * 12:.0f}", CORRIDOR_FT * 12 >= 44),
        _c("IBC-R1-027", "Dead-end corridor (ft)", "<= 50 (sprinklered R-1)", f"{sch.dead_end_ft:.0f}", sch.dead_end_ft <=
     50),
        _c("IBC-R1-028", "Egress door clear width (in)", ">= 32", f"{DOOR_FT * 12 - 2:.0f} (3'-0\" leaf)", True),
        _c("IBC-R1-003", "Automatic sprinklers", "NFPA 13 required", "assumed", "ASSUMED"),
        {"rule_id": "-", "check": "Fits buildable envelope", "required": f"{env_w:.0f} x {env_d:.0f} ft",
         "provided": f"{L:.0f} x {sch.levels[0].width_ft:.0f} ft",
         "status": "PASS" if L <= env_d and sch.levels[0].width_ft <= env_w else "FAIL", "source": "Site (synthetic lot)",
         "note": ""},
    ]
    from .details import ada_checks
    checks[-1:-1] = ada_checks(p)  # ADA scoping before the site check
    return checks
```

```
# src/rngd/hotel/details.py
def ada_required(rooms: int) -> tuple[int, int, int]:
    """(without roll-in, with roll-in, total) mobility rooms for `rooms` guest rooms."""
    for cap, a, b in ADA_MOBILITY:
        if rooms <= cap:
            return a, b, a + b
    if rooms <= 1000:
        return math.ceil(rooms * 0.02), math.ceil(rooms * 0.01), math.ceil(rooms * 0.03)
    extra = math.ceil((rooms - 1000) / 100)
    return 20 + extra, 10 + extra, 30 + 2 * extra
```

Placing a room and projecting to 3D:

```
# src/rngd/hotel/drawings.py
def room_xf(r, mirror: bool):
    """Local (u, v) of a guest room -> plan (x, y); v runs away from the corridor."""
    def xf(u, v):
        x = r.x + (r.w - u if mirror else u)
        y = r.y + r.d - v if r.side == "S" else r.y + v
        return x, y
    return xf

def __init__(self, az_deg=-55.0, el_deg=32.0):
    th, ph = math.radians(az_deg), math.radians(el_deg)
    self.c = np.array([math.cos(ph) * math.cos(th), math.cos(ph) * math.sin(th), math.sin(ph)])
    self.r = np.array([-math.sin(th), math.cos(th), 0.0])
    self.u = np.cross(self.c, self.r)
    self.items = []  # (depth, polys[(pts, fc, ec, lw)])

def p(self, pts):
    a = np.asarray(pts, float)
    return np.c_[a @ self.r, a @ self.u]
```

12.9 Layout optimizer

The fitness (Section 11.4) is a weighted mean of cached objective terms minus the penalty for an exit the corridor does not reach. It contains no model call:

```
# src/rngd/optimize/metrics.py
def fitness_from_terms(terms: dict, weights: dict, cores_reached: int, n_cores: int) -> float:
    """Re-score cached objective terms under new weights (no re-decoding needed)."""
    total_w = sum(weights[k] for k in terms) or 1.0
    f = sum(weights[k] * terms[k] for k in terms) / total_w
    return f - 0.5 * max(0, min(n_cores, LIMITS["exits"]) - cores_reached)
```

Stage 1 of the systematic search builds one genome per set of corridor spines, with every room-order key at 0.5 so that rooms claim bays in priority order:

```
# src/rngd/optimize/search.py
top = list(range(min(m, 7)))
combos = [list(range(m))] + [list(c) for k in range(1, min(4, len(top)) + 1)
                             for c in itertools.combinations(top, k)]
genomes = []
for c in combos:
    g = engine._seed(self.rng, n_inst, c)
    g[engine.N_SEG * 4:] = 0.5      # rooms in priority order
    genomes.append(g)
```

For the hotel, the elevator count and the corridor length each plate shape offers:

```
# src/rngd/optimize/typology.py
def elevators_for(keys: int) -> int:
    """Rule of thumb for select-service hotels: about one elevator per 75 keys, at least two."""
    return max(2, math.ceil(keys / 75))

def _run_ft(shape: str, a: int, b: int, wd: int) -> float:
    """Length of double-loaded corridor the shape offers (corners count once)."""
    return {"Bar": a, "L": a + b - wd, "U": a + 2 * b - 2 * wd, "T": a + b - wd, "H": 2 * b + a - 2 * wd,
            "Courtyard": 2 * a + 2 * b - 4 * wd}[shape]
```

The decision step, Pareto dominance and TOPSIS:

```
# src/rngd/optimize/decide.py
def pareto_mask(alts, keys=None) -> np.ndarray:
    keys = keys or list(CRITERIA)
    X = _matrix(alts, keys) * np.array([CRITERIA[k][0] for k in keys])   # larger = better
    n = len(alts)
    keep = np.ones(n, bool)
    for i in range(n):
        if not keep[i]:
            continue
        dom = np.all(X >= X[i], axis=1) & np.any(X > X[i], axis=1)
        if dom.any():
            keep[i] = False
    return keep

def topsis(alts, weights: dict) -> np.ndarray:
    keys = [k for k in CRITERIA if weights.get(k, 0) > 0]
    if not alts or not keys:
        return np.zeros(len(alts))
    X = _matrix(alts, keys)
    norm = np.sqrt((X ** 2).sum(0))
    norm[norm == 0] = 1
    w = np.array([weights[k] for k in keys], float)
    V = X / norm * (w / w.sum())
    sign = np.array([CRITERIA[k][0] for k in keys])
    ideal = np.where(sign > 0, V.max(0), V.min(0))
    worst = np.where(sign > 0, V.min(0), V.max(0))
    dp = np.sqrt(((V - ideal) ** 2).sum(1))
    dm = np.sqrt(((V - worst) ** 2).sum(1))
    return dm / np.where(dp + dm == 0, 1, dp + dm)
```

13 Evaluation: dynamic or hallucinating?

13.1 The hotel generator responds to inputs and repeats exactly

Four inputs were each built twice. The fingerprint is a SHA-256 of every room's position and size plus every check result.

Table 14: Four inputs to the hotel generator.

Case	Keys	Lvls	Keys/floor	Wing	L1	Gross SF	ADA req→prov	Fails	Hash	Same?
A1 L+L, 5 floors	117	6	24,24,23,23,23	217.0	234.1	76,508	7→7	stories	f1ecee4c	yes
A2 L+L, 4 floors	117	5	30,29,29,29	258.0	258.0	74,828	7→7	none	7b58316d	yes
A3 76-key program	76	5	19,19,19,19	167.5	167.5	48,564	5→4	ADA rooms	ff6fff58	yes
A4 scaled to 161 keys	161	6	33,32,32,32,32	288.4	288.4	100,359	8→10	stories, envelope	b4fea144	yes

- **Geometry follows the input.** Fewer floors give a longer wing; a smaller program gives a shorter wing; a larger program overflows the 265 ft envelope.
- **Checks follow the input.**
  - A2 passes the story limit (Level 1 plus four floors makes five stories).
  - A3 fails ADA because 76 rooms need 5 accessible rooms and the program has 4.
  - A4 does not fit the lot.
- **Identical inputs give identical outputs**, room for room.



Figure 33: Level 2 of cases A1–A4 at the same scale.

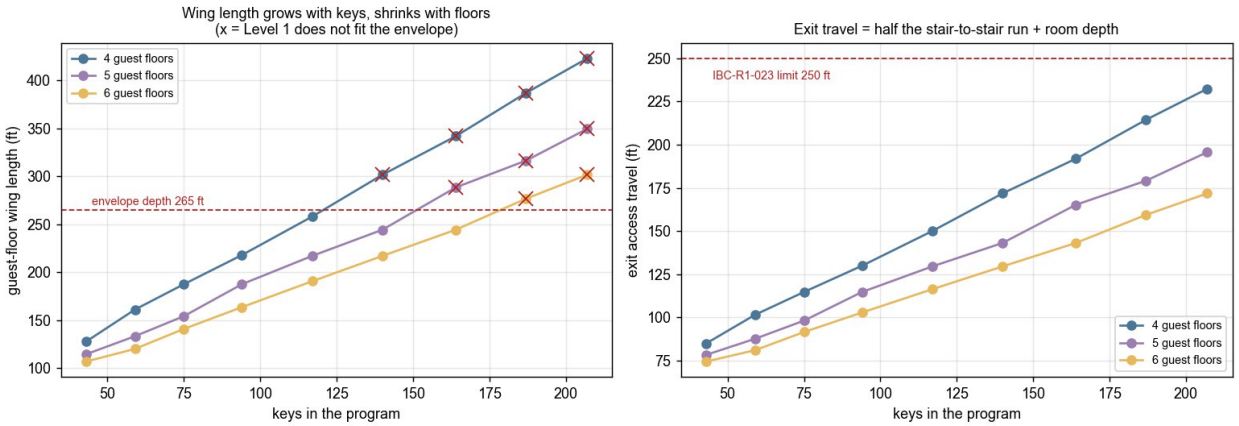


Figure 34: Sweep from 43 to 207 keys at 4, 5 and 6 guest floors. Wing length grows with keys and shrinks with floors (× marks cases where Level 1 does not fit the envelope). Exit travel follows $T = (L - 10)/2 + 26$ and stays below 250 ft.

13.2 The layout optimizer improves on the reference and repeats exactly

The reference GA's delivered designs and three later variants were scored by one evaluator on the prototype's three sites, with the same 72-room program per site (Table 9; budget 24 × 30 evaluations for B, C and D, seed 7, no LLM).

Table 15: Totals over the three sites (216 rooms asked): rooms placed, guest rooms without a window, mean fitness and run time for all three sites. Run time was not recorded for the reference; its history holds 500 generations.

Variant	Rooms	No window	Fitness	Time
A reference GA	171	24	0.863	n/a
B GA, free-form corridors (first attempt)	174	14	0.889	336 s
C updated GA, architectural decoder	197	0	0.928	72 s
D systematic search, same decoder (default)	192	0	0.922	59 s

- **The decoder gives the gain.** B keeps the reference representation with better scoring and does not remove the windowless rooms. C and D share the new decoder and both do.
- **The search method matters less.** At the same budget the GA (C) scores slightly above the systematic search (D). D is the default because it needs no population or crossover and returns the same layout on every run, which was checked by running it twice on each site.
- **The comparison uses one measure.** The dead-end measure was corrected during this work (distance to the nearest exit door or through-route) and is applied to the reference designs as well.
- **Hotel mode.** The exploration of the Living + Lodging program kept 740 distinct layouts from 158 plate and floor-count variants in 36 s, and a second run returned the same list. The first-ranked layout (Bar 265x58 · center core, 4 guest floors) places 117 of 117 keys and passes 7/7 checks, but only 142 of 150 program rooms: some Level 1 rooms do not fit a plate sized for the guest floors.

13.3 The LLM agents extract what the documents say

Table 16: Intake agent on the original and an edited request.

Input	Units	Floors	Mix	Budget	Hard requirements
original, run 1	120	4	.20/.45/.35	$28.0M	"120 units"
original, run 2	120	4	.20/.45/.35	$28.0M	"120 units"
edited	**84**	**6**	**.10/.50/.40**	**$19.5M**	"84 units firm requirement", "parking per local code"

Table 17: Zoning agent versus the values printed in the knowledge base (two runs).

Limit	Knowledge base	Run 1	Run 2
height / stories / FAR / coverage	65 ft / 5 / 2.5 / 60%	same	same
setbacks front / side / rear	15 / 10 / 20 ft	same	same
parking ratio / fire lane	1.25 per unit / 20 ft	same	same
all 9 limits match		**yes**	**yes**

Every number in the saved arbiter narrative (1.25, 15, 120, 135, 150) appears in the computed data: the parking ratio, detected stalls, units, the shortfall $150 - 15$, and required stalls 1.25×120.

13.4 Run-to-run variability

Table 18: Three full pipeline runs of the same project (2026-10-05).

	Browser run 15:55	Test-suite run 16:12	Run 17:29
Decision	REVISE (0.95)	REVISE (0.95)	REVISE (0.93)
Conflicts	2	2 (critical, medium)	1 (high)
Critic	stopped, shortfall unresolved	tried $+50 \times 50$ ft, then stopped	stopped after iteration 1
Stalls found by CV	15/150	34/150	15/150

- **Stable across runs:** the extracted program, zoning limits, envelope, module counts, feasibility numbers, the decision (REVISE in 3 of 3) and its reason (surface parking cannot fit).
- **Varies across runs:** how the LLM rates conflicts, and whether the critic tries one more geometry change. In the 16:12 run it enlarged the building; vision QA caught the resulting setback violation, and the arbiter still returned REVISE.

Each narrative's numbers came from its own run's measurements.

13.5 Tests and timings

All 51 automated tests pass. They cover the pipeline agents, the IFC engine with an IFC round trip, the hotel generator, ADA tables, interactive figures, the layout optimizer (decoder, fitness, determinism of the systematic search, hotel exploration and ranking), and app tests that drive the real Streamlit app, including a full pipeline run with real LLM calls. In the browser run, the pipeline took 105 s, opening the 89 MB IFC file 52 s, generating the hotel PDF 24 s and an assistant reply 37 s, with no on-page errors across 31 screenshots.

14 Discussion

14.1 Guardrails against hallucination

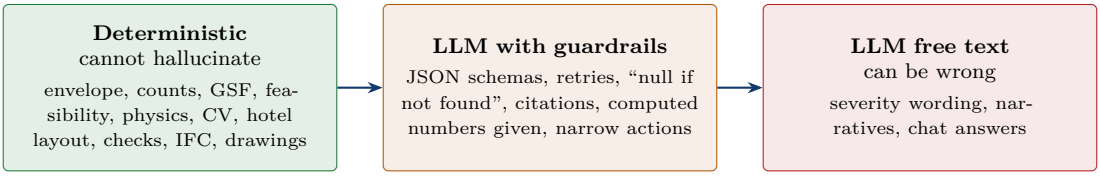


Figure 35: Where errors can and cannot come from.

The code enforces eight guardrails:

1. Arithmetic is never left to the model.
2. Agent outputs are validated against schemas, with retries.
3. The zoning prompt says to return `null` rather than invent a number, and to cite the source section.

4. Vision QA measures the drawing independently of the simulation.
5. The critic can change only three parameters, and the loop is capped in code.
6. The arbiter must not approve with an unresolved critical or high conflict.
7. The hotel and IFC designs use no LLM, and neither does the layout optimizer's fitness or search.
8. LLM requests time out after 90 s.

One observed error shows the remaining risk. The chat assistant explained the story-count failure with an "8 ft" story definition that is not in RNGD's rules; the code check itself was right. Chat answers are explanations; the code screens are the facts.

14.2 Engineering lessons (bugs found and fixed)

- **Physics sign error.** Separation pushed bodies together and the solver diverged. The fixed version converges in six steps.
- **DPI mismatch.** The calibration used 100 dpi while the PNG was 140 dpi, a 1.4× error in every CV measurement. The vision QA's independent measurement exposed it.
- **Corridor orientation.** The floor plates ran along the short axis, so most units were deferred.
- **An ambiguous field caught by the arbiter.** A single `passed` flag ignored parking; the arbiter flagged the inconsistency, and the field was split into `geometry_passed` and `parking_shortfall`.
- **Missing LLM timeout.** One stalled request could freeze a run for up to 30 minutes; a 90 s timeout was added.
- **Provenance.** The test suite's pipeline run overwrote saved results; captures now keep their own copy.
- **Real-IFC outliers.** Four objects 896 m away shrank the view; they are now recentred and excluded.
- **LLM reviewer in the fitness.** The first optimizer asked a language model to review plans during the search. Runs took about ten minutes and changed between runs; the reviewer was removed from the fitness.
- **Windowless bays taking the facade.** With crossing corridors the interior side of one corridor took the facade strip another corridor needed (39 rooms on the square site). Building window bays first fixed it (Section 11).

14.3 Positions on RNGD's kickoff questions

- **Schematic scope.** The output is a site plan, ground and typical floors (pipeline), and a full hotel set: all levels, elevations, enlarged units, 3D.
- **Output format.** 2D plans and PDF first, as preferred in the meeting; IFC4 for BIM exchange.
- **Written report.** It is generated automatically (arbiter narrative, hotel drawing set and appendices).
- **Insufficient information.** Agents are instructed to say so rather than fabricate a number.
- **Real codes and BaaP data.** Swapping the remaining synthetic sources for RNGD's is a data change, not an architecture change.

14.4 Limitations

- **Synthetic inputs.** The pipeline and IFC-studio inputs are synthetic.
- **Physics.** The engine cannot add parking, and there is no attraction force.
- **Floor plates.** They pack one wing and defer units.
- **Hotel layout.** The drawing set still shows the single straight wing of Section 10; the optimized layouts of Section 11 are shown as interactive plans and 3D and exported as JSON, not yet as sheets. There is no parking, structure or MEP.
- **Layout optimizer.** Level 1 shares the guest-floor plate, so large ground-floor rooms can be left out. Rooms are rectangles on a 1 ft grid, and the upper bounds on keys are loose. The reference GA's run time is not recorded, so speed is compared only among the later variants.
- **Code coverage.** 11 of the 42 IBC rules are evaluated, and communication-feature rooms are not yet designated.
- **LLM variation.** The LLM's judgments vary between runs.

14.5 Open questions for RNGD

1. Which area of the Level 1 / Level 2 reference drawing was meant? The coordinate note (scale 1.08 to the original image) did not name a room, stair or gridline.

2. Is a Type IA podium (IBC 510.2) intended, as in the L+L prototype? Six levels exceed the five-story rule.
3. Which gross-up rule should be used: 33% + 8% from the meeting, or the source sheets' 1.25 ratio?
4. What are the real lot, setbacks, frontage and fire-lane requirements?
5. Is parking in scope for this phase?
6. What is the preferred structural module, and the prefab panel catalogue?

15 Conclusion and future work

The RNGD AI Studio shows a practical split of work for AI-assisted early design:

- language models read requests and documents and make judgments;
- deterministic engines compute, simulate, measure, lay out and draw.

In the experiments this made the designs repeatable and responsive to their inputs, and kept invented numbers out of the decisions. The hotel generator already produces a drawing set in RNGD's own format from a detailed program. The layout optimizer shows the same split inside a search: once the representation only produces plans that make architectural sense, a small deterministic search is enough, and it placed more rooms than the reference genetic algorithm with no guest room left without a window.

The next steps are:

- drawing sheets from the optimized layouts (L, U, T, H and courtyard plates), and a wider Level 1 podium;
- parking and structured parking;
- column-grid-aware room widths;
- more of the IBC rule set;
- majority voting over repeated LLM judgments;
- replacing the remaining synthetic sources with RNGD's data.

References

[1] James C. Bean. Genetic algorithms and random keys for sequencing and optimization. *ORSA Journal on Computing*, 6(2):154–160, 1994.

[2] Kalyanmoy Deb. *Multi-Objective Optimization Using Evolutionary Algorithms*. John Wiley & Sons, 2001.

[3] David E. Goldberg. *Genetic Algorithms in Search, Optimization, and Machine Learning*. Addison-Wesley, 1989.

[4] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585:357–362, 2020.

[5] John H. Holland. *Adaptation in Natural and Artificial Systems*. University of Michigan Press, 1975.

[6] Robert Hooke and T. A. Jeeves. "direct search" solution of numerical and statistical problems. *Journal of the ACM*, 8(2):212–229, 1961.

[7] John D. Hunter. Matplotlib: A 2D graphics environment. *Computing in Science & Engineering*, 9(3): 90–95, 2007.

[8] Ching-Lai Hwang and Kwangsun Yoon. *Multiple Attribute Decision Making: Methods and Applications*. Springer, 1981.

[9] IfcOpenShell contributors. IfcOpenShell: the open source IFC toolkit and geometry engine. https://ifcopenshell.org, 2024.

[10] International Code Council. *2021 International Building Code*. International Code Council, Country Club Hills, IL, 2020.

[11] International Organization for Standardization. ISO 16739-1:2018 industry foundation classes (IFC) for data sharing in the construction and facility management industries. ISO standard, 2018.

[12] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. Retrieval-augmented generation for knowledge-intensive NLP tasks. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

[13] Helena R. Lourenço, Olivier C. Martin, and Thomas Stützle. Iterated local search. In Fred Glover and Gary A. Kochenberger, editors, *Handbook of Metaheuristics*, pages 320–353. Kluwer Academic Publishers, 2003.

[14] Pablo Moscato. On evolution, search, optimization, genetic algorithms and martial arts: Towards memetic algorithms. Technical Report C3P Report 826, Caltech Concurrent Computation Program, 1989.

[15] National Fire Protection Association. NFPA 13: Standard for the installation of sprinkler systems. NFPA, 2022.

[16] M. E. Newell, R. G. Newell, and T. L. Sancha. A solution to the hidden surface problem. In *Proceedings of the ACM Annual Conference*, pages 443–450, 1972.

[17] H. Penny Nii. Blackboard systems: The blackboard model of problem solving and the evolution of blackboard architectures. *AI Magazine*, 7(2):38–53, 1986.

[18] OpenRouter. OpenRouter: a unified interface for large language models. `https://openrouter.ai`, 2025.

[19] Plotly Technologies Inc. Collaborative data science (plotly). `https://plot.ly`, 2015.

[20] Nils Reimers and Iryna Gurevych. Sentence-BERT: Sentence embeddings using Siamese BERT-networks. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*, 2019.

[21] RNGD. Project renegade: Living + lodging prototype, drawing set. Project data supplied to the semester team, 2025. Cover sheet space allocation and plans dated 06/20/25.

[22] RNGD. RNGD IBC 2021 R1 Five-Story AI Rules v0.2 (rule database and companion workbook). Project data supplied to the semester team, 2026. `RNGD_IBC_2021_R1_Five_Story_AI_Rules_v0_2.csv` and `.xlsx`.

[23] Streamlit Inc. Streamlit: a faster way to build and share data apps. `https://streamlit.io`, 2025.

[24] U.S. Department of Justice. 2010 ADA standards for accessible design. Department of Justice, Civil Rights Division, 2010. Supplied by RNGD as `ada_2010_standards_for_accessible_design.pdf`.

[25] Stéfan van der Walt, Johannes L. Schönberger, Juan Nunez-Iglesias, et al. scikit-image: image processing in Python. *PeerJ*, 2:e453, 2014.

[26] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: Fundamental algorithms for scientific computing in Python. *Nature Methods*, 17:261–272, 2020.

[27] Wenhui Wang, Furu Wei, Li Dong, Hangbo Bao, Nan Yang, and Ming Zhou. MiniLM: Deep self-attention distillation for task-agnostic compression of pre-trained transformers. In *Advances in Neural Information Processing Systems*, volume 33, 2020.

A Glossary

A.1 Building, codes and real estate

ADA / 2010 ADA Standards
U.S. accessibility standards under the Americans with Disabilities Act. For hotels (“transient lodging”) they set how many guest rooms must have mobility features (Table 224.2) and communication features (Table 224.4), and how those rooms are spread over room types (224.5).

Mobility features / roll-in shower
Accessible-room features for wheelchair users, such as clear floor space and 60 in turning circles; a roll-in shower has no curb.

Communication features
Features for guests who are deaf or hard of hearing, such as visual alarms and notification devices.

BaaP (Building-as-a-Product)
RNGD's approach of designing buildings from standard, repeatable, prefabricated modules rather than one-off designs.

BIM / IFC
Building Information Modelling: a 3D model whose elements carry data. IFC (Industry Foundation Classes, ISO 16739) is the open file format for exchanging BIM models; IFC2X3 and IFC4 are versions.

Program (space allocation)
The list of rooms and spaces a building must contain, with counts and areas.

Keys
Hotel guest rooms. "King" has one king bed, "QQ" two queen beds, "Suite" a separate living room, "Spa" a tub.

Net / gross area (SF, GSF)
Net is the area of the programmed rooms; gross includes corridors, walls, shafts and stairs. The net-to-gross ratio, circulation factor and wall burden convert one to the other.

Schematic design
The first design phase: massing, plans and key decisions, before design development and construction documents.

Massing
The overall 3D shape and size of the building.

Zoning district (e.g. R-4)
A local land-use category with its own limits on height, density, setbacks and parking.

Lot, frontage
The parcel of land; frontage is the side facing the street.

Setback
The minimum distance from a property line to the building.

Easement
A strip of land others have rights over (e.g. utilities); usually not buildable.

Buildable envelope
The area left for building after setbacks and easements.

FAR (floor area ratio)
Total gross floor area divided by lot area.

Lot coverage
Building footprint as a percentage of lot area.

Parking ratio, stall, drive aisle
Required stalls per unit; a stall is one parking space (9×18 ft here); a drive aisle is the lane serving stalls (24 ft for two-way).

Fire lane / fire apparatus access road
A clear route for fire trucks (20 ft wide in the sample code).

IBC
The International Building Code: the model building code adopted by most U.S. jurisdictions.

Occupancy group R-1 / R-2
R-1 is transient residential (hotels); R-2 is permanent residential (apartments).

Construction type IA, IIA, IIB
Fire-resistance classes. Type I and II are noncombustible; "A" has fire-resistance-rated elements and "B" does not. Type IA has the highest rating and is used for podiums.

Podium (IBC 510.2)
A Type IA base separated horizontally from the building above, so the upper part can count as a separate building for height and story limits.

Story above grade plane
A story whose floor is at or above the ground reference plane. Level 1 counts, which is why six levels exceed a five-story limit.

Sprinklered (NFPA 13)
Protected by an automatic sprinkler system to NFPA 13 [15]; this raises allowable heights and travel distances.

High-rise
A building with an occupied floor more than 75 ft above fire-department access.

Means of egress
The path out of a building: exit access, the exit (e.g. a stair enclosure) and exit discharge.

Exit access travel distance
The longest walk from any point to an exit (250 ft for sprinklered R occupancies).
Dead-end corridor
A corridor part from which only one direction leads to an exit (50 ft maximum, sprinklered R-1).
Exit separation
Two exits must be at least one third of the floor's diagonal apart (sprinklered).
Occupant load
The design number of people, from area divided by an occupant load factor (IBC Table 1004.5).
Double-loaded corridor
A corridor with rooms on both sides.
Core
The vertical services: stairs, elevators and shafts.
Wing
A linear part of a building, here a double-loaded corridor with its rooms.
Floor plate / floor-to-floor height
One floor's layout; the vertical distance between floor levels.
Column grid, bay, gridline bubble
The structural grid; a bay is the space between gridlines; bubbles label gridlines on drawings.
Plan, elevation, axonometric, enlarged plan
A plan is a horizontal cut seen from above; an elevation is a facade seen straight on; an axonometric is a 3D view without perspective; an enlarged plan shows one room at a larger scale.
Sheet set, title block
Drawings numbered by discipline (G general, A architectural); the title block carries the project, sheet name, date and number.
Prefab panel / panelization
Building parts made off site as panels; the elevations show an indicative panel breakdown at gridlines and floor lines.
BOH / FOH
Back of house (staff and service areas) and front of house (guest areas).
MEP
Mechanical, electrical and plumbing systems.
White box retail
Tenant space delivered as a basic shell for the tenant to fit out.
RFP
Request for proposal: the customer's written request.

A.2 AI and software

LLM
Large language model: a neural network that generates text, used here to read requests and documents and to make judgments.
Agent
An LLM-backed component with one task, fixed inputs and a fixed output schema.
Blackboard architecture
Components cooperate by reading and writing one shared state object [17].
RAG (retrieval-augmented generation)
Retrieving relevant passages and putting them in the prompt so the answer rests on documents [12].
Embedding, cosine similarity
A vector representing a text's meaning; cosine similarity measures how close two vectors point.
Chunk
A short passage of a document stored in the knowledge base.
Hallucination
A confident model output not supported by its inputs or sources.
JSON schema / pydantic
A strict description of the fields an output must have; pydantic validates outputs against it.
Deterministic
Same inputs always give the same output.
Painter's algorithm
Drawing surfaces from far to near so nearer ones cover farther ones [16].

Semi-implicit Euler
A time-stepping method that updates velocity first, then position with the new velocity.

Genetic algorithm (GA)
A search that keeps a population of candidate solutions and improves it by selection, crossover (mixing two candidates) and mutation (small random changes) [5]. A memetic GA also improves each candidate by local search.

Genome, decoder
The genome is the list of numbers the search changes; the decoder turns a genome into a full layout.

Fitness, objective
The score the search maximises; here a weighted mean of ten objectives, each between 0 and 1.

Pattern search, iterated local search
Pattern search tries small changes one at a time and keeps the first that improves the score. Iterated local search restarts it from a perturbed copy of the best solution.

Pareto front
The solutions that no other solution beats on every criterion at once.

TOPSIS
A ranking method: the best alternative is the one closest to the ideal and farthest from the worst on the weighted criteria [8].

Spine, bay
In the optimizer, a spine is a straight corridor line; a bay is one module of room frontage along it.

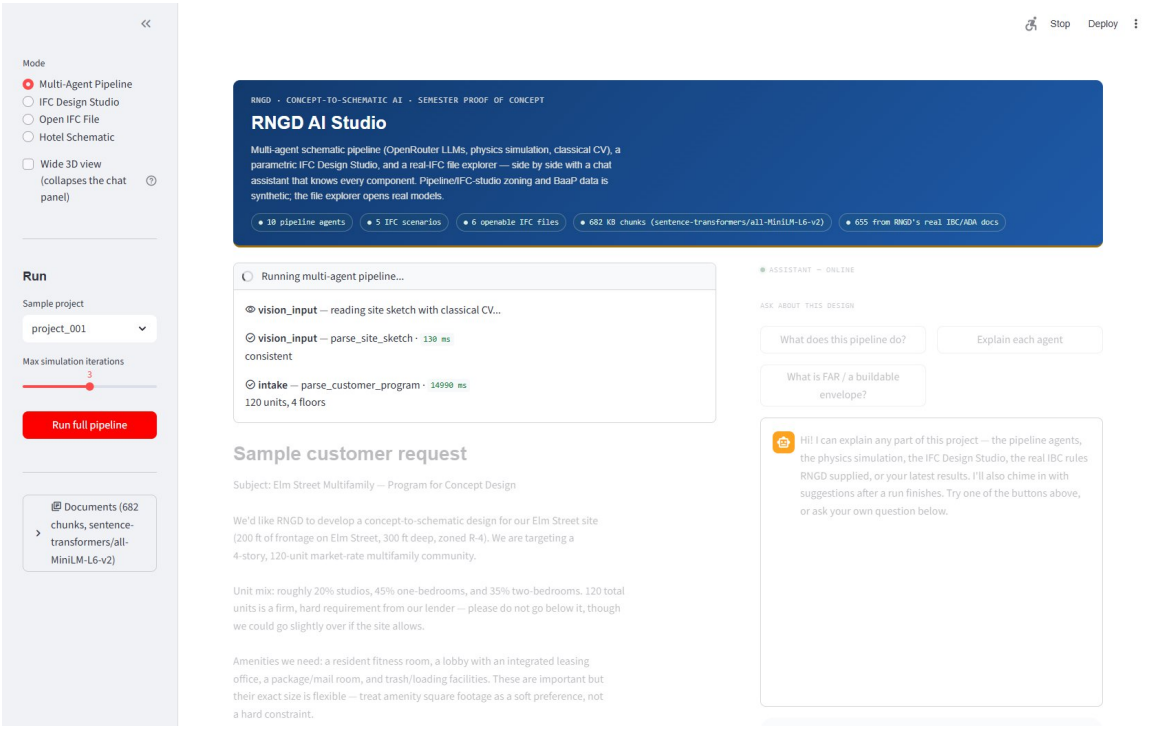
B Screenshot gallery

Every screen captured in the browser run (2026-10-05), in order, with the caption recorded while it was taken. Screens 01–02 come from a later rerun of the pipeline; screens 10, 16 and 17 were retaken with the wide 3D view.

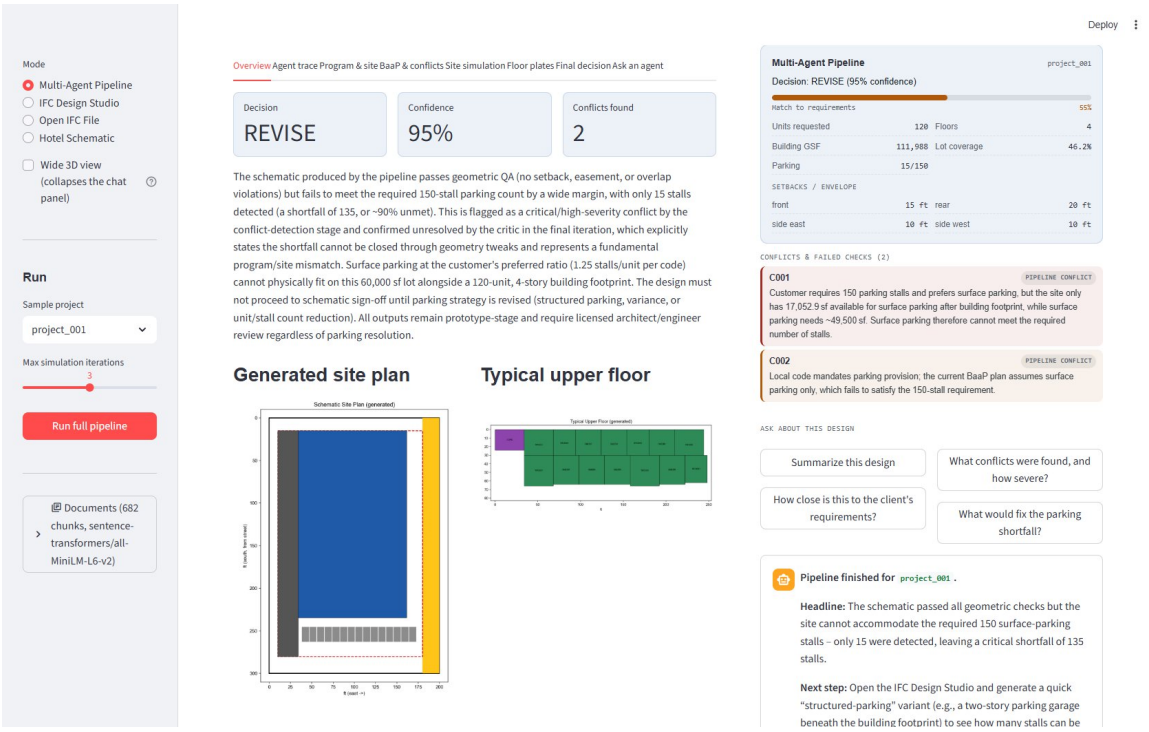
Multi-Agent Pipeline



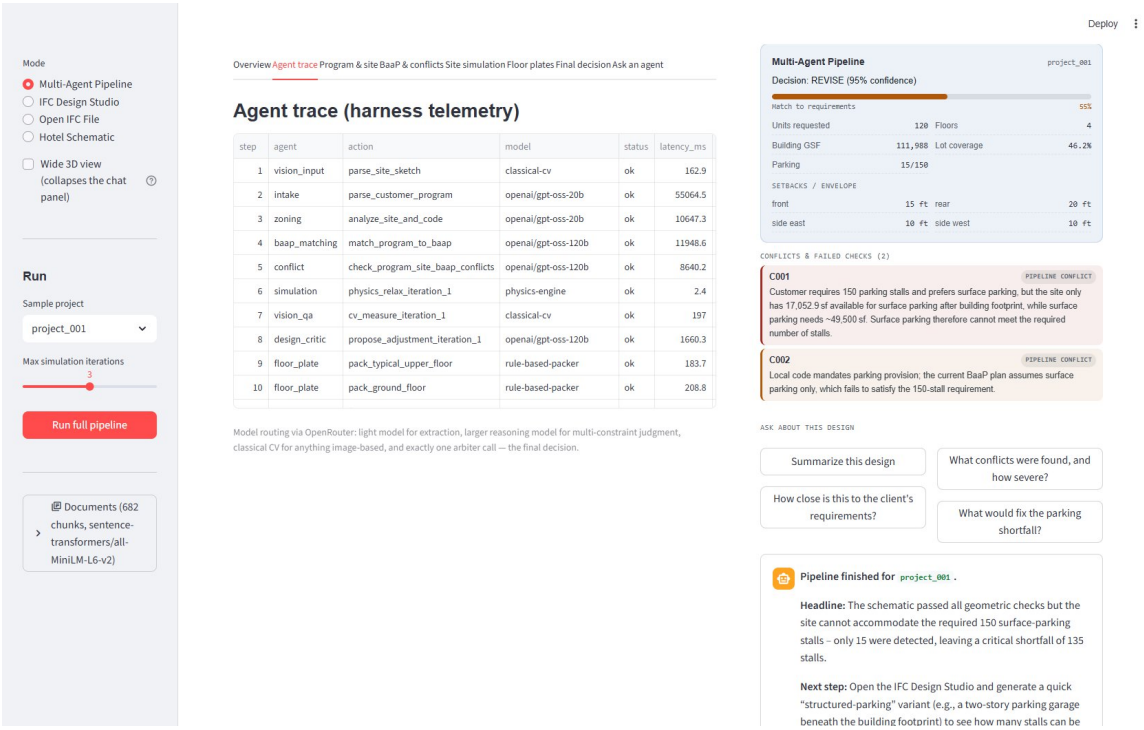
Screen 01. Multi-Agent Pipeline before a run: the sample customer request and site sketch that feed the agents; the assistant panel on the right.



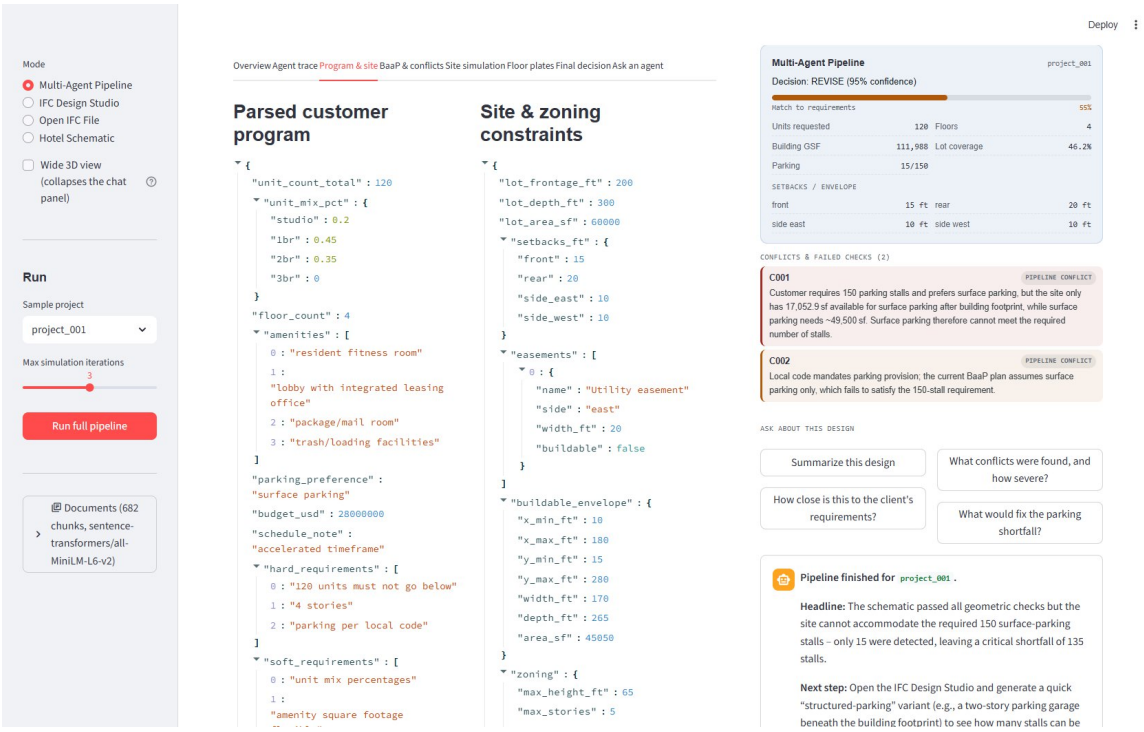
Screen 02. The pipeline running live: each agent reports its step as it finishes.



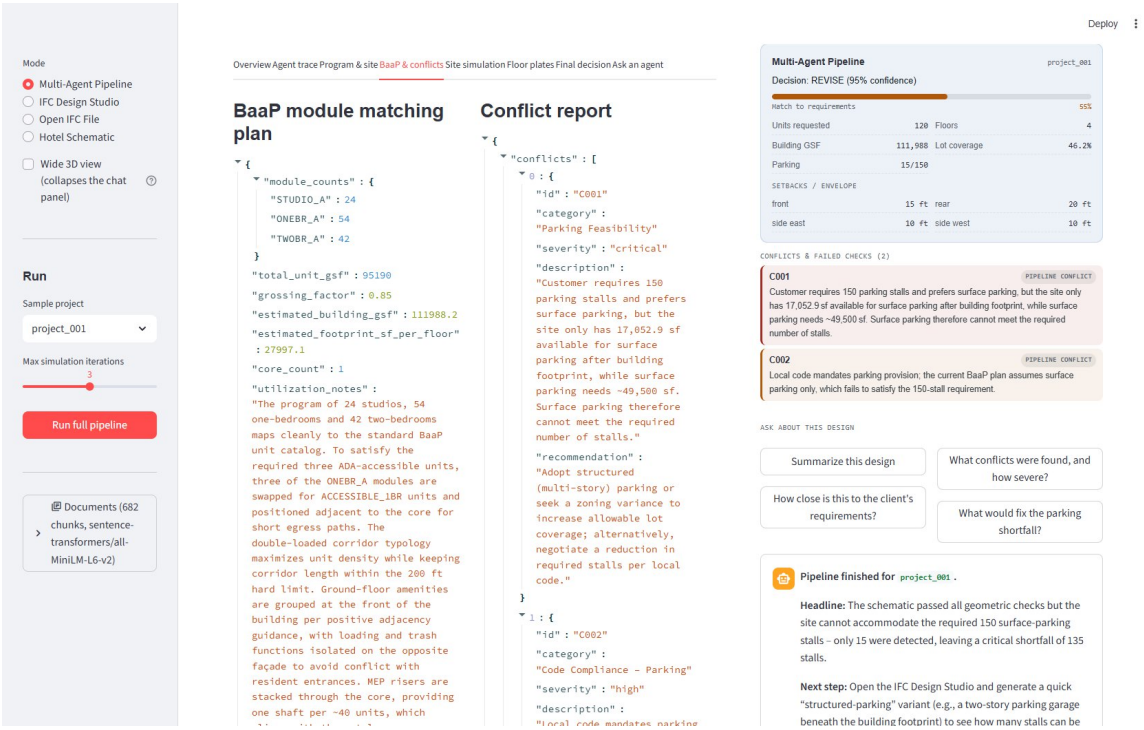
Screen 03. Pipeline result overview: the arbiter's decision, confidence and conflict count.



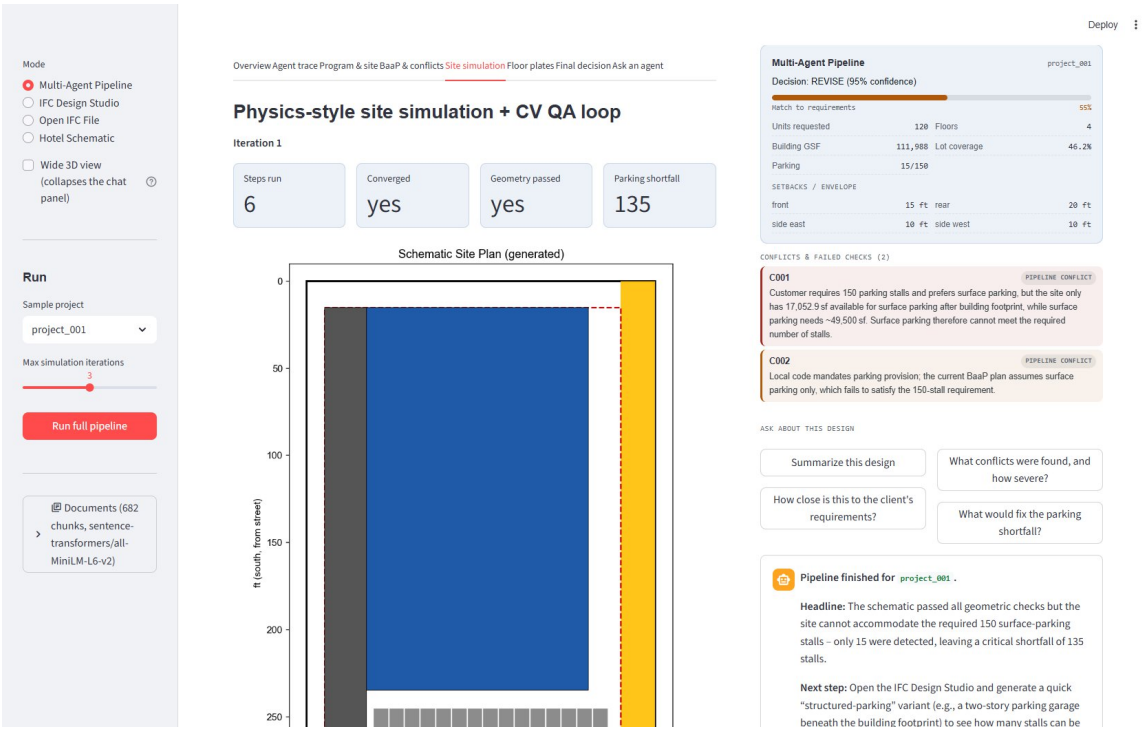
Screen 04. Agent trace: every agent step with model, status and latency.



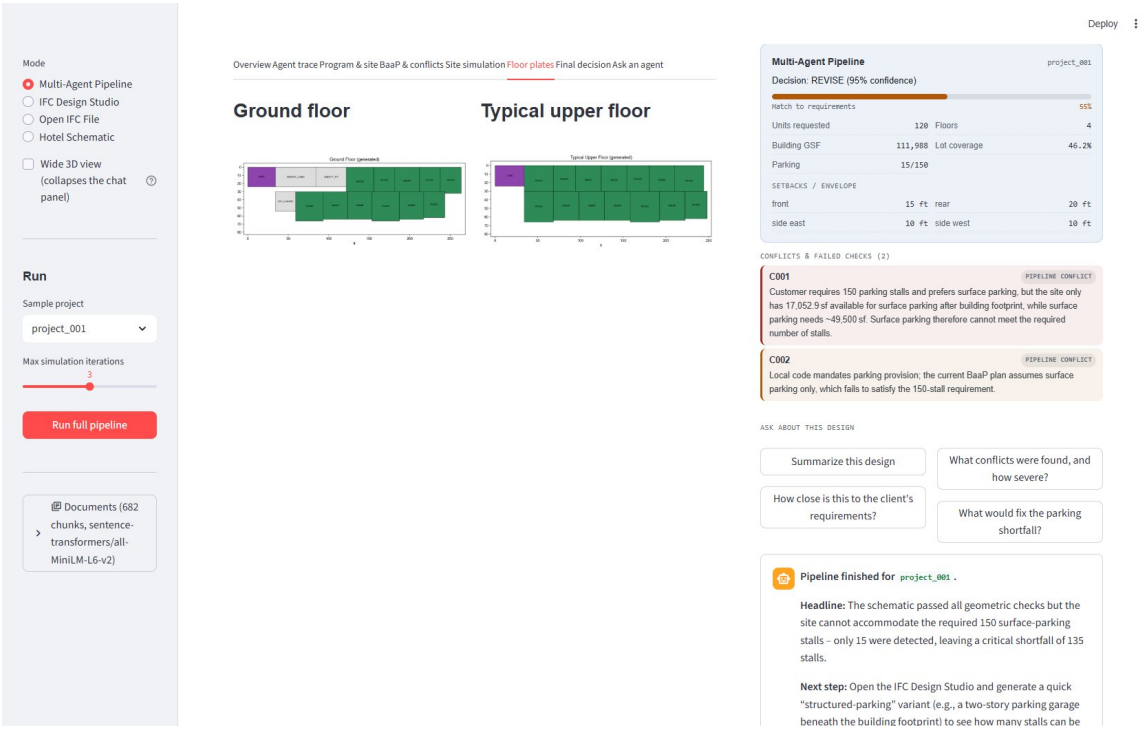
Screen 05. Program parsed from the request and the computed site constraints.



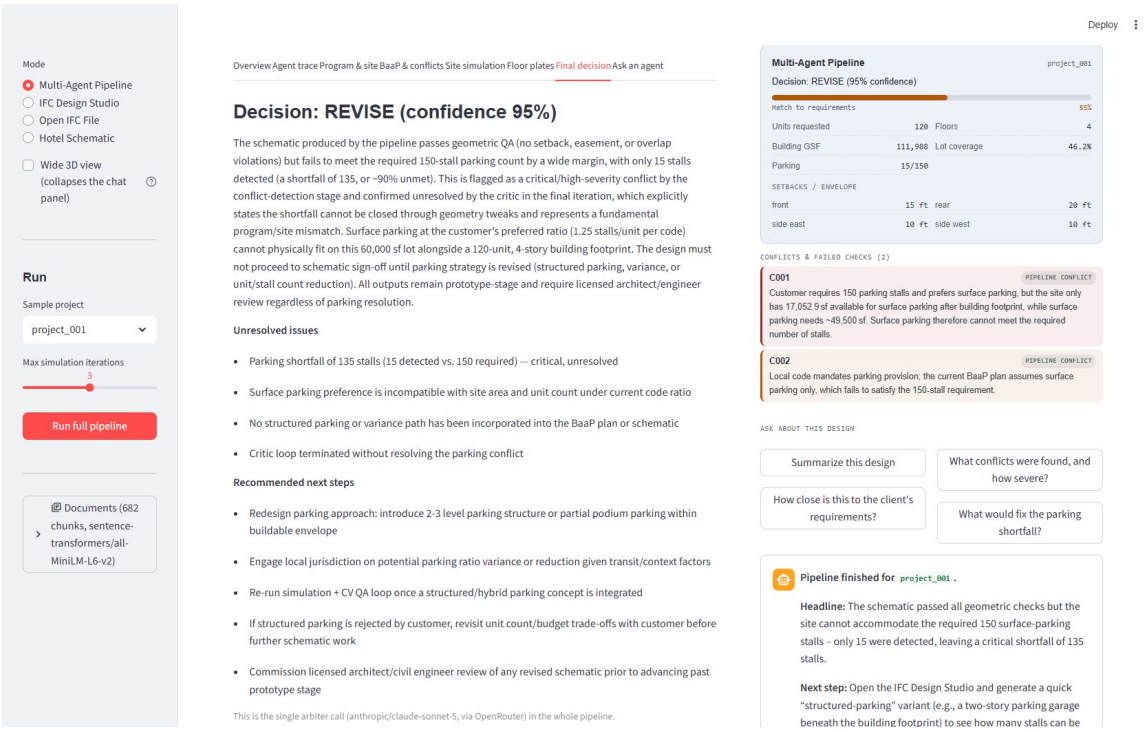
Screen 06. BaaP module counts and the conflicts the Conflict Agent found.



Screen 07. Physics-style site simulation and the computer-vision QA of the rendered plan.

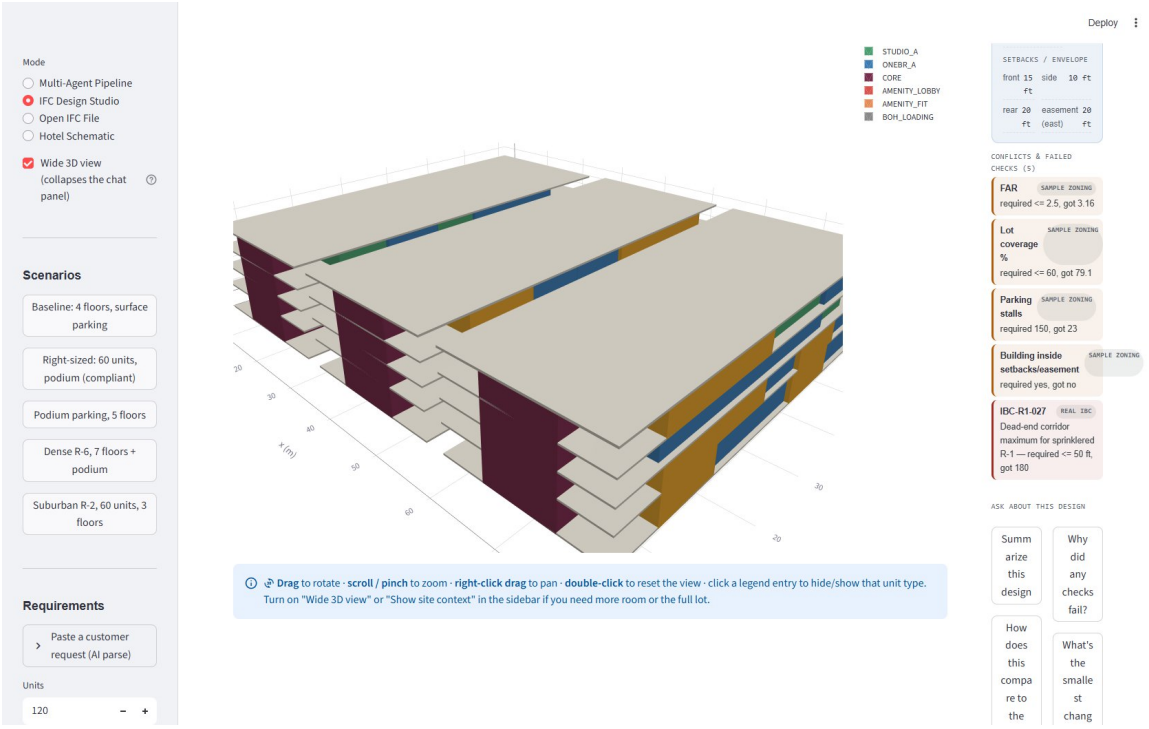


Screen 08. Rule-based floor plates (ground and typical floor).

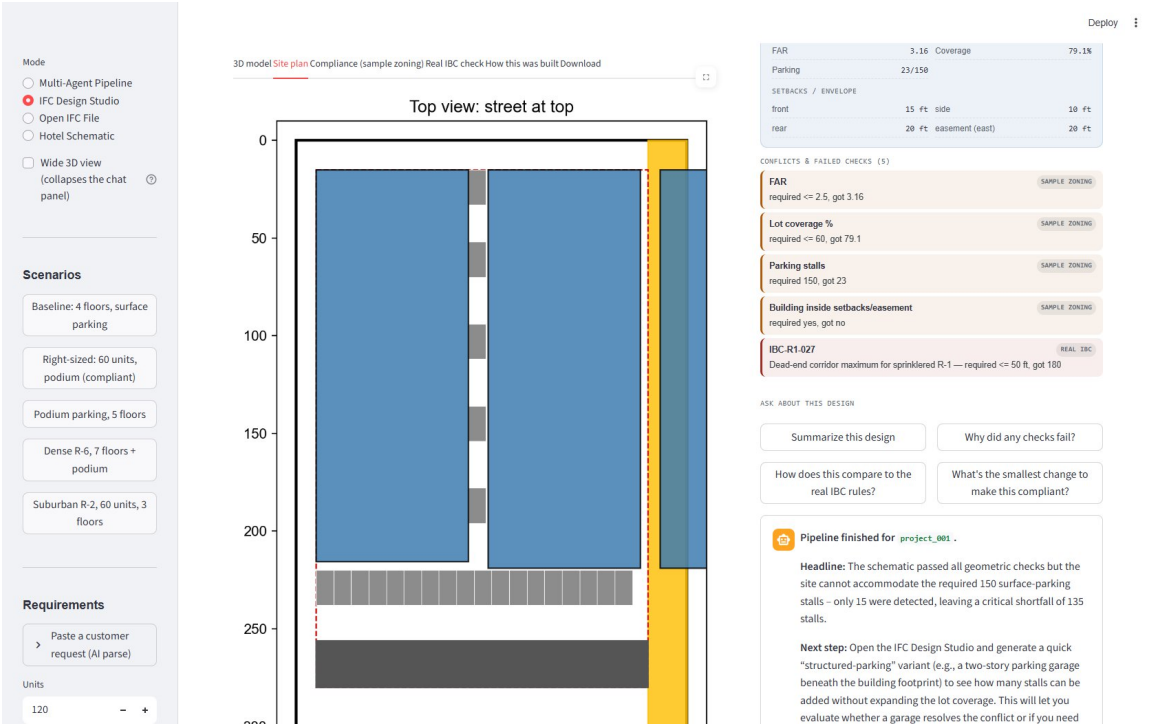


Screen 09. The arbiter's final decision, unresolved issues and next steps.

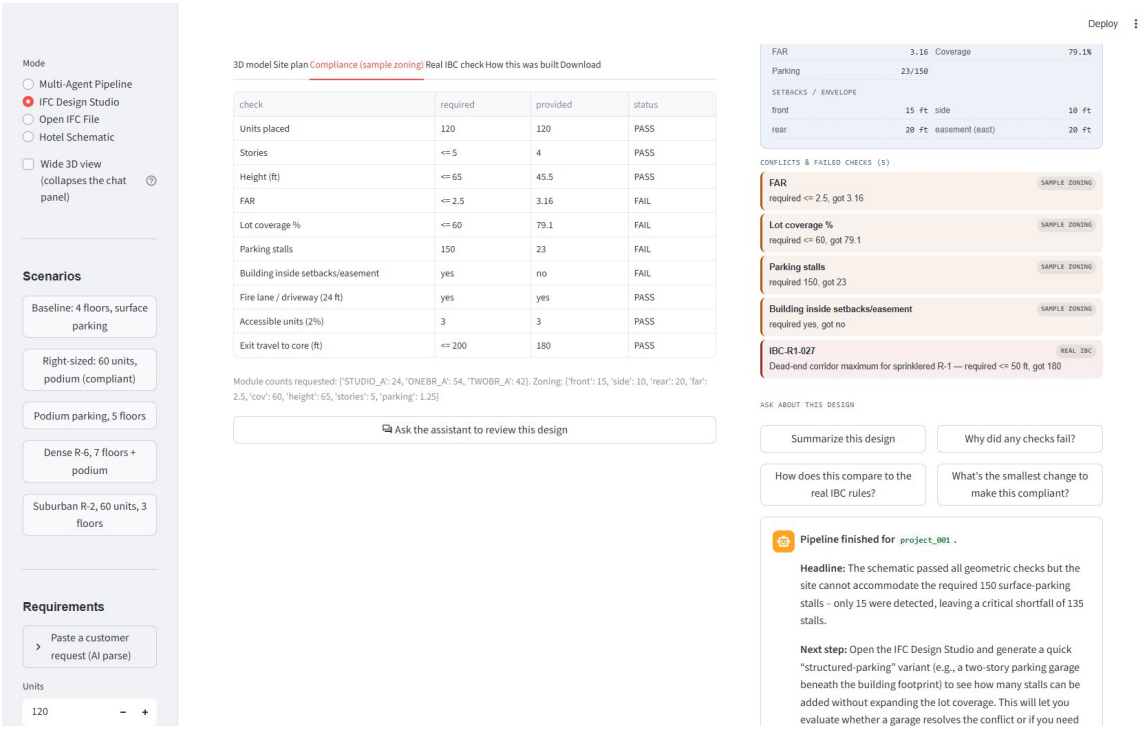
IFC Design Studio



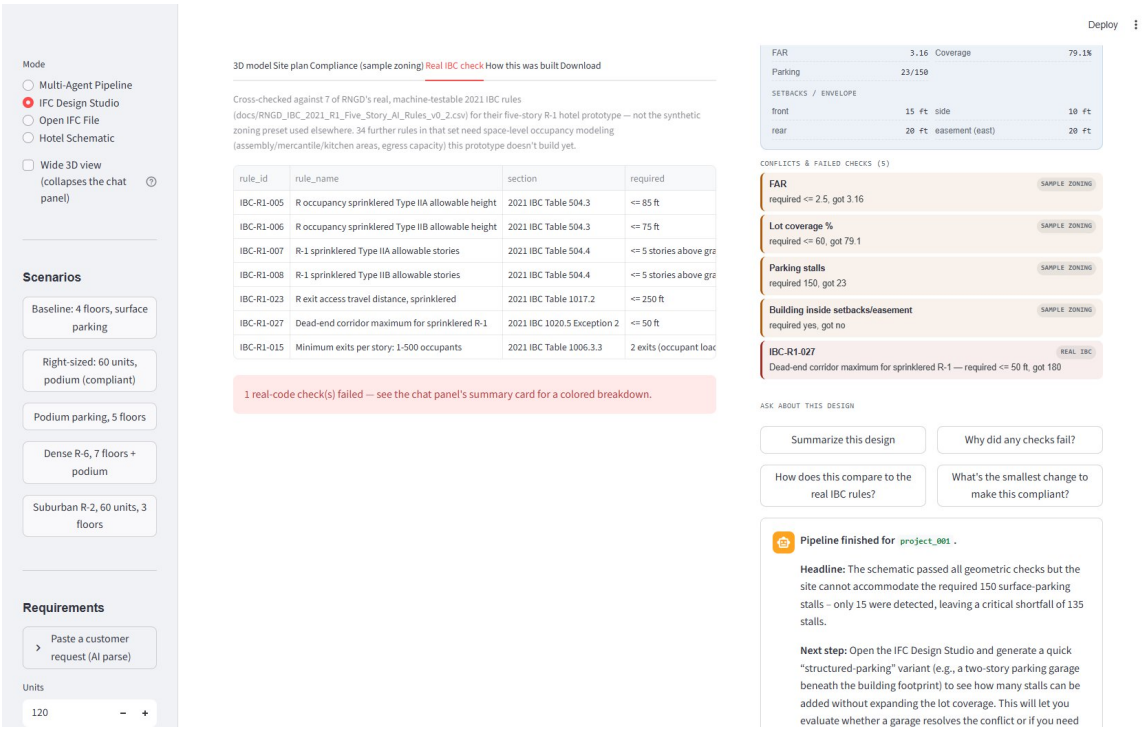
Screen 10. IFC Design Studio, Baseline scenario (120 units, 4 floors, surface parking): the generated IFC4 model in the 3D viewer.



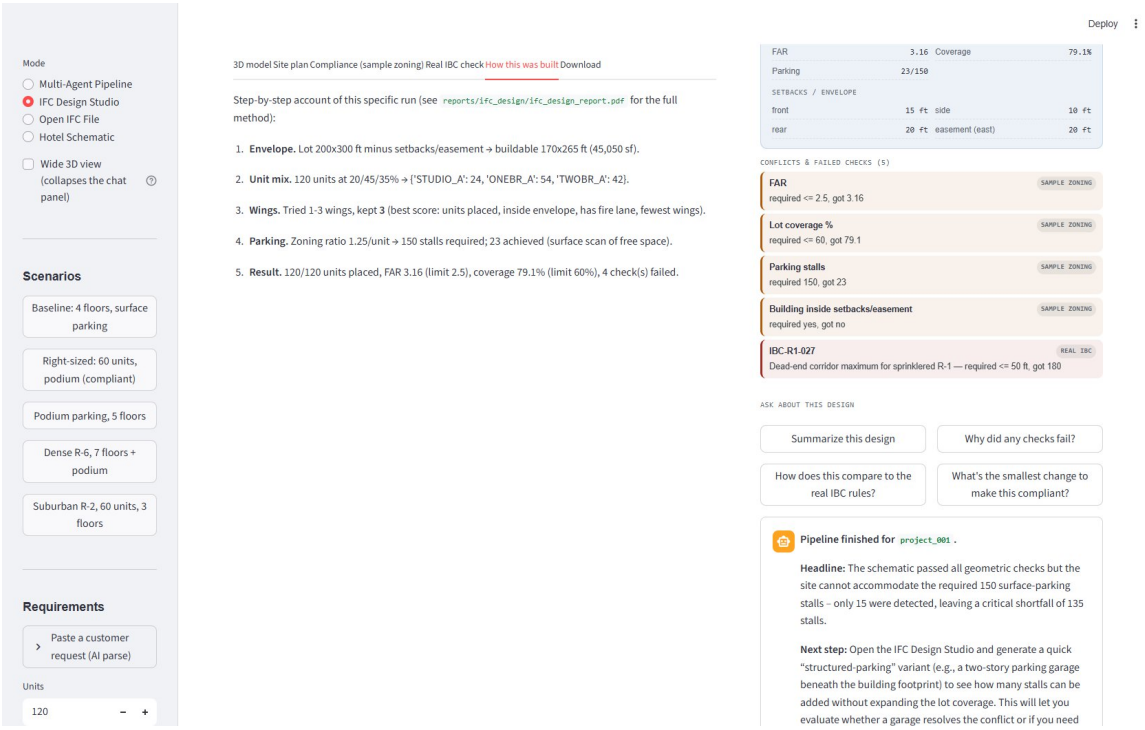
Screen 11. Baseline site plan: building, parking, easement and fire lane on the synthetic lot.



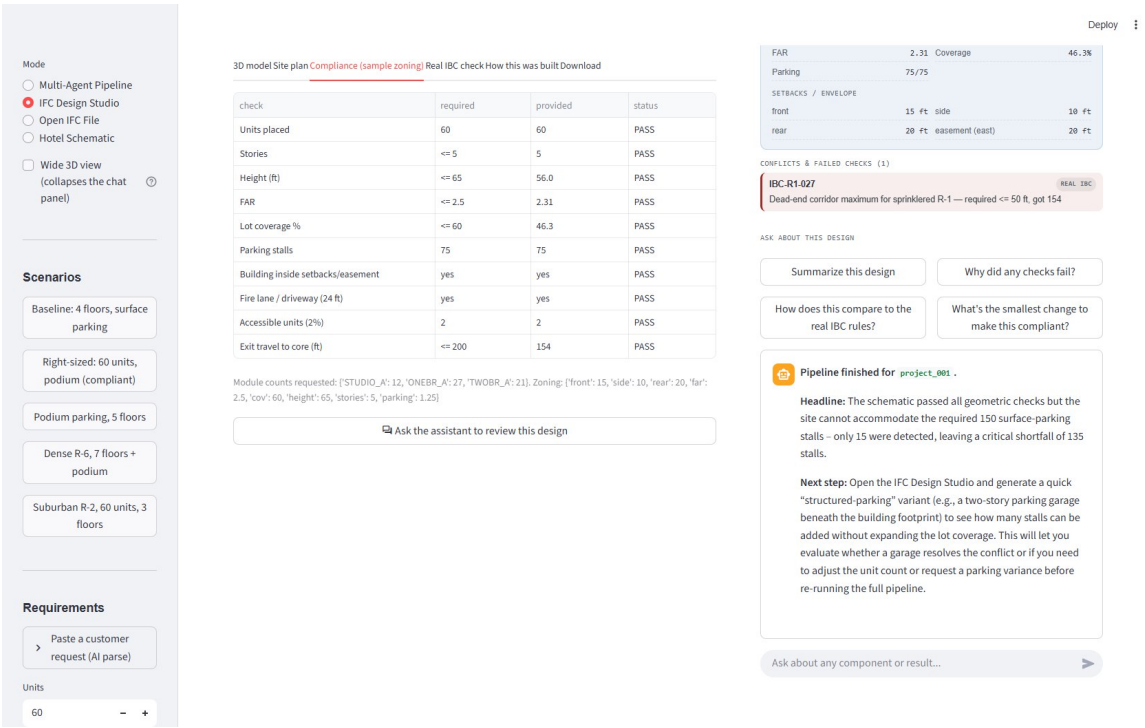
Screen 12. Baseline compliance against the ten sample-zoning checks.



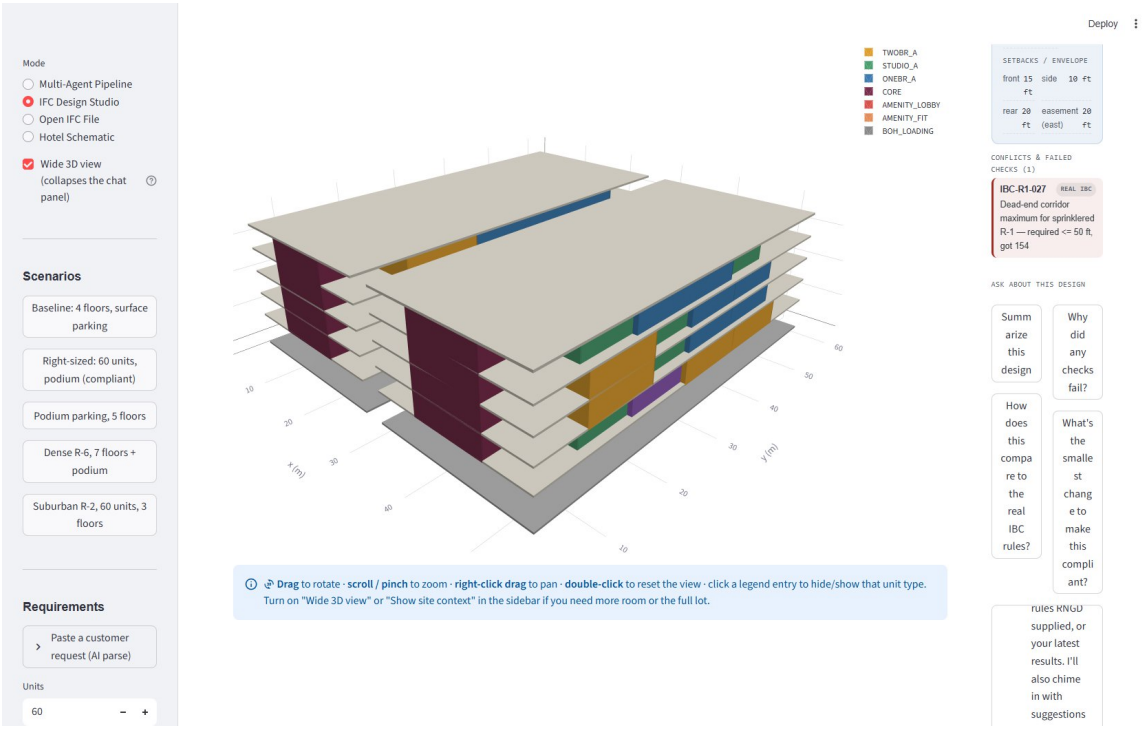
Screen 13. Baseline cross-checked against RNGD's real 2021 IBC rules.



Screen 14. Step-by-step explanation of how the layout was generated.

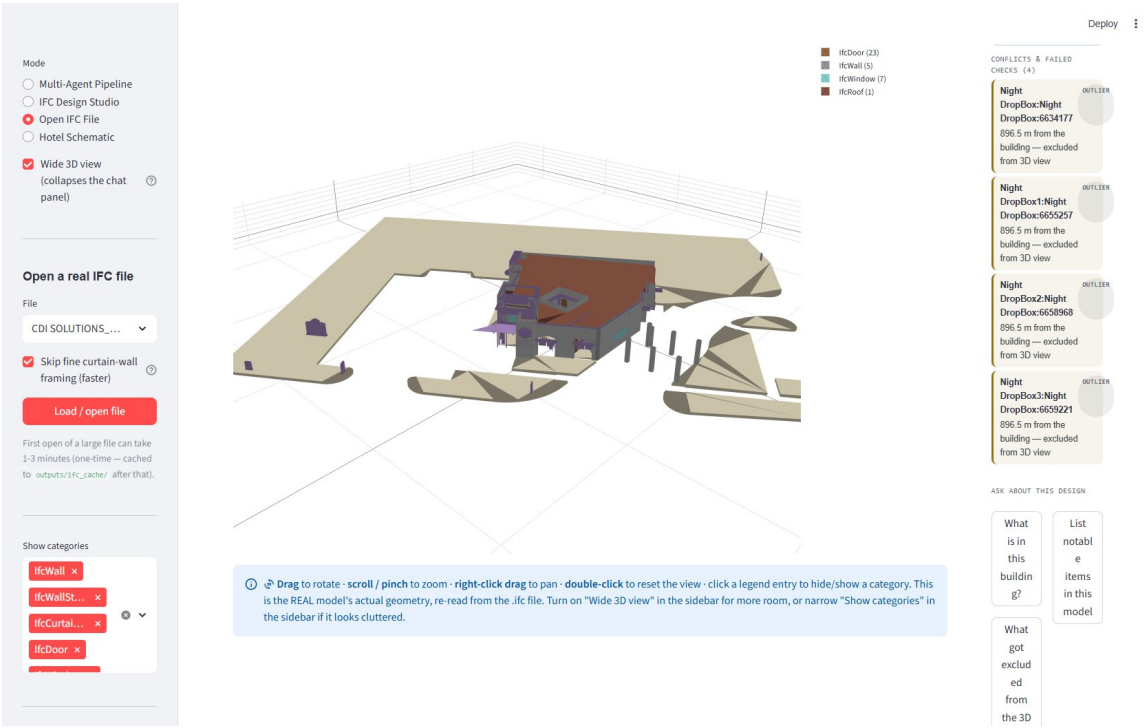


Screen 15. Right-sized scenario (60 units, podium parking): the compliance checks after the fix.

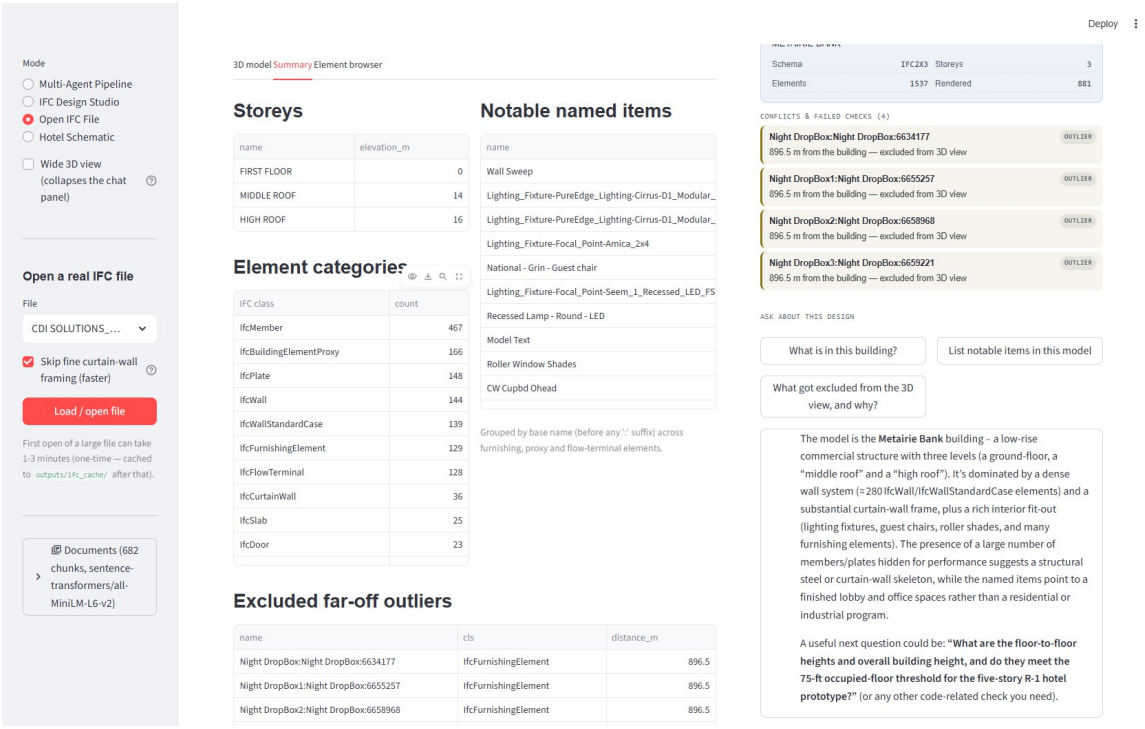


Screen 16. Right-sized scenario in 3D.

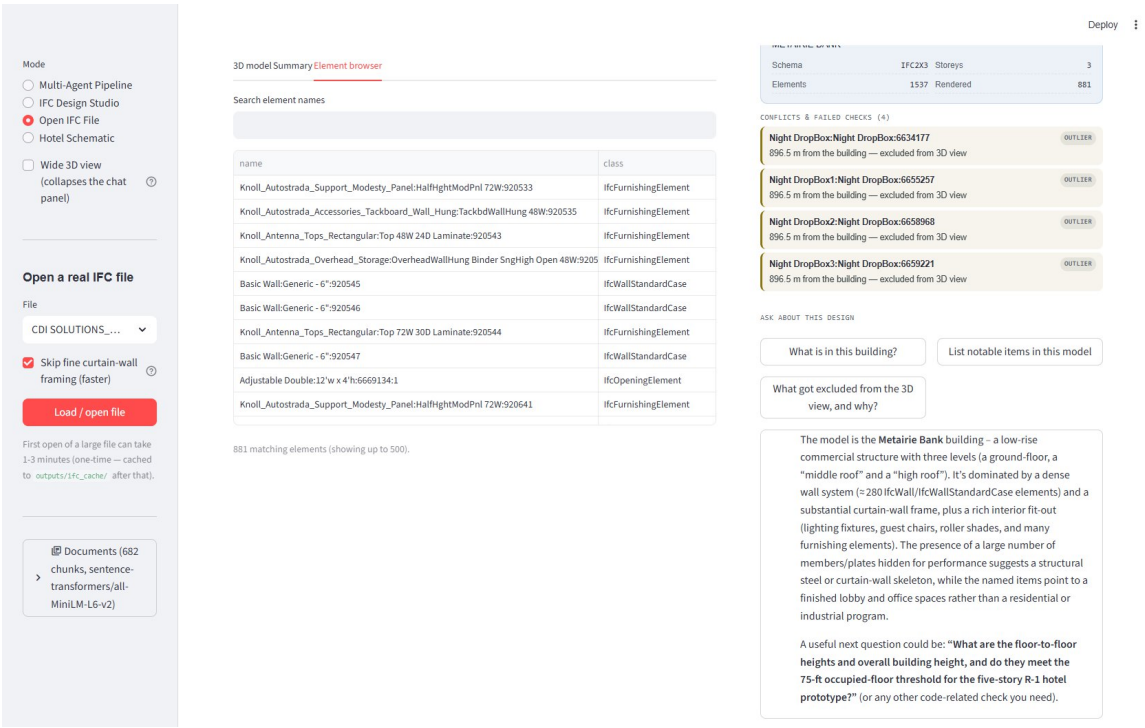
Open IFC File



Screen 17. A real IFC file (CDI Solutions, Metairie Bank, IFC2X3, 89 MB) recentred and shown in 3D.

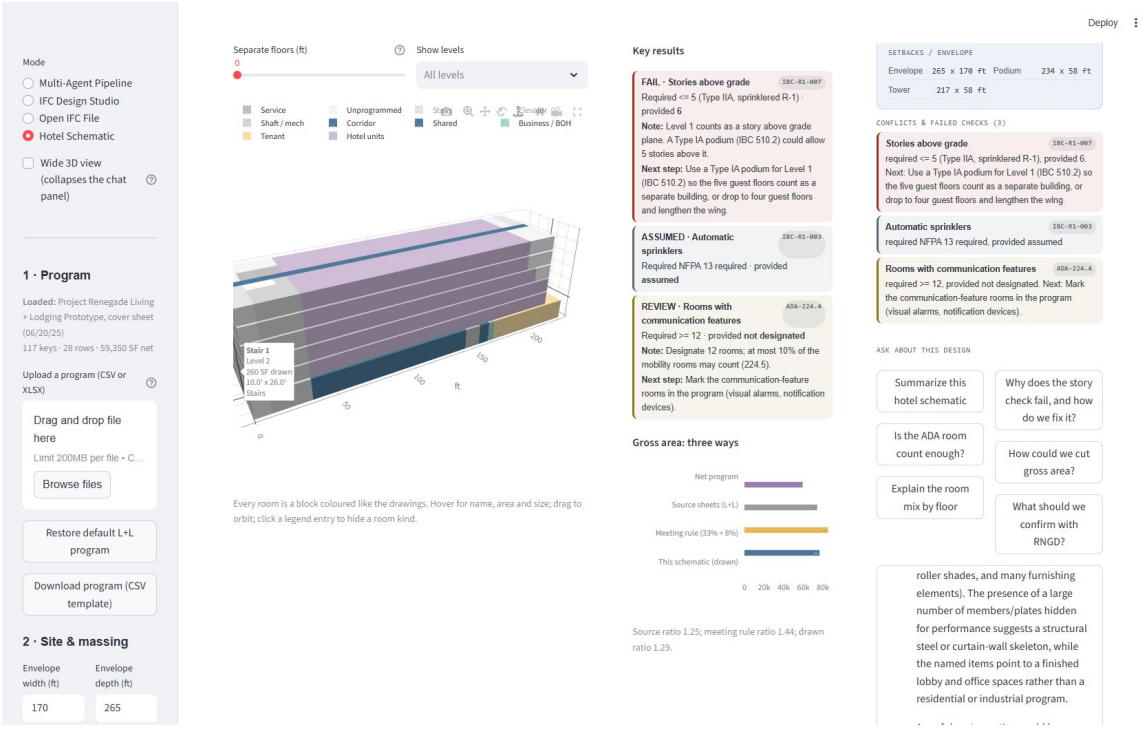


Screen 18. What is in the real model: schema, storeys and element counts by IFC class.

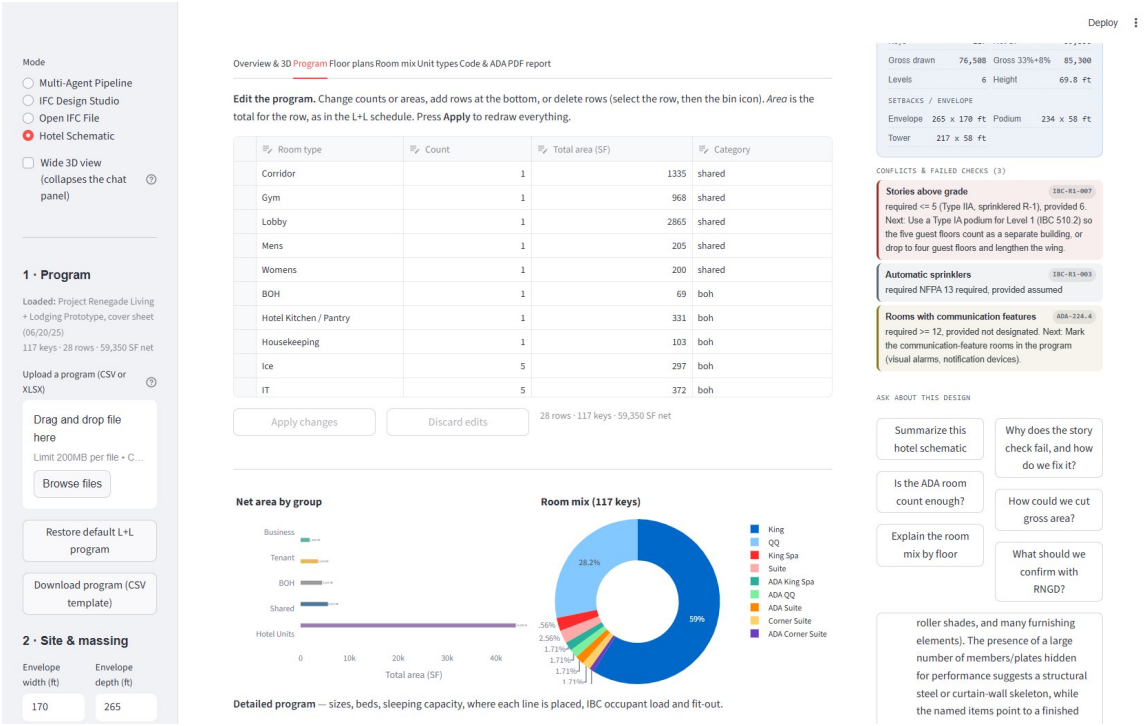


Screen 19. Searchable list of the model's elements.

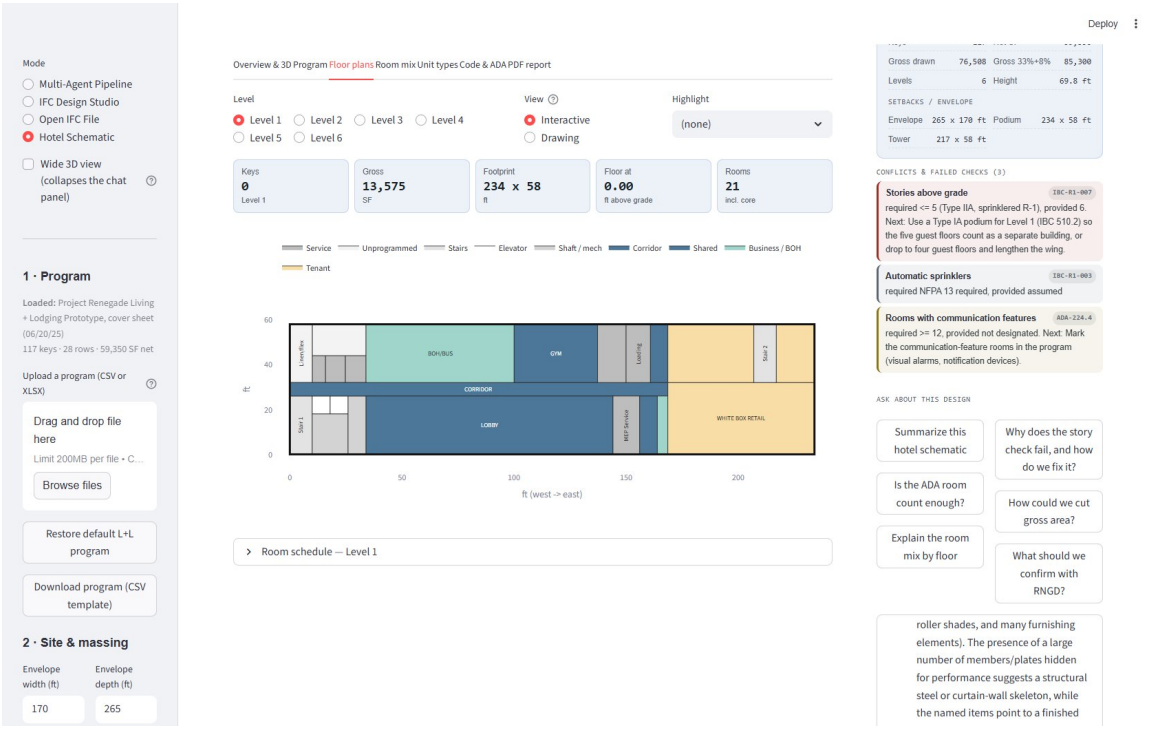
Hotel Schematic



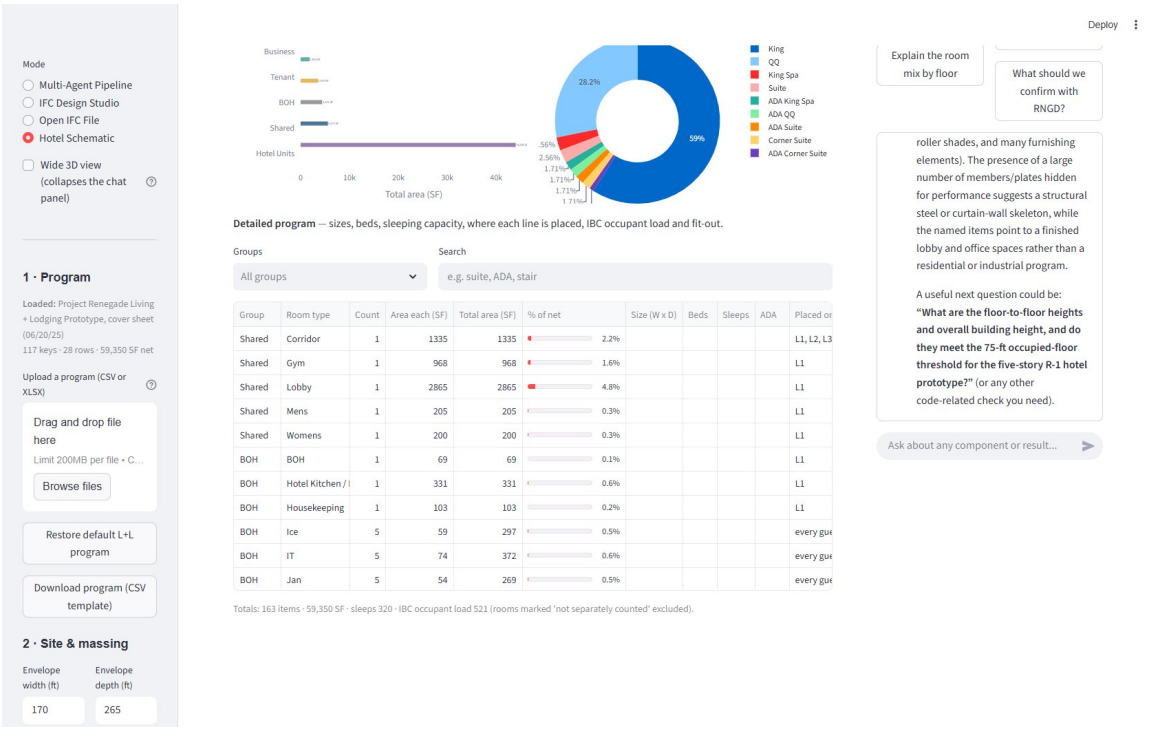
Screen 20. Hotel Schematic workspace: KPI tiles, the room-by-room 3D model and the key results with suggested fixes.



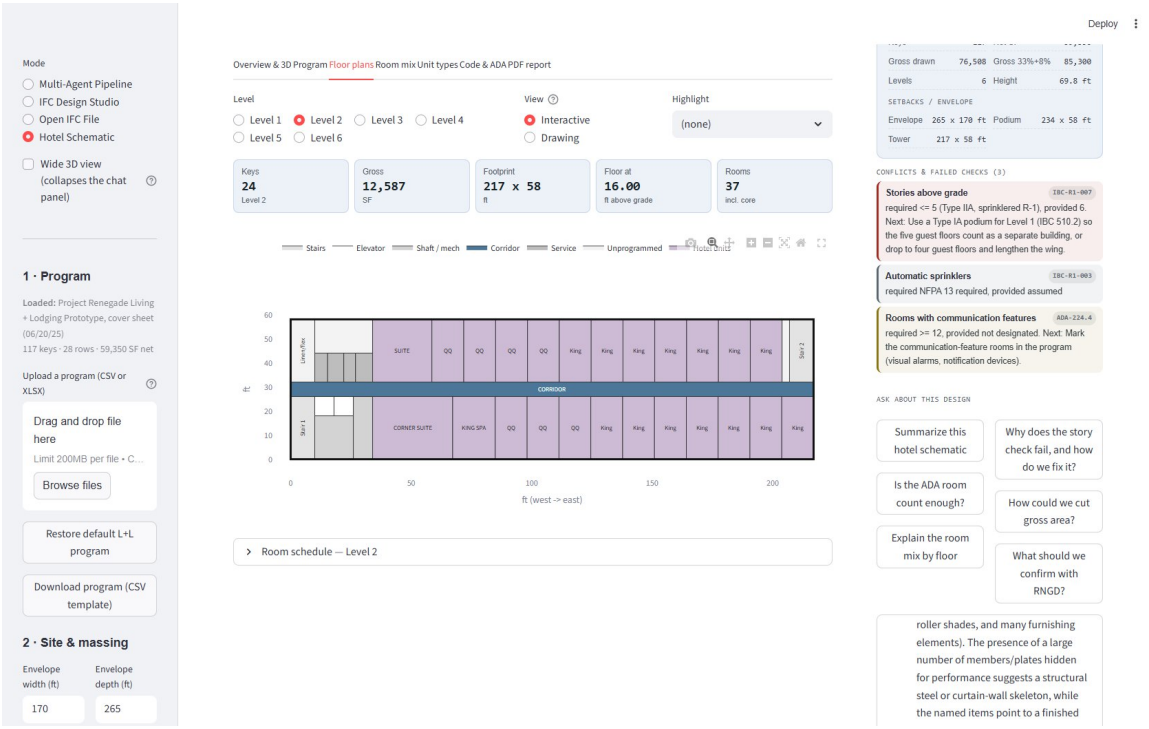
Screen 21. The editable program (Project Renegade L+L space allocation, 28 rows) with area and room-mix charts.



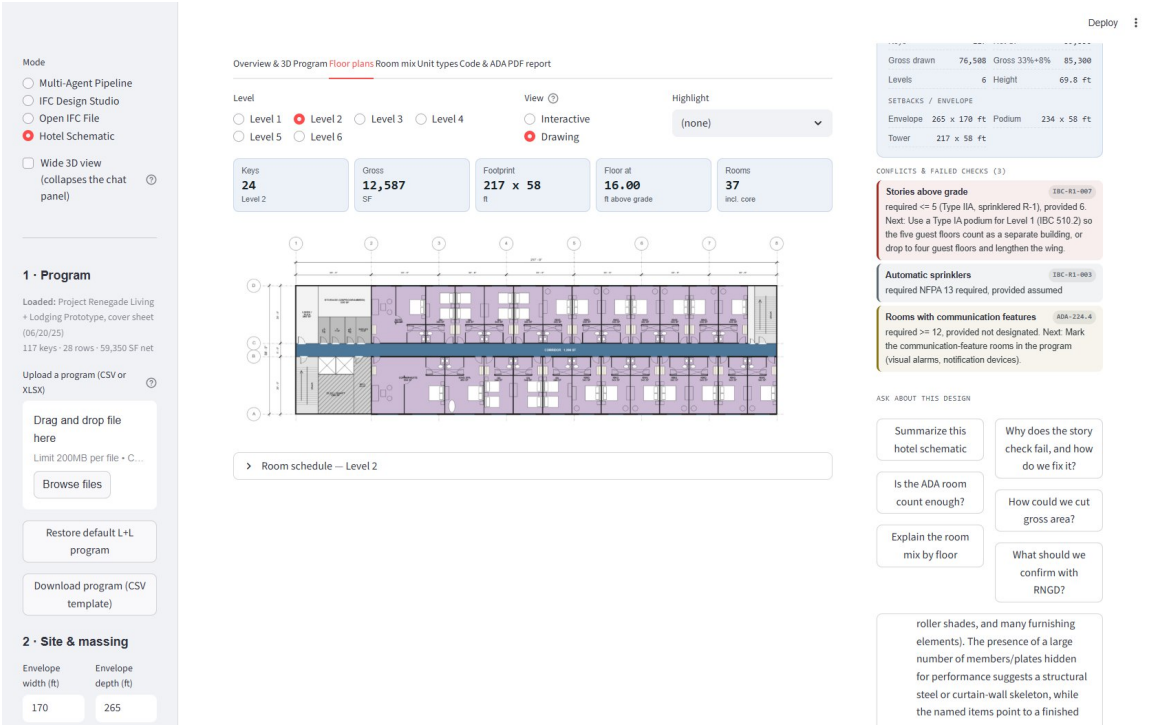
Screen 22. Interactive floor plan of Level 1 (hover shows each room's area and size).



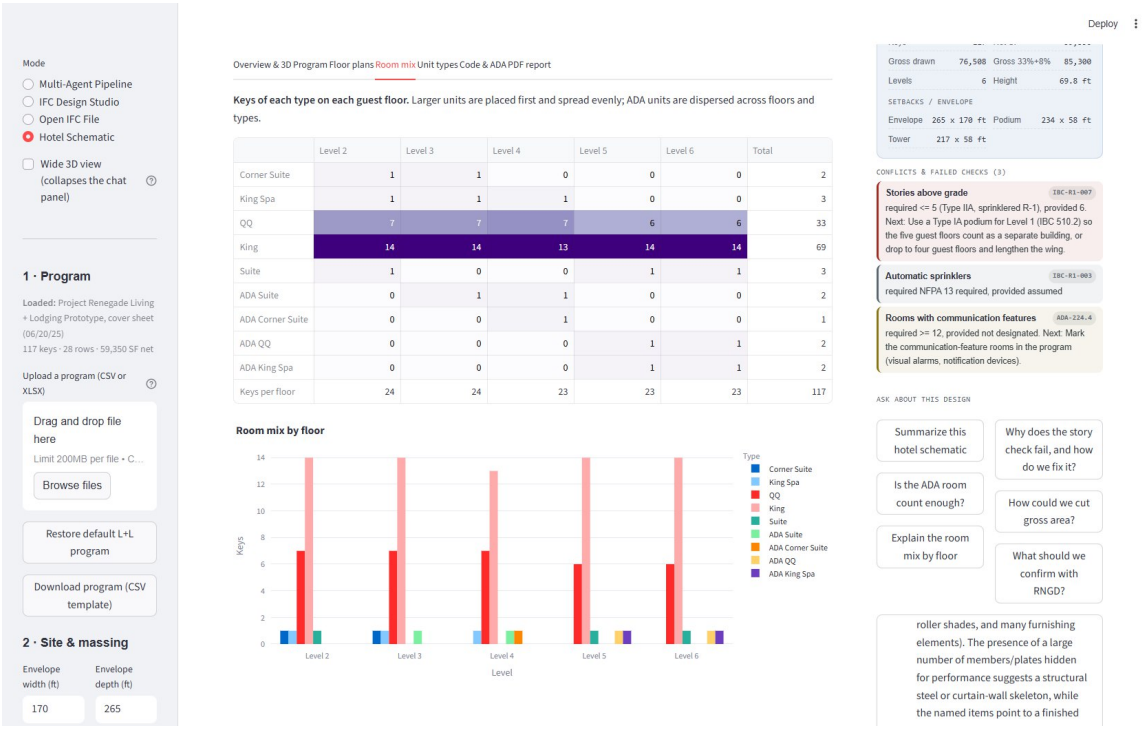
Screen 23. Detailed program: per-room size, beds, sleeping capacity, placement, IBC occupant load and fit-out.



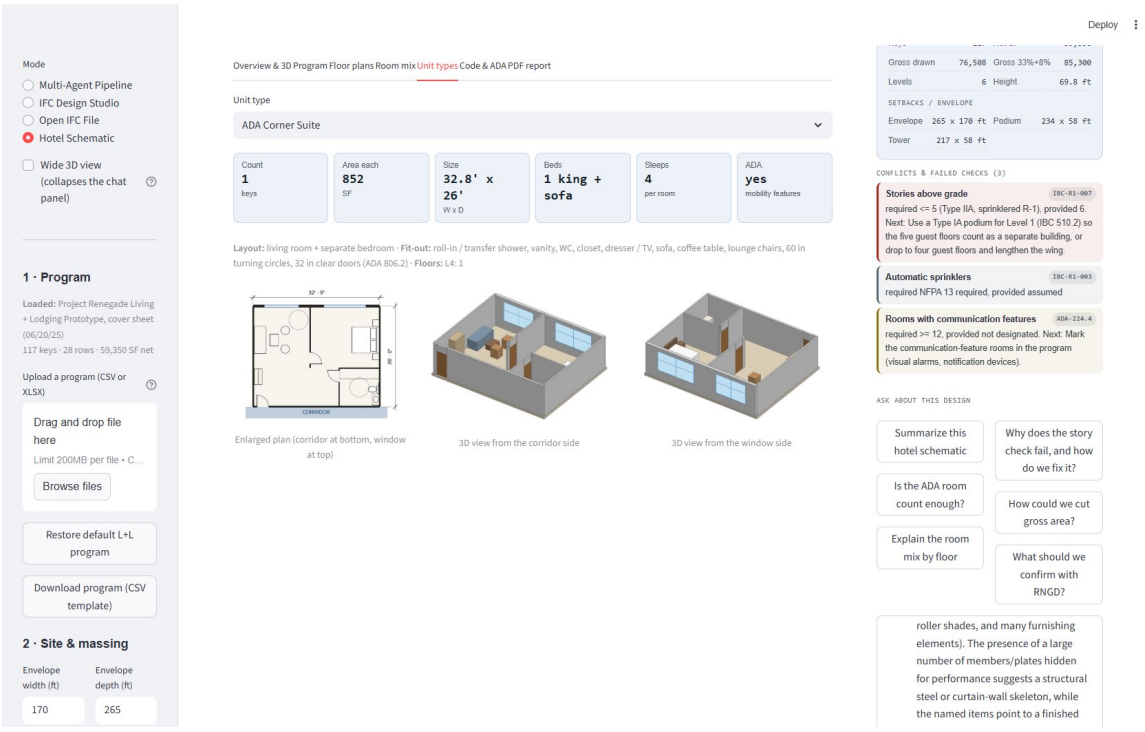
Screen 24. Interactive plan of Level 2, a typical guest floor (24 keys).



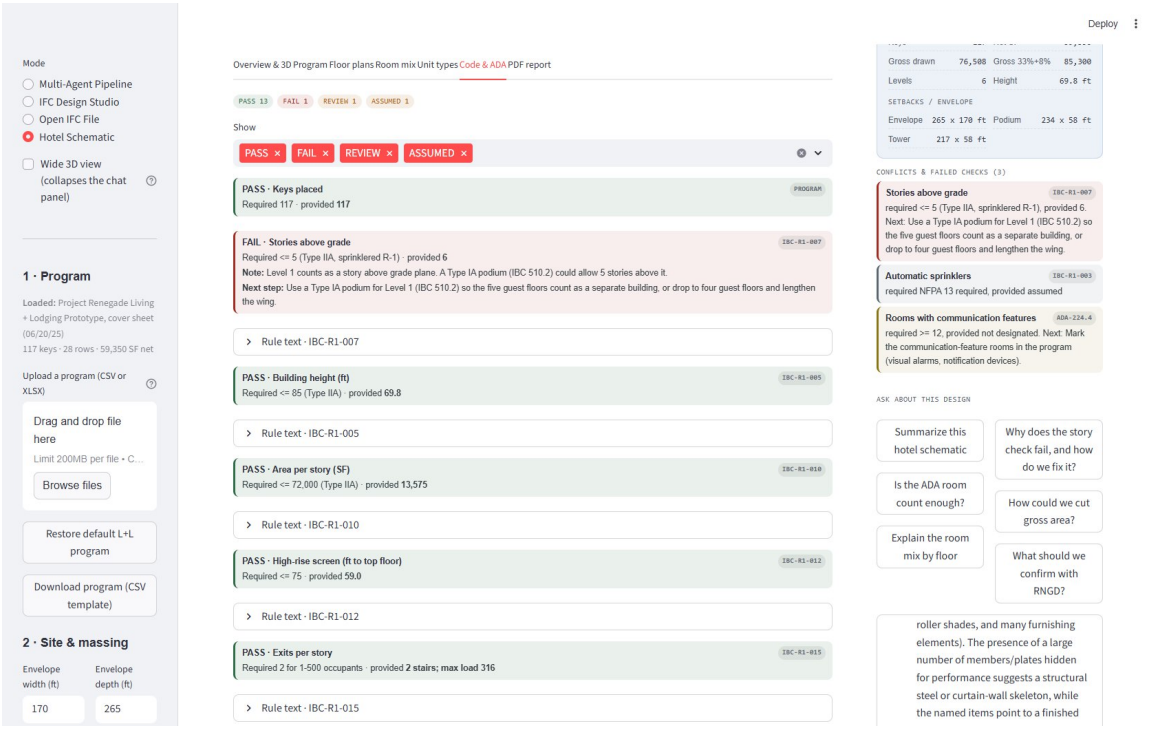
Screen 25. The same level in drawing style (as used on the PDF sheets).



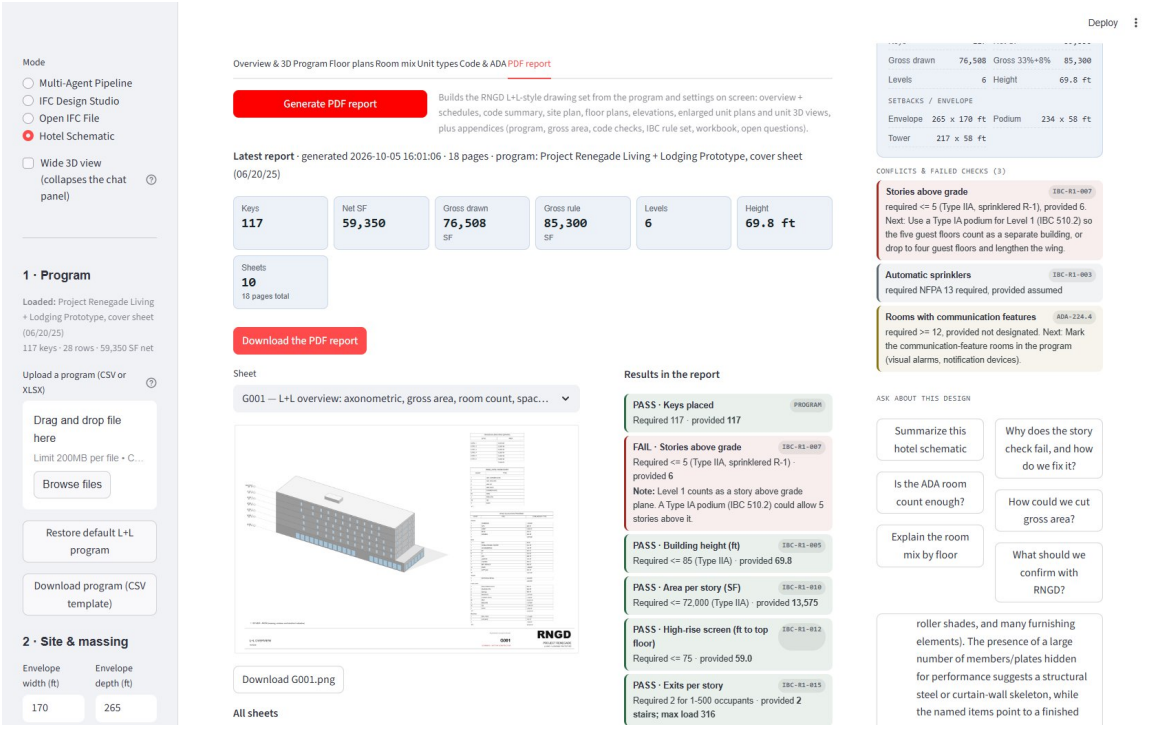
Screen 26. Keys of each room type on each guest floor.



Screen 27. Unit type detail: enlarged plan and two 3D interior views.



Screen 28. Code screen against RNGD's IBC rule set and the 2010 ADA guest-room tables, with rule text.

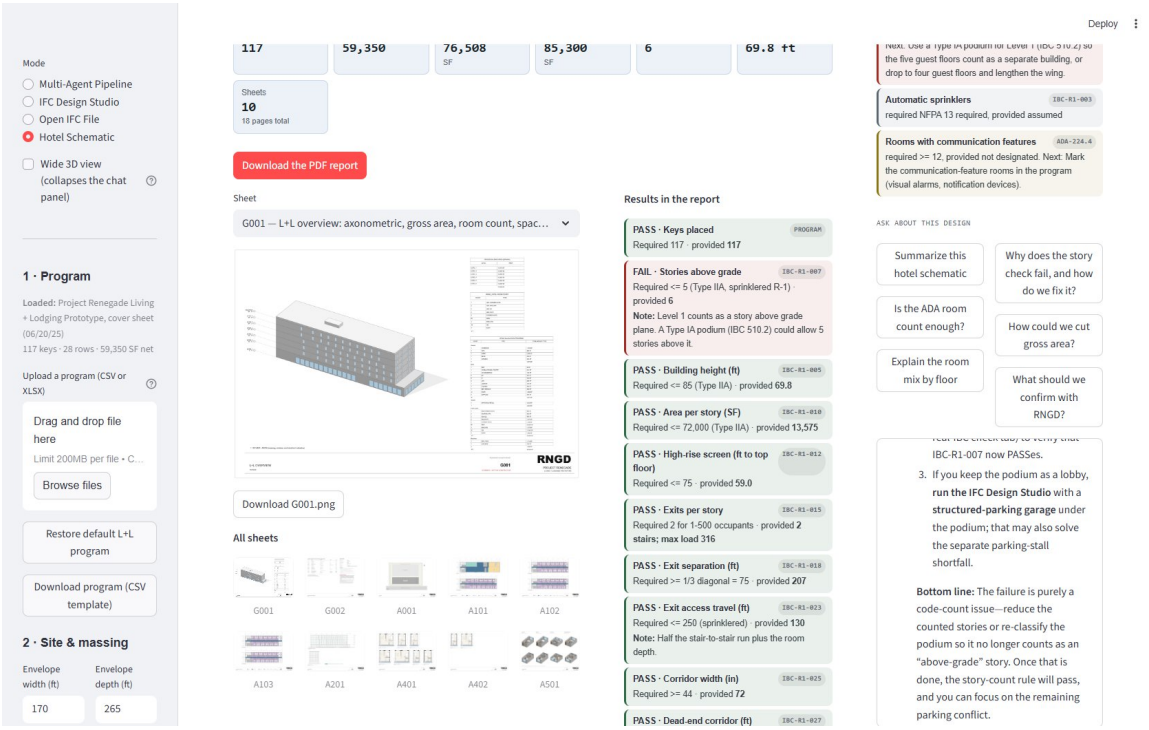


Screen 29. PDF report tab after generating: summary numbers and the results that went into the report.



Screen 30. Sheet gallery of the generated drawing set.

Assistant



Screen 31. The assistant answering a one-click question about the hotel design, using the live design state.

C Reproducibility

```
streamlit run src/app.py                              # the app (four modes)
python src/run_pipeline.py                            # one pipeline run
python src/build_final_report.py                      # hotel drawing set + report PDF
python src/how_it_works_experiments.py                # experiments in Section 11
python src/capture_app.py                             # browser run + screenshots
python src/run_optimizer.py --method systematic       # one optimizer run on a QA site (--method ga for the GA)
python src/build_optimizer_study.py                   # optimizer comparison study (about 12 minutes)
python src/build_paper.py --no-figures                # this paper (optimizer section, code listings, PDF, Overleaf zip)
python -m pytest tests                                # 51 tests
```

Table 19: Where the code lives.

`src/rngd/orchestrator.py`	pipeline order, loop, floor plates
`src/rngd/agents/*.py`	the six LLM agents
`src/rngd/vectorstore.py`, `real_docs.py`	knowledge base and retrieval
`src/rngd/simulation/physics_engine.py`	physics engine
`src/rngd/vision/layout_qa.py`	vision QA
`src/rngd/ifc/`	IFC studio, real-IFC viewer, IBC cross-check
`src/rngd/hotel/`	hotel program, layout, code screen, drawings, details, interactive views
`src/rngd/optimize/`	optimizer: plates, decoder, fitness, search, decision, views
`src/app.py`, `src/hotel_ui.py`	the app
`src/hotel_opt_ui.py`, `src/optimizer_ui.py`	the two optimizer screens